A FUZZY ART MAP NEURAL NETWORK SPEECH RECOGNITION SYSTEM BASED ON FORMANT RATIOS

by

Captain D.P. Young, CD, B.Sc., rrmc

Canadian Forces

A Thesis

Presented to the School of Graduate Studies

in the

Department of Electrical and Computer Engineering

The Royal Military College of Canada

Kingston, Ontario

In partial fulfillment of the requirements for the degree

Master of Engineering

May 1995

(c) Copyright by D.P. Young 1995

This thesis may be used within the Department of National Defense but copyright for open publication remains the property of the author.

ABSTRACT

This thesis investigates the applicability of a Fuzzy ART (Adaptive Resonance Theory) Map ANN (Artificial Neural Net) to the task of ASR (Automatic Speech Recognition). The motivation for this work stems from two observations. First, from an Internet discussion group survey, it is evident that the Fuzzy ART Map ANN is being applied to tasks in a variety of disciplines. These disciplines include Computer Science, Geology, and Medical Science, as well as Mechanical, Civil and Software Engineering. However, a survey of the literature has turned up no references to the application of the Fuzzy ART Map ANN to the ASR task. Secondly, ASR is a vital component of speech understanding systems that promise to improve the ease with which humans can interact with computers. Spoken communication with computers will allow users to remotely access computer based services through standard telephone links. This would make every telephone a potential computer terminal. As well, many of the world telephones use PC (Pulse Code) signaling and hence more sophisticated services that require DTMF (Dual Tone Multi-Frequency) signaling are not yet available. It is for these reasons that the development of ASR over telephone channels is imperative.

This thesis provides some basic background information about ANNs, ART, fuzzy logic, ASR, and formant ratio theory. It then describes how a Fuzzy ART Map ANN is implemented as the recognizer component of an ASR system to learn and identify formant ratio vectors extracted from spoken phrases. The audio frequency analysis is limited to a 300 to 3300 Hz bandwidth so that the system may operate on verbal commands over telephone channels. The recognition accuracy of this system is then tested using a database with eight phonetically rich phrases. The database was recorded by twelve males and two females over a four month period using a high quality microphone in a computer room environment. The tests are repeated using a multi-layer feedforward backpropagation ANN as the recognizer component to provide a relative performance benchmark. Finally, the performance of the Fuzzy ART Map ASR system is tested over the telephone channel.

ACKNOWLEDGMENTS

I wish to express sincere appreciation to my thesis advisor, Dr. Paul E. Allard, for his guidance and constructive criticism.

I would also like to thank the following people who volunteered their time and efforts to help build the RMC Speech Database that was used extensively throughout this thesis:

Ann Hopkinson, Bernard Tremblay, Colin Wortley, Danielle Bossart, David Gravel, Don Hamilton, Ian Hawes, Jean Dolbec, Liam Porter, Larry Reeves, Murray Johnson, Matthew Oliver, and Waldemar Borys.

Finally, I would like to acknowledge the support of my family -- Donna, Cassandra, Kody and Keegan. They have made this effort possible.

VITA

Name:

David Peter Young

Born:

Sudbury, Ontario, 1963

Son of Norman and Helen Young

Education:

Primary and Secondary School in Sudbury, Ontario. Completed Grade 13 in 1982. Royal Roads Military College, Victoria, British Columbia, 1982-86. Awarded Bachelor of Science Degree in Computer Science and Physics in 1986.

Experience:

Aerospace Engineering Officer, Canadian Forces from 1986 to date including:

- Programmer/Analyst, Aurora Software Development Unit (ASDU), CFB Greenwood, Nova Scotia, 1986.
- Staff Officer Automated Data for Aerospace Maintenance (ADAM), Air Command HQ, Winnipeg, Manitoba, 1987-88.
- Directing Staff Canadian Forces Officer Candidate School (CFOCS), CFB Chilliwack, British Columbia, 1988-90.
- Aircraft Maintenance Control and Records Officer (AMCRO), 419 Tac F(T) Sqn, CFB Cold Lake, Alberta, 1990-93.
- Post Graduate Student, Royal Military College of Canada, Kingston, Ontario, 1993-95

TABLE OF CONTENTS

Title Page i

Abstract ii

Acknowledgments iii

Vita iv

Table of Contents v

List of Figures x

List of Tables xii

One Introduction

1.1 Introductory Comments 1

1.2 Thesis Outline 3

Two Neural Networks Overview

2.1 Introduction 5

2.2 ANN Programs 7

2.3 The Artificial Neuron Model 7

2.4 Differentiating Between ANNs 10

2.4.1 ANN Architecture 10

2.4.2 ANN Learning Process 10

2.4.3 ANN Recall Process 11

2.4.4 Major ANN Architectures 13

2.5 Backpropagation Learning in Multi-layer Feedforward ANNs 14

2.6 Chapter Summary 15

Three The Fuzzy Art Map ANN

3.1 Introduction to Adaptive Resonance Theory 16
3.2 The Origin of ART Development 17
3.3 Competitive Learning Models 19
3.4 Stability Issues 21
3.5 The ART Cognitive Process 21
3.6 When Does Adaptive Resonance Occur? 23
3.7 The ART Advantage 24
3.8 Fuzzy Logic 25
3.8.1 Parallels in Fuzzy Logic and ANNs 25
3.8.2 The Proposition 26
3.8.3 The Possibility Distribution Defined By Proposition 27
3.8.4 The Quantified Proposition 27
3.8.5 Fuzzy Logic Applied To Adaptive Resonance Theory 28
3.9 The Fuzzy ART ANN 29
3.10 An Eight Stage Fuzzy ART Algorithm 30
3.11 The Fuzzy ART Map ANN 35
3.12 Chapter Summary 38

Four Automatic Speech Recognition

4.1 Introduction 39
4.2 ASR System Classification 39
4.2.1 Speaker Dependency 40
4.2.2 Manner of Speech 40
4.2.3 Active Vocabulary Size 41
4.3 A Four Stage ASR Process 42
4.4 Applicable ASR Research Areas 44

4.4.1 Rapid Training 44
4.4.2 Robust Speech Recognition 44
4.4.2.1 The Acoustic Environment 45
4.4.2.2 The Microphone System 45
4.4.2.3 The Transmission Channel 46
4.4.2.4 The Variability of Speech 47
4.5 Human Speech Communication 49
4.5.1 Speech Production 49
4.5.2 Speech Perception 50
4.6 Formant Ratio Theory 51
4.7 Chapter Summary 55

Five A Fuzzy Art Map ASR System

5.1 Introduction 57
5.2 Stage #1 -- Speech Recording 58
5.3 Recording Method and Apparatus 60
5.4 Storage Method 62
5.5 Stage #2 -- Signal Processing 67
5.6 Stage #3 -- Feature Extraction 70
5.6.1 Noise Masking 70
5.6.2 Silence Reduction 71
5.6.3 Frequency Amplitude Normalization (Pre-emphasis) 71
5.6.4 Peak Picking 73
5.6.5 Formant Screening 73
5.7 Feature Representation 75
5.7.1 Tri Formant Vector (TFV) Representation 75
5.7.2 Bi Formant Ratio Vector (BFRV) Representation 77
5.7.3 Normalized Polar Angle (NPA) Representation 77
5.7.4 Tri Formant Ratio Vector (TFRV) Representation 79

5.8 Stage #4 -- Utterance Classification 79
5.8.1 Modifications to the Fuzzy ART Map ANN 83
5.8.2 Network Training 83
5.8.3 Information Retrieval 85
5.9 Chapter Summary 86

Six Experimental Results

6.1 Introduction 87
6.2 Testing Goals 87
6.2.1 Testing Control Measures 88
6.2.2 Testing Methods 88
6.3 Test Results 91
6.4 Evaluation Of Results 97
6.4.1 Benchmark ANN Performance Comparison 98
6.4.2 Speaker Dependency Performance Comparison 98
6.4.3 Transmission Channel Performance Comparison 98
6.4.4 Feature Representation Performance Comparison 99
6.4.5 Error Analysis 99
6.5 Chapter Summary 102

Seven Concluding Remarks

7.1 Thesis Summary 103
7.2 Conclusions 104
7.3 Future Work 105

References 106

Annexes

Annex A ASR System Hardware and Software

Annex B ASR Demo Program Code (*Stdemo Player 1.5*)

Annex C MAKESPEC.M Program Code (*Matlab 4.0*)

Annex D FORMANT3.M Program Code (*Matlab 4.0*)

Annex E FARTMAPT.M Program Code (*Matlab 4.0*)

Annex F BPMAPPER.M Program Code (*Matlab 4.0*)

Annex G BPTESTER.M Program Code (*Matlab 4.0*)

LIST OF FIGURES

Figure 2.1	Simplified Biological Neural Network	5
Figure 2.2	A Basic Artificial Neuron Model	9
Figure 2.3	Typical Neuron Activation Functions	9
Figure 2.4	ANN Architecture Nomenclature	12
Figure 3.1.A	Serial List Learning Error -- Experimental Results	18
Figure 3.1.B	Serial List Learning Error -- Classical Theory Prediction	18
Figure 3.2	Competitive Learning Model	20
Figure 3.3	The Fuzzy ART ANN Model	34
Figure 3.4	Fuzzy ART Map Architecture	37
Figure 4.1	A Four Stage ASR Process	43
Figure 4.2	A Speech Variability Pyramid	48
Figure 4.3	Vowel Formant Frequencies from Table 4.2	54
Figure 4.4	Vowel Formant Ratios from Table 4.2	54
Figure 4.5	Normalized Polar Angle Formant Ratios of Table 4.2 Vowels	55
Figure 5.1	RMC Speech Database File Naming Convention	63
Figure 5.2	ASR System Hardware Components	65
Figure 5.3-M	Microphone Speech Signal MDY12204.WAV	66
Figure 5.3-T	Telephone Speech Signal TDY12204.WAV	66
Figure 5.4-M	Chunk #1 of MDY12204.WAV	68
Figure 5.4-T	Chunk #1 of TDY12204.WAV	68
Figure 5.5-M	Chunk #2 of MDY12204.WAV	68
Figure 5.5-T	Chunk #2 of TDY12204.WAV	68
Figure 5.6-M	Chunk #3 of MDY12204.WAV	68
Figure 5.6-T	Chunk #3 of TDY12204.WAV	68
Figure 5.7-M	Spectrogram of MDY12204.WAV	69
Figure 5.7-T	Spectrogram of TDY12204.WAV	69
Figure 5.8-M	Microphone Noise Mask	69
Figure 5.8-T	Telephone Noise Mask	69
Figure 5.9-M	Noise Masking of MDY12204.WAV	69

Figure 5.9-T	Noise Masking of TDY12204.WAV	69
Figure 5.10-M	STE Curve for MDY12204.WAV	72
Figure 5.10-T	STE Curve for TDY12204.WAV	72
Figure 5.11-M	Silence Reduced MDY12204.WAV	72
Figure 5.11-T	Silence Reduced TDY12204.WAV	72
Figure 5.12-M	Octave Enhanced MDY12204.WAV	72
Figure 5.12-T	Octave Enhanced TDY12204.WAV	72
Figure 5-13-M	Local Maxima in MDY12204.WAV	74
Figure 5-13-T	Local Maxima in TDY12204.WAV	74
Figure 5.14-M	Formants in MDY12204.WAV	74
Figure 5.14-T	Formants in TDY12204.WAV	74
Figure 5.15-M	Formant Vectors in MDY12204.WAV	74
Figure 5.15-T	Formant Vectors in TDY12204.WAV	74
Figure 5.16-M	Tri Formant Vector Representation of MDY12204.WAV	76
Figure 5.16-T	Tri Formant Vector Representation of TDY12204.WAV	76
Figure 5.17-M	Bi Formant Ratio Vector Representation of MDY12204.WAV	78
Figure 5.17-T	Bi Formant Ratio Vector Representation of TDY12204.WAV	78
Figure 5.18-M	Tri Formant Ratio Vector Representation of MDY12204.WAV	80
Figure 5.18-T	Tri Formant Ratio Vector Representation of TDY12204.WAV	80
Figure 5.19-M	Fuzzy ART Map Recognition of MDY12204.WAV	81
Figure 5.19-T	Fuzzy ART Map Recognition of TDY12204.WAV	81
Figure 5.20	ASR System Software Components	82
Figure 5.21	Modified Fuzzy ART Map ANN	84
Figure 6.1	Benchmark ANN -- Sample Training Progress Record	101
Figure 6.2	Benchmark ANN -- Sample Training Error Summary	101

LIST OF TABLES

Table 2.1	Comparison of Conventional and Neural Network Computation	8
Table 2.2	Some Major ANNs Classified By Learning and Recall Modes	13
Table 3.1	ART Architecture Learning Properties	24
Table 3.2	Fuzzy Logic Compared To Binary Logic	25
Table 3.3	An Eight Stage Fuzzy ART Algorithm	33
Table 4.1	Performance Classification Of Some Commercial ASR Systems	41
Table 4.2	A Comparison of Vowels by Formant Frequencies and Formant Ratios.....	53
Table 5.1	RMC Speech Database Speaker Profiles	59
Table 5.2	RMC Speech Database Phrase List	60
Table 6.1	Benchmark ANN / BFRV / Microphone / Speaker Dependent	92
Table 6.2	Benchmark ANN / BFRV / Microphone / Speaker Independent	92
Table 6.3-M	Fuzzy ART Map / BFRV / Speaker Dependent / Microphone	93
Table 6.3-T	Fuzzy ART Map / BFRV / Speaker Dependent / Telephone	93
Table 6.4-M	Fuzzy ART Map / BFRV / Speaker Independent / Microphone	94
Table 6.4-T	Fuzzy ART Map / BFRV / Speaker Independent / Telephone	94
Table 6.5-M	Fuzzy ART Map / TFRV / Speaker Dependent / Microphone	95
Table 6.5-T	Fuzzy ART Map / TFRV / Speaker Dependent / Telephone	95
Table 6.6-M	Fuzzy ART Map / TFRV / Speaker Independent / Microphone	96
Table 6.6-T	Fuzzy ART Map / TFRV / Speaker Independent / Telephone	96
Table 6.7	ASR System Testing Summary	97

CHAPTER ONE

INTRODUCTION

1.1 Introductory Comments

The current interest in neural networks is fueled by the hope that this technology, based on a small model of the human brain, will be able to address classes of problems that are beyond the scope of today's computers. Tasks that are computationally complex for conventional digital computers, such as understanding auditory and visual stimulus, are more efficiently performed by the neural systems of biological organisms. The study of **ANNs (Artificial Neural Networks)** has produced many classes of network architectures in the pursuit of modeling the brain's abilities. A complete description of the various ANNs can be found in Jacek M. Zurada's ***Introduction to Artificial Neural Systems*** [Zur92]. This thesis applies a particular class of such networks developed by Stephen Grossberg et al., under the umbrella of **ART (Adaptive Resonance Theory)**. This theory confronts issues of unsupervised learning, self-organization, stability-plasticity and biological plausibility thus forming a solid foundation on which to construct models of human intelligence.

The Fuzzy ART Map ANN is the latest member of the ART class of networks. Its strengths make it a worthy candidate for further investigation concerning its practical applications. Discussions with various researchers by way of the Internet have revealed some of the applications of the Fuzzy ART Map currently being studied. Daehyon Kim (daehyon.kim@newcastle.ac.uk) is using a Fuzzy ART Map ANN in his research of 3D object recognition for his Ph.D. thesis in Computer Science. Simon Field (s.d.h.field@bnr.co.uk) at the Bell Northern Research neural networks group in Europe is studying Fuzzy ART Map's ability to analyze software complexity metrics in Software Engineering. Gerson Tontini (5mu7tontinig@vmsa.csd.mu.edu) is attempting pattern recognition in quality control charts using Fuzzy ART Map for his Ph.D. thesis in

Mechanical Engineering. Artur Dubrawski (artur.dubrawski@imag.fr) at the Adaptive Systems Lab of the Polish Academy of Sciences in Warsaw has applied Fuzzy ART Map to mobile robot navigation and the strength prediction of high performance concrete. Patrick Wong (patrick@research.petrol.unsw.edu.au) at the Center for Petroleum Engineering at the University of New South Wales in Australia has applied Fuzzy ART Map to rock classification in a petroleum reservoir based on electrical, sonic, and radioactivity measurements. Mehmet Akif Saglam (saglam@bmsr14.usc.edu) at the University of Southern California is applying a Fuzzy ART Map ANN to the function approximation of neurophysiological data collected from different parts of the brain. One can see that the Fuzzy ART Map ANN is being applied to tasks in a variety of disciplines including Computer Science, Geology, and Medical Science, as well as Mechanical, Civil and Software Engineering.

A survey of the literature has turned up no references to the application of the Fuzzy ART Map ANN to the task of **ASR (Automatic Speech Recognition)**. ASR is a vital component of speech understanding systems that promise to improve the ease with which humans can interact with computers. Spoken communication with computers will allow users to remotely access computer based services through standard telephone links, thus making every telephone a potential computer terminal. The demand for this technology is fueled by economic pressures and the rise of mobile cellular technology. As well, many of the world telephones use **PC (Pulse Code)** signaling and hence more sophisticated services that require **DTMF (Dual Tone Multi-Frequency)** signaling are not yet available. It is for these reasons that the development of ASR over telephone channels is imperative. The suitability of the Fuzzy ART Map ANN to the task of ASR is not known. This question could be answered by implementing an ASR system employing a Fuzzy ART Map ANN and testing its performance.

This thesis investigates the applicability of a Fuzzy ART Map ANN to the ASR task. An ASR system employing a Fuzzy ART Map ANN is implemented on a desktop computer using the Matlab programming environment. The audio frequency analysis is

limited to a 300 to 3300 Hz bandwidth so that the system may operate on verbal commands over telephone channels.

1.2 Thesis Outline

This thesis is organized into seven chapters. The first four chapters give a summary of the various fields involved in the subject of this thesis. This material provides the reader with the background information prerequisite to understanding the interdisciplinary discussion that follows in the last three chapters.

Following this introductory chapter, chapter two provides a general summary of neural networks. This chapter discusses the biological inspiration for ANNs by contrasting the manner in which computers and the human brain process information. ANNs are then described at both the individual neuron level and the network level. After a taxonomy of ANN architectures based on learning and recall characteristics, this chapter briefly describes the multi-layer feedforward backpropagation ANN.

Chapter three introduces ART and describes how it is combined with certain aspects of fuzzy logic to produce the Fuzzy ART Map ANN. A discussion of the historical development of the ART model includes a brief description of the competitive learning model from which it evolved. In keeping with its roots in the field of psychology, the mechanics of the ART model are then described with a four stage cognitive process. The merits of ART architectures are presented through a comparison of its learning properties with those of other learning schemes. This chapter then pauses to introduce fuzzy logic, its relationship to neural networks, and the role of the quantified proposition in the Fuzzy ART model. The Fuzzy ART learning process is then presented with an eight stage algorithm. Propositional calculus equations are included along with the text descriptions to cater to readers with a background in formal logic. Finally, the Fuzzy ART Map ANN is presented as a compound or hybrid network consisting of two tightly coupled Fuzzy ART ANNs.

Chapter four highlights the field of ASR including the taxonomy of ASR systems, the stages in the ASR process, and the research areas in which this thesis is involved. The production and perception of speech in humans are then briefly described. The chapter ends

with an overview of **FRT (Formant Ratio Theory)** which attempts to explain how humans perceive speech sounds in terms of the ratios of the resonant frequencies present. This background on FRT is important since speech formant ratios are used in the final ASR system to create temporal pattern vectors which the Fuzzy ART Map ANN component will be trained to recognize.

Chapter five details the implementation of a Fuzzy ART Map ANN as the recognizer component in an ASR system. The chapter is presented within the framework of the four stage ASR process introduced in chapter four. It starts with a description of the creation of the RMC Speech Database, which was used to tune and test the finished system, followed by an account of the recording method and apparatus used to capture the speech signal. Since this system was implemented using a common desktop personal computer, a section on the data storage techniques used to hold the speech database is included. The core of the ASR system is then described in the following three sections which cover the extraction, representation and classification of features present in the speech signal.

Chapter six describes the testing methods used to measure the recognition accuracy of the ASR system. Identical testing is carried out using another ANN in place of the Fuzzy ART Map ANN to provide a performance benchmark. A comparison of the training and testing performance of the two ANNs is discussed. Last, the performance of the Fuzzy ART Map ASR system is tested over a local telephone line. The same training and test sets were used throughout the testing phase to assess the relative effect of the changes made to the ASR system on its recognition accuracy.

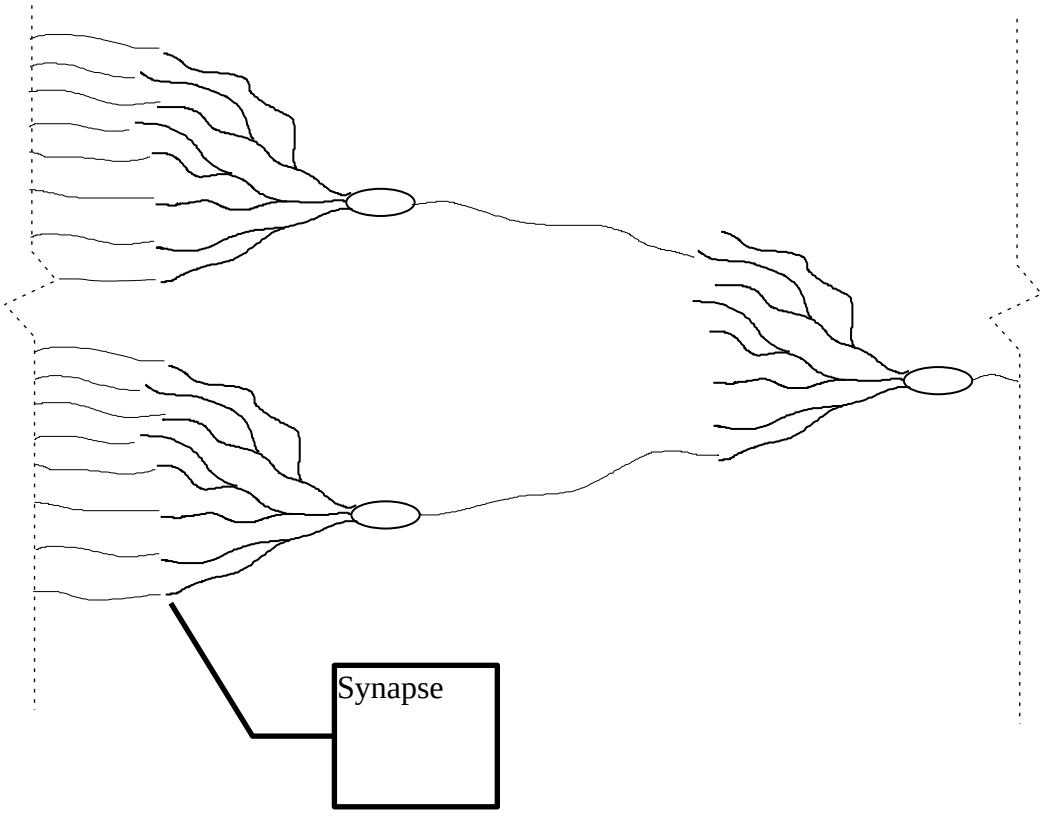
Last, chapter seven provides concluding remarks. A summary of the thesis work is presented along with the conclusions reached regarding the use of Fuzzy ART Map ANN in ASR systems. The chapter closes with some suggestions for future work.

CHAPTER TWO

NEURAL NETWORK OVERVIEW

2.1 Introduction

It may seem strange that while computers excel at tasks that we find mentally challenging (such as large number calculations) they in turn fail at tasks that we find the easiest to do. To understand this distinction of abilities between the human brain and computers we must examine how these two systems differ.



The human brain consists of highly interconnected network of billions of simple processors called neurons as shown in the sketch in Figure 2.1 above. The neuron processes information as relatively slow electro-chemical pulses transmitted between neurons over synaptic connections. Each neuron may have many input connections that enable it to accept or “see” the output pulses of many other neurons simultaneously. It is this pattern of input pulses that triggers a neuron to transmit a pulse of its own [Zur92]. All neurons process and communicate information concurrently and in parallel with one another. Also, the brain can adjust its field of view on incoming data as well as detect patterns within this data allowing it to excel at tasks involving pattern recognition.

In contrast, conventional computers are controlled by a single complex processor called a **CPU (central processing unit)** operating in a segmented manner**.** The CPU processes information as relatively fast electrical pulses transmitted between other system components by way of a single fixed data channel called a data bus. Although the CPU can process information at a very high rate, it must retrieve and store this information in a sequentially. The heart of the CPU’s power lies in its ability to compare two numbers very rapidly; however, it must handle all of its data one element at a time. This makes it very inefficient for handling large amounts of data simultaneously (as required for pattern recognition) but very efficient at performing sequential calculations.

By designing networks simulating the computational capabilities of the human brain, engineers hope to construct systems capable of performing complex tasks with comparable efficiency. Such systems have been dubbed **ANNs (artificial neural networks)** as they attempt to model the most basic structure and dynamics of the brain. To take advantage of the parallel processing capability of ANNs, they must be built as hardware systems. Software simulations of ANNs are also attractive because they run (albeit slower) on conventional computers and their flexibility allows easier experimentation. This discussion is limited to the implementation of ANNs as software systems and refers to them as ANN programs (covered in section 2.2).

It should be mentioned to the novice that the study of artificial neural systems has many aliases in the literature. Some synonyms are neural networks, neurocomputing, network computation, parallel distributed processing, connectionism, self-organizing networks, layered adaptive systems and neuromorphic systems.

2.2 ANN Programs

A major difference between computers and ANNs is the programming paradigm used. Conventional computer programs consist of a collection of instruction sequences that are assembled by a programmer before the execution of the program and remain unchanged as the program executes. In contrast, ANN programs are able to alter their processing structure in response to the system input by storing and updating "memories" with learning algorithms that employ various theories about basic brain dynamics. ANN programs model a densely interconnected network containing cells of processing elements and interconnecting links. The processing elements are referred to as **nodes** or **artificial neurons** since they mimic the function of real neurons in the brain. The interconnecting links are referred to as **weights** since their numerical values affect the signal strength between the processing elements much like the synaptic bonds between real neurons. ANN programs model the structure and dynamics of a network that combine to dictate how the program acquires, stores, and uses learned knowledge. Thus, the "programmed" ANN model determines **how** input is learned, while the interaction with the system environment determines **what** input is learned. Table 2.1 below compares conventional computation to that of neural networks.

2.3 The Artificial Neuron Model

The operation of an artificial neuron can be visualized as a four stage process illustrated in figure 2.2. **Stage one** involves the collection of input elements to the neuron. The number of elements that comprise the total input and the source of each element is determined by the design layout of the connections within the network. Typically, the

sources of inputs are the neural outputs of the preceding layer in the network, although the output from any neuron (including itself) in any layer may be used as an input source. **Stage two** involves adjusting the amplitude of each input element using a weighting function. This mimics the effect of the synaptic bond strength, which attenuates or amplifies signals transmitted between brain neurons. Typically the output of the weighting function is simply the product of the input element and a variable weight value. **Stage three** involves the summation of all the weighted input elements from stage two. This net value becomes the argument to the activation function used in **stage four** to determine the final output of the neuron. Typical activation functions are shown in figure 2.3. It is the non-linearity of the activation functions that give ANNs the ability of decision making [Zur92].

Task	**Conventional Computation**	**Neural Network Computation**
Problem Solving	Algorithm formulation	Selection of architecture and definition of the set of representative examples
Input Data	Numerical form	Numerical, but also perceptual representation allowed
Knowledge acquisition	Programming	Training
Knowledge retrieval	Sequential computation	Recall in the form of collective processing
Computation	High-precision arithmetic	Low precision non-linear mapping
Internal data	Internal representation in control of the algorithm	Internal representation in control of input data
Fixed or intermediate data storage	ROM, RAM high-precision binary memories	Interconnecting weights of typically continuous values

Incoming Axons from other Neurons

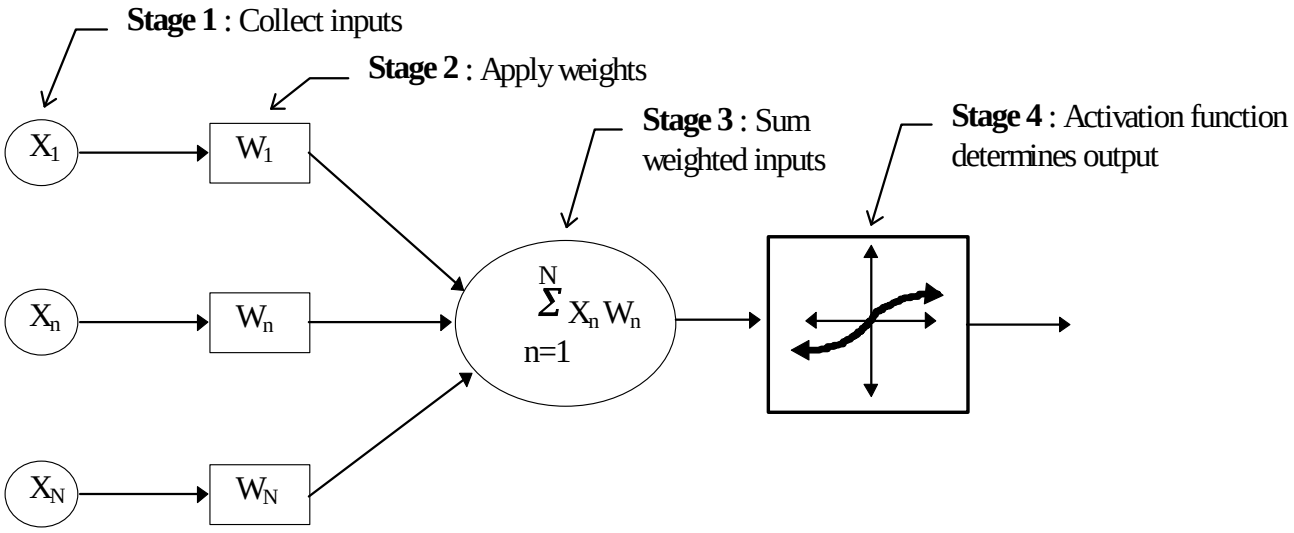


Fig 2.2 A Basic Artificial Neuron Model

Table 2.1 Comparison of Conventional and Neural Network Computation [Zur92]	Axon	Impulse	Neuron	**Fig 2.1 Simplified Biological Neural Network**
Dendrites				
Cell Body				

Unipolar

2.4 Differentiating Between ANNs

Different ANNs derive their unique computational abilities from the manner in which their artificial neurons are connected. Their distinctive structure is referred to as their **architecture** and it is the feature that characterizes them the most. ANNs also differ from one another in the manner in which they adapt their weights in response to input stimulus. Different learning rules control this **learning process** and govern how and when it occurs. The manner in which ANNs retrieve internally stored information is governed by their architecture and is described as their **recall process**.

2.4.1 ANN Architecture

Networks are typically organized into layers of neurons. The number of the layers classifies the ANN as single-layered, bi-layered, or multi-layered. The types of connective pathways between the neuron layers classify the ANN as feedforward, feedback or laterally connected. Figure 2.4 shows the naming conventions used to describe the various neuron layers and neuron connections.

2.4.2 ANN Learning Process

The behaviors of ANNs are determined by the values of the weights associated with the connections between their processing elements. The adaptation of ANN weights to produce a specific stimulus response behavior is herein referred to as the ANN learning process. The manner in which learning takes place is governed by learning rules that are said to perform in a supervised or unsupervised manner. **Supervised learning** is so denoted because the ANN adjusts its weights to produce a predefined output value (given by an external source) in response to a given input stimulus. **Unsupervised learning** does not provide a teacher’s solution and requires the ANN to discover it’s own relationships within the input data.

Supervised learning incorporates a set of training data in which network inputs are matched with the known desired outputs. The network weights are adjusted until the actual network output matches the desired network output within a desired error tolerance. The supervised adaptation of weights may be carried out one input/output pair at a time (**incremental training**), or it may be conducted using the complete training set in a single training step (**batch training**) without involving feedback information produced by the network itself.

Unsupervised learning combines the learning and recall modes. The network learns to recognize features in the input data by adjusting its weight values to produce a localized output in response to a given distribution of input patterns. This self-organizing capability holds great potential for producing a system capable of autonomously adapting to a task environment.

2.4.3 ANN Recall Process

When in recall mode, the network weights remain fixed and are not influenced by the system input. Previously viewed training input patterns produce the corresponding trained output pattern. Novel input patterns from a test data set produce an output that is a generalization of the training set.

The ANN recall process can be described as operating in one of two modes. The first is a **feedforward mode** in which the input stimulus is transformed by the connection weights as it is propagated forward across the network connections to produce an immediate output response. For a given input stimulus, the output response is temporally static and isolated, meaning that it does not change over time and it does not influence future network output. The second mode is a **recurrent mode** that is similar to the feedforward mode but uses feedback connections (see Fig 2.4) to augment the input stimulus. As a result the output response varies over time in response to the current input stimulus and the output response at

the previous time instant. The granularity of this time instant depends on whether the network operates in the **discrete or continuous time domain** during recurrent recall mode.

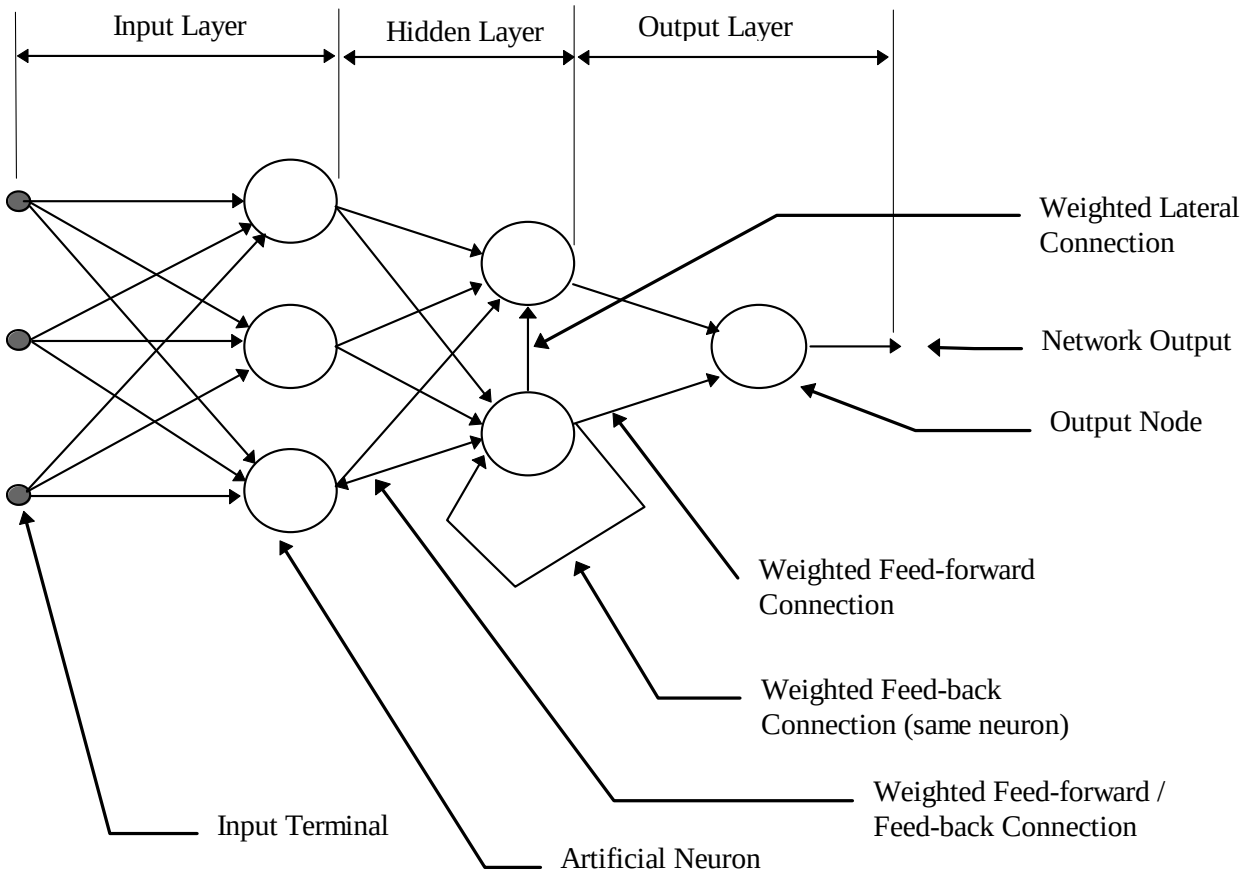


Fig 2.4 ANN Architecture Nomenclature

2.4.4 Major ANN Architectures

Table 2.2 lists some of the major types of ANN architectures classified by their modes of learning and recall. This list is not exhaustive but is presented here to give the reader some idea of the variety of ANNs that currently exist. The Adaptive Resonance Theory networks, at the bottom of the list, will be dealt with specifically in chapter three since their understanding is central to this thesis. The multilayer feedforward ANN will be used as a benchmark for performance comparisons in chapter six and is described in section 2.5 below.

Network Architecture	Learning Mode	Recall Mode	Recall Time Domain
Single-layer Network of Discrete and Continuous Perceptrons	Supervised	Feedforward	----
Multilayer Network of Discrete and Continuous Perceptrons (e.g. Backpropagation ANN)	Supervised	Feedforward	----
Gradient-type Network	Supervised	Recurrent	Continuous
Linear Associative Memory	Supervised	Feedforward	----
Autoassociative Memory	Supervised	Recurrent	Discrete or Continuous
Bi-directional Associative Memory	Supervised	Recurrent	Discrete or Continuous
Temporal Associative Memory	Supervised	Recurrent	Discrete or Continuous
Hamming Network	Supervised	Feedforward	
MAXNET	Supervised	Recurrent	Discrete or Continuous
Clustering Network	Unsupervised	Feedforward	----
Counterpropagation Network	Unsupervised / Supervised	Feedforward	----
Self-Organizing Neural Array	Unsupervised	Feedforward	----
Adaptive Resonance Theory Networks	Unsupervised / Supervised	Recurrent	Discrete or Continuous

Table 2.2 Some Major ANNs Classified By Learning and Recall Modes [Zur92]

2.5 Backpropagation Learning in Multi-layer Feedforward ANNs

Backpropagation is a supervised learning algorithm used to train multilayer feedforward ANNs. Although multi-layered ANNs have been around for more than 25 years, the lack of a successful training algorithm had limited their practical use. The development of the backpropagation algorithm has helped make multi-layer feedforward ANNs one of the most prevalent architectures in use today [Zur92]. The greatest strength of this ANN is that it can learn an input/output mapping of any complexity. Examples of tasks performed with this architecture include function approximation, handwritten character recognition, plant control, robot movement and trajectory generation, and speech recognition.

The backpropagation algorithm involves the repeated presentation of the training set and the incremental adjustment of all network weights between successive presentations. As each training vector is presented to the network inputs, the values of the network outputs are compared to the training vector's target response to produce an error signal. This signal is then used to make the appropriate adjustments to the network weights to reduce the total network error (the particular of the backpropagation algorithm can be found in [Zur92]). In this manner, the network is programmed to map the training set to the target set with a given degree of accuracy. The number of training cycles required to reach the desired mapping accuracy varies depending on several factors. These include the relationship between the training and target set, the number and composition of the network layers, and the starting weights chosen to initialize the network.

The process of training the network can be visualized as a search for the global minimum on a multidimensional error surface. The direction of travel along this error surface is determined by the error signal mentioned above, while the distance traveled with each iteration is set by the programmer. Ideally, this distance or learning rate should match the complexity of the error surface "terrain". A relatively large learning rate can jump over narrow valleys in mountainous terrain, while a relatively small learning rate will waste time slowly inching along relatively flat regions.

Training can be improved through the use of a momentum term and an adaptive learning rate. The momentum term prevents the learning algorithm from becoming trapped in shallow error mimima and thus improves its chances of discovering a solution that satisfies the desired accuracy goal. The adaptive learning rate varies the size of the incremental weight adjustments to match the rate at which the network output error is improving. For example, if the rate at which the network error is improving is positive , then the percentage of the error signal applied to amending the value of the weights is increased) . In chapter six we will see the implementation of these two learning aids in the ANN used to benchmark the performance of the Fuzzy ART Map ANN in the task of automatic speech recognition.

It should be noted that the mapping function performed by the backpropagation algorithm is based on the weight values of all network connections. This distributed method of knowledge storage contrasts the local method used by ART described in chapter three. Each method has certain advantages depending on the task being performed and will be discussed further in chapter six.

2.6 Chapter Summary

This chapter has given an overview of artificial neural networks (ANNs). They are "*... computational systems, either in hardware or software, which mimic the computational abilities of biological systems by using large numbers of simple, interconnected artificial neurons. Artificial neurons are simple emulations of biological neurons; they take in information from sensor(s) or other artificial neurons, perform very simple operations on this data, and pass the results on to other artificial neurons. Neural networks operate by having their many artificial neurons process data in this manner. They use logical parallelism (for all neurons in the same layer) combined with serial operations (as information in one layer is transferred to neurons in another layer).*" [Mar90]. They represent "*... collective, non-algorithmic, low-precision, non-linear computing machines that learn during training from examples and are data-controlled.*" [Zur92].

CHAPTER THREE

THE FUZZY ART MAP ANN

3.1 Introduction to Adaptive Resonance Theory

ART (**Adaptive Resonance Theory**) was developed by Stephen Grossberg at Boston University [Gro76]. ART is a theory about the learning dynamics of the brain which, loosely stated, says we learn new things in relation to things we already know. The use of analogies by teachers to help students relate to new concepts illustrates this learning dynamic. We pigeon-hole new information into our perceptual framework of experiences or past encounters with our environment. If the new piece of information does not fit anywhere, we expand our framework to accommodate it by broadening and redefining existing categories or creating new ones. More precisely, ART states that our learned expectations focus our attention upon sensory information in a way that protects **LTM (long term memory)** from being eroded by new learning, and enables new learning stored in **STM (short term memory)** to be automatically incorporated into LTM in a consistent manner.

ART provides a solution to the basic design problem of preserving the system's previously learned knowledge (stability) while ensuring it continues to learn new things (plasticity) constructively. This problem of competing design goals has been termed the **stability-plasticity dilemma** [Car88]. This dilemma must be successfully addressed in the design of any self-adapting system. Carpenter and Grossberg have used neural networks to implement various ART architectures (e.g., ART1 [Car87a], ART2 [Car87b], ART3 [Car90], Fuzzy ART [Car91], and Fuzzy ART Map [Car92]). Their latest architecture, the Fuzzy ART Map, combines fuzzy logic with ART neural networks by taking advantage of the similarity between the computations of fuzzy subsethood and ART category choice, resonance, and learning. It accepts both binary and fuzzy analog inputs, performs rapid learning while maintaining stability and plasticity, and it uses 100% of its memory capacity.

It is because of these attractive performance characteristics that the Fuzzy ART Map neural network architecture was chosen for further investigation in this thesis.

Before describing the Fuzzy ART Map network, the development of ART and the role of fuzzy logic is given. The diagrams, notation and vocabulary presented reflect the conventions used in the ART literature and reflect the terminology of the field psychology from which ART was spawned.

3.2 The Origin Of ART Development

In 1957, 19 year old Stephen Grossberg was a psychology undergraduate at Dartmouth College. His exposure to a great deal of structured data and classical theory about learning and behavior prompted him to reach the revelation that the meaning of these data centered in issues concerning the self-organization of individual behavior in response to environmental pressures [Gro82]. Grossberg then sought to mathematically describe a learning theory that unified what he considered to be various cases of some neurophysiological law. His blending of psychology, physiology and mathematics makes his theories interesting but not widely understood due to their inter-disciplinary nature.

Grossberg views his theories from the network perspective. He decided that the fundamental property of his developing theory would be the representation of behavioral units as nodes and the ability for his model to perform evolutionary aggregation of these units into new representations through experience. Grossberg makes the distinction that sensory activation is the input source and short term memory is the node's reaction to the input. The propagation of these STM signals to other nodes in the network takes place over directed pathways. LTM traces are a collection of variables that can discriminate which combination of STM signals should reach their target nodes based on past experience.

From his study of serial list learning a question arose. The figure 3.1.A represents the relationship between cumulative error and list position. It illustrates the relative difficulty of

recalling items from a list of items presented serially over time. The cumulative error curve in serial verbal learning is a skewed bowed curve. Items between the middle and end of the list are hardest to learn. Items at the beginning of the list are easiest to learn. However, if the position dependent difficulty of learning was all due to interference from previously presented items, the error curve would be monotone increasing as in figure 3.1.B. Because the error curve is bowed, and the degree of bowing depends on the length of the interval between successive list presentations**,** the nonoccurrence of list items after the last item is presented somehow improves learning across several prior list items**.**

Serial List Learning Error - A Function Of List Position [Gro82]

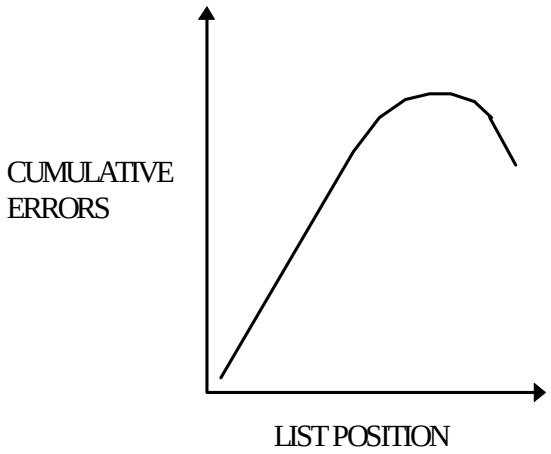


Fig 3.1.A Experimental Results

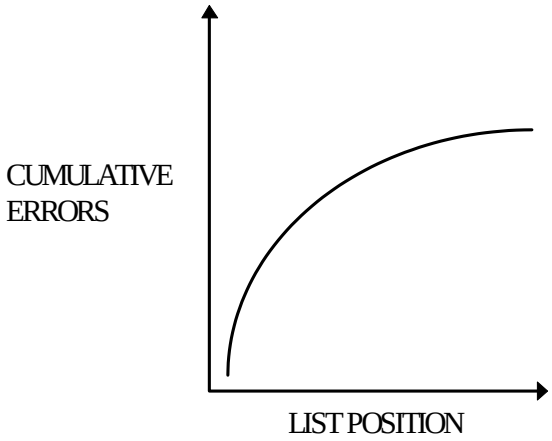


Fig 3.1.B Classical Theory Prediction

Grossberg felt that the backward effect that causes the bowed serial curve could only be explained using a real-time theory that could parameterize the temporal unfolding of events -- both the occurrences and nonoccurrences. The positive effect due to nonoccurring items lead him to believe that trace “images” of prior list occurrences must endure beyond the presentation time of the last item in the list, so that they could be influenced by the future nonoccurrence of items. This lead to the idea of STM traces, wherein each STM signal decays at a rate slower than the presentation rate of the sensory input. Thus as information items are serially presented to stimulate the network input nodes, a resulting STM trace captures the temporal pattern transformations of the input source. Grossberg believes that the key to developing successful learning theory is to think in terms of pattern transformations

(rather than local features) as the functional units that regulate the neural designs subserving behavioral adaptation.

3.3 Competitive Learning Models

ART models grew out of an analysis of competitive learning models[Car88]. Thus, we will describe the competitive learning model as a prelude to a discussion of the ART model. Refer to Figure 3.2 for the following description of the model dynamics:

- An M element input vector (***I***) is presented to an input field (F1) of M nodes and induces a normalized F1 STM input trace (***X***) across the entire input field given by equation 3.1.
- Adaptive weights (W_{mn}) or LTM traces connect pathways between the M nodes of the input field and the N nodes of the coding field (F2) and multiply the elements of the F1 STM input trace (***X***) as it is transmitted to F2. The total signal received at the nth node in F2 is given by equation 3.2 where m=[1..M].
- The F2 coding field is a competitive network capable of choosing the node that receives the largest input in a "winner-take-all" manner. The output of the winning node (Yn) is set to one while the output for all other nodes is set to zero. If two or more nodes of the same maximum value, the tie goes to the node with the lowest index. The F2 STM trace at the nth node in F2 is given by equation 3.3.
    - The F2 STM coding trace (***Y***) identifies the winning node and triggers associative pattern learning within the vector of LTM traces ($\mathbf{W_n}$) that abut the winning node. The learning rule for adapting the weights is given by equation 3.4 where m=[1..M], n=[1..N] and β is a scalar learning parameter.

The literature is rich with synonyms describing the function of the network described above. The F2 STM trace is said to ***code, cluster, classify, categorize, compress, partition, orthoganalize,*** or ***vector quantize*** the input vector presented to F1. If only one node in F2 is activated in response to the presentation of an input vector, then F2 is said to have made a

choice. In more general competitive learning models, compressed yet distributed F2 outputs are generated.

$$\boldsymbol{X = I \div |I|} \tag{3.1}$$

$$\mathbf{T_n = \sum X_m W_{mn}} \tag{3.2}$$

$$\mathbf{Y_n \equiv 1 \ \wedge \ Y_{i \neq n} \equiv 0 \ \leftrightarrow \ (\forall k \mid k \neq n : T_n \geq T_k) \wedge (\neg \exists i \mid T_i = T_n \wedge i < n)} \tag{3.3}$$

$$\mathbf{W_{mn} = \beta Y_n (X_m - W_{mn})} \tag{3.4}$$

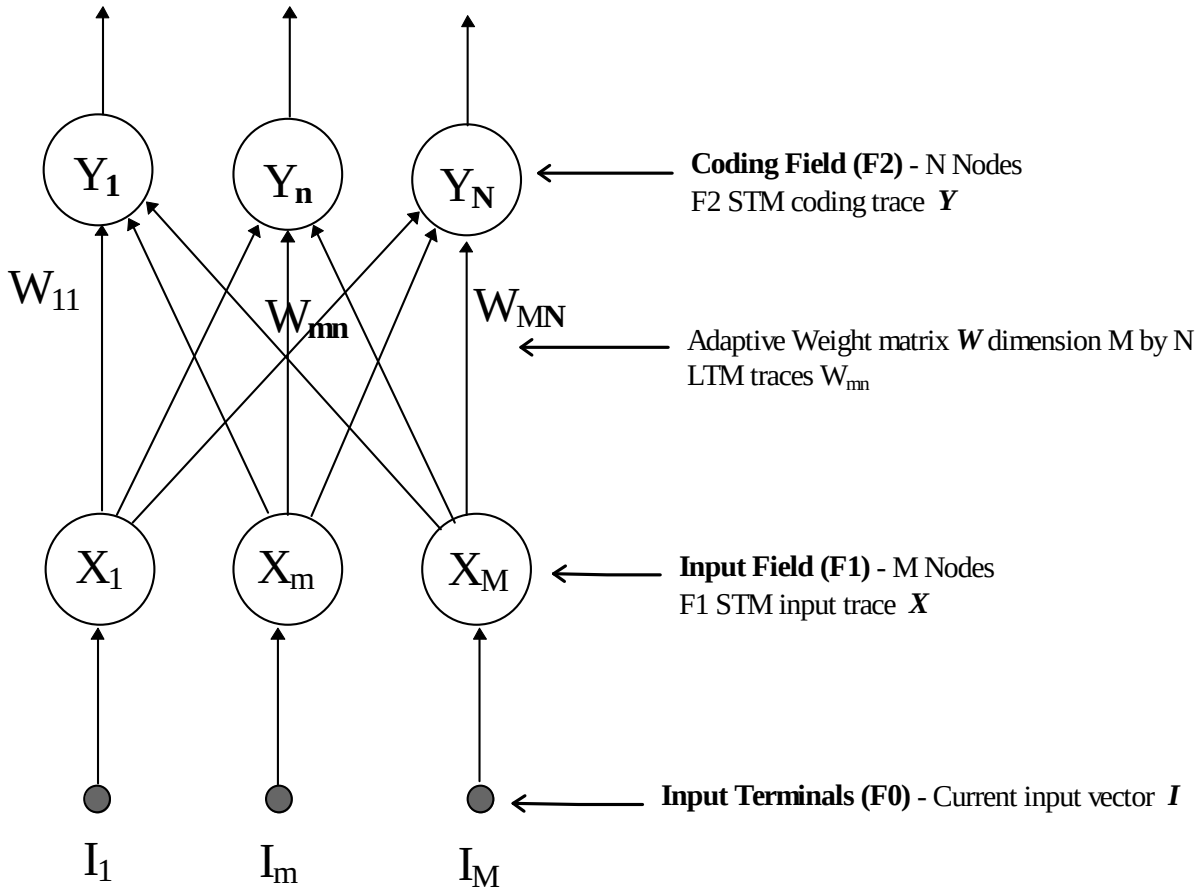


Fig 3.2 Competitive Learning Model

3.4 Stability Issues

Stability in competitive learning networks depends on the number of categories recognized by the network relative to its memory capacity or number of coding nodes in F2. If this capacity is not exceeded, then the learning of the recognition codes may stabilize

depending on the structure of the input environment. As a list of input patterns is presented to F1, the recognition code generated by F2 for a given pattern in the list may differ on each successive presentation of that pattern and may never stabilize. This is due to the adaptation of the LTM traces that occurs in response to other intervening inputs. As well, stable learning may suddenly become unstable due to simple changes in the input environment (i.e., the values of the input vectors and the order in which they are presented to the network). This washing away of prior learning by new learning is the stability-plasticity dilemma. Grossberg's ART model solves this dilemma.

3.5 The ART Cognitive Process

ART was introduced by Grossberg [Gro76] in 1976 to show how a competitive learning model could be made to perform autonomous recognition and learning of an arbitrary sequence of input patterns in a stable and efficient manner. The ART networks are similar to that shown in Fig 3.2 except they have a second top-down adaptive filter contained in feedback pathways from F2 to F1 (Fig 3.3 illustrates one example of an ART network). This allows the winning node in F2 to influence the coding of new patterns based on previous experience stored in the LTM. This top-down adaptive signal plays the role of learned expectations in an ART system. The process of adaptive pattern recognition in an ART system will now be described as a special case of the more general cognitive process of hypothesis selection, test, search, classification and learning:

Stage 1 -- Hypothesis Selection:

An input vector ***I*** activates input field F1 that reacts to produce the STM input trace vector ***X***. This signal from F1 activates the "hypothesis" category in F2. The F2 node receiving the largest total signal from F1 is selected as the candidate category for further testing.

Stage 2A -- Hypothesis Testing:

The candidate category selected in stage 1 is then tested when the output from F2 is sent back along adaptively controlled pathways to F1. The input vector ***I*** is then compared with the candidate category to produce a distance measure indicating the degree to which it matches previously learned expectations. A **vigilance parameter (ρ)** acts as a similarity threshold and determines the level of similarity required between the vector pair before a match is said to occur (a match with a higher vigilance requires a shorter distance between the two vectors). If a mismatch occurs, the hypothesis search continues in stage 2B.

Stage 2B -- Hypothesis Search:

The occurrence of a mismatch in stage 2A sustains a cycle of hypothesis selection and testing until a match is found or all candidate categories have been exhausted. Each mismatch event triggers an orienting subsystem to issue a reset burst to F2 to eliminate the mismatched category from further consideration in the hypothesis search by inhibiting the output of node that represents it. Thus after each iteration, the learned expectation output of F2 is conditioned to ignore prior mismatched categories. The cycle continues until classification occurs or the full memory capacity of the system is used.

Stage 3 -- Classification:

Classification of the input vector ***I*** occurs in one of two ways: either an F2 node is chosen whose top-down expectation approximately matches input vector ***I*** , or a previously uncommitted F2 node is selected to form a new category. The latter case is performed automatically after all committed nodes in F2 have been tested without a successful match.

Stage 4 -- Learning:

When classification occurs, the LTM trace that abuts the F2 category node is updated to incorporate the data from input vector ***I***. If the full capacity of the network has been exhausted and classification does not occur, learning is automatically inhibited.

3.6 When Does Adaptive Resonance Occur?

During the hypothesis search cycle the values of the STM traces change faster than the more slowly varying LTM traces can follow and essentially no learning occurs. When a match is found, the search cycle stops and the system becomes locked into a resonant state of STM activation, which is produced by the fusion of mutually reinforcing signal patterns from the bottom-up input vector and the top-down learned expectation feedback signal generated by the selected category in F2. This steady resonant state stabilizes the STM values to produce an attentional focus that allows the LTM traces to learn or “adapt” to new information represented in the fused activation pattern across F1. **The fact that learning only occurs in a state of resonance suggested the name adaptive resonance theory [Gro82].**

3.7 The ART Advantage

An ART architecture differs from other popular neural networks in a number of ways. Its main advantage is that it is designed to work well in an autonomous setting, learning in a quick yet stable manner in an ever changing environment while being exposed to an arbitrary sequence of input patterns, until its full memory capacity is utilized. Table 3.1, drawn from [Car88], lists the learning properties of ART architectures.

ART Architecture Learning Properties
Real-time (on-line) learning
Self-organizing (unsupervised)
Nonstationary world
Memory self-stabilizes in response to arbitrarily many inputs
Effective use of full memory capacity
Maintain plasticity in an unexpected world
Learn internal top-down expectations
Active attentional focus regulates learning
Slow or fast learning
Learn in approximate-match phase
Use self-regulating hypothesis testing to globally reorganize the energy landscape
Fast adaptive search for best match
Rapid direct access to codes of familiar events
Variable error criterion (vigilance parameter) sets coarseness of recognition code in response to environmental feedback
All properties scale to arbitrarily large system capacities

Table 3.1 ART Architecture Learning Properties

3.8 Fuzzy Logic

Before our description of the Fuzzy ART and Fuzzy ART Map ANNs in the next sections of this chapter, we pause to review that portion of fuzzy logic which plays a key role in the distance measures used by the these ANNs. The following introduces some basic concepts of fuzzy logic to the reader in order to appreciate its contribution. To start with, let us compare its characteristics with those of binary logic in table 3.2 below.

	Binary Logic	**Fuzzy Logic**
Truth Value	true or false	a fuzzy subset of a truth value set
Predicates	crisp (e.g. is even)	crisp or general (e.g. is tall)
Quantifiers	all, some	all, some, most, many, several, few, much of, frequently, occasionally, etc.
Predicate Modifiers	not	not, very, more or less, extremely, slightly, etc.
Qualification	truth value (e.g. is true) modal operator (e.g. is necessary) intentional operator (e.g. know that)	truth value (e.g. is not quite true) probability (e.g. is unlikely) possibility (e.g. is almost impossible)

Table 3.2 Fuzzy Logic Compared To Binary Logic

3.8.1 Parallels in Fuzzy Logic and ANNs

Fuzzy logic is concerned with the formal principles of approximate reasoning. Like ANNs, it aims at modeling the imprecise modes of reasoning that play an essential role in the remarkable human ability to make rational decisions in an environment of uncertainty and

imprecision. Fuzzy logic represents the meaning of a lexically imprecise proposition as an elastic constraint on a variable and deduces the answer to a query through a propagation of elastic constraints. Compared to ANNs, the proposition plays the role of artificial neuron that maps an input vector (a collection of variables) to an output vector (elastic constraint). The propagation of elastic constraints is analogous to the propagation of input values through the nodes of an ANN from input to output layer. A fuzzy logic system is typically implemented in software and consists of a fuzzy algorithm and control rules, much like an ANN program whose architecture and weight matrix define the relationship between its nodes.

A main concern to both fuzzy logic and neural networks is the development of the ability to generalize or infer an output based on accumulated experience. A branch of fuzzy logic, termed dispositional logic [Zad88], deals with propositions that are preponderantly but not necessarily true (e.g., snow is white). The main concern of dispositional logic lies in the development of rules of inference from commonsense knowledge. This is analogous to the development of ANN designs to improve their ability to generalize an output based on an interpolation from known information provided in a training set.

3.8.2 The Proposition

A proposition ***p*** may be viewed as a collection of elastic constraints, $C_1,...,C_k$, which restrict the values of a collection of variables $\boldsymbol{X} = [X_1,...,X_n]$ [Zad88]. The proposition defines a transfer function that maps $\boldsymbol{X}$ to conform to $\boldsymbol{C}$. This is analogous to the mapping performed by a single artificial neuron or a network of neurons. The ANN architecture enables it to learn certain types of mappings, but the propositional knowledge required to perform these mappings is stored in the weights. Training the network to produce desired outputs (elastic constraints) when presented with specific input vectors (collections of variables) is similar to the definition of a fact and rule base for a fuzzy logic program.

3.8.3 The Possibility Distribution Defined By Proposition

The proposition can be viewed as a definition of the possibility distribution of a given variable. Consider the canonical form of a proposition according to [Zad88] shown in equation 3.5. This infers that the possibility distribution of ***X*** is equal to ***C*** as in equation 3.6. This in turn infers equation 3.7, where μ_C is the membership function of ***C*** and **Poss{ *X* = u }** is the possibility that ***X*** may take **u** as its value.

$$\boldsymbol{p \rightarrow X \text{ is } C} \tag{3.5}$$

$$\boldsymbol{\Pi_X = C} \tag{3.6}$$

$$\mathbf{Poss\{ X = u \} = \mu_C(u),\ u \in U} \tag{3.7}$$

The proposition ***p*** constrains the possible values that ***X*** can take and thus defines its possibility distribution. In other words ***p*** applied to ***X*** produces ***C*** or $\mathbf{p \rightarrow X \text{ is } C}$, which takes us back to the original canonical form in equation 3.5 . The canonical form of a proposition is seen in the field 1 activity equation of the Fuzzy ART ANN shown in figure 3.3 and given by equation 3.8. Here the proposition $\boldsymbol{W_n}$ maps the variable ***I*** to the elastic constraint ***X.***

$$\min(\boldsymbol{W_n, I}) = \boldsymbol{X} \quad \text{which can be restated as} \quad \boldsymbol{W_n \rightarrow I \text{ is } X}. \tag{3.8}$$

3.8.4 The Quantified Proposition

A quantified proposition, $\boldsymbol{p \rightarrow}$ **proportion of A's that are B's is Q ,** infers the degree to which ***A*** is a fuzzy subset of ***B***. Take for example the quantified proposition "most tall men are thin". This proposition has a definition of the form $\boldsymbol{p \equiv}$ **Q A's are B's** , where Q = "most", A = "tall men" and B = "thin men". According to this proposition, the proportion of tall men that are thin men is the quantity called "most". Restated, this proposition asserts that the degree to which the set of tall men (***A***) is a fuzzy subset of the set of thin men (***B***) is defined by the quantity most (Q).

In fuzzy logic the **relative sigma count** $\sum$count(***B***/***A***) described in equation 3.9 is used to measure the relative population of ***B*** in ***A*** .This is equivalent to the proportion of A's that are B's or **the degree to which *A* is a fuzzy subset of *B*.**

$$\sum\text{count}(\boldsymbol{B}/\boldsymbol{A}) = \frac{\sum_j \min(\mu_A(u_j), \mu_B(u_j))}{\sum_j \mu_A(u_j)} \qquad \textbf{(3.9)}$$

where $j = [1, ...,m]$ and ***U*** is a finite set $\{u_1,...,u_m\}$

3.8.5 Fuzzy Logic Applied To Adaptive Resonance Theory

The relative sigma count function is seen in the Fuzzy ART ANN in both the choice function (equation 3.10) and match function (equation 3.16). Although formally introduced in the next sections, they are previewed here for convenience to appreciate their origin in fuzzy logic quantified propositions:

Choice Function T:

$$T_n(\boldsymbol{I}) \equiv \frac{|\min(\boldsymbol{I}, \boldsymbol{W_n})|}{\alpha + |\boldsymbol{W_n}|}$$

The proportion of $\boldsymbol{W_n}$ that is ***I***
The degree to which $\boldsymbol{W_n}$ is a fuzzy subset of ***I***
The relative population of ***I*** in $\boldsymbol{W_n}$

Match Function M:

$$M(\boldsymbol{I}, \boldsymbol{W_n}) = \frac{|\min(\boldsymbol{I}, \boldsymbol{W_n})|}{|\boldsymbol{I}|}$$

The proportion of ***I*** that is $\boldsymbol{W_n}$
The degree to which ***I*** is a fuzzy subset of $\boldsymbol{W_n}$
The relative population of $\boldsymbol{W_n}$ in ***I***

Note: norm operator $|\boldsymbol{P}| \equiv \text{sum}(\text{abs}(\boldsymbol{P}))|$

Comparing the quantified proposition, ***p* → proportion of A's that are B's is Q,** to the canonical form ***p* → *X* is *C***, one can see the constrained variable ***X*** is the relative sigma count $\sum$count(***B***/***A***) and Q is the elastic constraint to which ***X*** is subjected.

3.9 The Fuzzy ART ANN

This section marks the beginning of a detailed discussion of the mechanics of the ANN used in the construction of the ASR system detailed in chapter five. The information in the preceding chapters was presented to provide a general background to the origins of this ANN. The Fuzzy ART ANN presented here forms the key component of the Fuzzy ART Map ANN presented later in section 3.11.

The Fuzzy ART network [Car91] combines fuzzy logic with ART neural networks by taking advantage of the similarity between the computations of fuzzy subsethood and ART category choice, resonance, and learning. Its fuzzy input values range continuously from zero to one, thus it is capable of accepting analog as well as binary input. It is capable of performing rapid single presentation unsupervised learning of arbitrarily presented input patterns while maintaining its stability and plasticity. Just as other ART architectures, it can use 100% of its memory capacity.

The Fuzzy ART ANN model is shown in figure 3.3. (Note: the orienting subsystem and reset mechanism have been omitted here for clarity -- they can be seen in the Fuzzy ART Map model depicted in figure 3.4.) It contains three fields labeled F0, F1 and F2. The first field (F0) consists of M input terminals across which the current fuzzy valued input pattern $\boldsymbol{I}$ is presented. The second field (F1) consists of M artificial neurons with fuzzy activation functions. The third field (F2) consists of N artificial neurons that perform a category choice function. The nodes in F1 and F2 are fully inter-connected and an adaptive weight matrix $\boldsymbol{W}$ modifies signals transferred between the two fields. W_{mn} is subscript notation used in ART network literature referring to the weight connection between the mth F1 node and the nth node in F2. When the network is first initialized, $\boldsymbol{W}$ takes on the value of an M by N unity matrix.

3.10 An Eight Stage Fuzzy ART Algorithm

The learning algorithm for the Fuzzy ART ANN will now be described as an eight stage process. The reader is encouraged to refer to the network diagram in figure 3.3 for further clarification. As well, table 3.3 provides a bird's eye view of all eight stages to help the reader gain perspective of each stage in the following algorithm:

Stage One -- Initial Input:

The input signal from each F0 terminal (I_m) feeds directly to the F1 neuron opposite to it. The F1 neurons combine this input with the F2 output feedback to produce the F1 STM activity vector ***X***. On initial presentation of the input vector ***I*** when there is no feedback yet produced by F2 output, the value of ***X*** is equivalent to ***I***.

Stage Two -- Choice Function:

The input to the nth F2 node is defined by a **choice function $T_n(\ \boldsymbol{I}\ )$** defined in equation 3.10. The choice function provides a similarity measure describing the degree to which the nth LTM trace is a fuzzy subset of the input vector ***I***. The nth LTM trace refers to the weight vector ($\boldsymbol{W}_n$) that abuts the nth F2 node given by $\boldsymbol{W}_n = [\mathbf{W}_{1n}, \mathbf{W}_{2n}, \ldots \mathbf{W}_{mn}, \ldots \mathbf{W}_{M-1,n}, \mathbf{W}_{Mn}]$. Equation 3.10 introduces the reader to the positive non-zero **choice parameter** α . The **norm operator** is restated for convenience in equation 3.11.

$$T_n(\ \boldsymbol{I}\ ) \equiv \frac{|\ \min(\ \boldsymbol{I}, \boldsymbol{W}_n\ )\ |}{\alpha + |\ \boldsymbol{W}_n\ |} \qquad \text{Choice Function} \quad \textbf{(3.10)}$$

$$|\ \boldsymbol{Vector}\ | \equiv \text{sum}(\ \text{abs}(\ \boldsymbol{Vector}\ )\ ) \qquad \text{Norm Operator} \quad \textbf{(3.11)}$$

Stage Three -- Category Choice:

The system is said to make a **category choice** when the F2 neuron with the maximum input signal is chosen. If more than one neuron has maximal input, then the neuron with the smallest index is chosen. The conditions for category choice are given by equation 3.12.

$$T_{max} \equiv \max(T_n) \wedge (\neg \exists\, i \mid T_i = T_{max} \wedge i < n) \qquad \text{Category Choice} \quad \textbf{(3.12)}$$

Stage Four -- STM Vector Y:

The output of the chosen neuron is set equal to one and all other neuron outputs are set equal to zero. The output of F2 is the STM active category vector ***Y*** given by equation 3.13.

$$(Y_n \equiv 1) \wedge (Y_{i \neq n} \equiv 0) \leftrightarrow T_n = T_{max} \qquad \text{F2 STM Vector } Y \quad \textbf{(3.13)}$$

Stage Five -- STM Vector X:

The F2 output vector is sent back into F1 through the adaptive weight matrix ***W***. Since only $\mathbf{Y_n}$ is non-zero and is of unit value, the feedback vector from F2 to F1 is equal to $\boldsymbol{W_n}$ **.** Equation 3.14 describes the STM activity vector ***X*** output by F1 as the "fuzzy AND" or minimum of this top-down F2 feedback vector and the bottom-up F0 input vector ***I*** . It should be re-emphasized that upon initial exposure to the input vector ***I***, F2 is inactive and there is no feedback vector from F2 to F1. This initial condition is described by equation 3.15 and indicates the output from F1 is simply that of the input vector ***I*** .

$$\boldsymbol{X} \equiv \min(\boldsymbol{I}, \boldsymbol{W_n}) \leftrightarrow Y_n = 1 \qquad \text{F1 STM Vector } X \quad \textbf{(3.14)}$$

$$\boldsymbol{X} \equiv \boldsymbol{I} \leftrightarrow \neg \exists\, i \mid Y_i = 1 \qquad \text{X when F2 Inactive} \quad \textbf{(3.15)}$$

Stage Six -- Match Function:

Next the match function given by equation 3.16 provides a similarity measure of the degree to which the input vector ***I*** is a fuzzy subset of the F2 feedback vector. (Note: this feedback vector is equal to $\boldsymbol{W_n}$ and represents the centroid of the cluster chosen by the category choice.)

$$M(\boldsymbol{I}, \boldsymbol{W_n}) = \mid \min(\boldsymbol{I}, \boldsymbol{W_n}) \mid / \mid \boldsymbol{I} \mid \qquad \text{Match Function M} \quad \textbf{(3.16)}$$

Stage Seven -- Mismatch Condition:

If the match value is less than the vigilance parameter , then the match does not support resonance and a mismatch condition defined by equation 3.17 results. A mismatch triggers an orienting subsystem to issue a **reset** burst to F2 to eliminate the mismatched category from further consideration in the hypothesis search by inhibiting the output of the node that represents it. Equation 3.18 shows that the category function for the node is reset to zero. After this, the learning cycle returns to the hypothesis search by the selection of a new category choice.

$$M(\boldsymbol{I}, \boldsymbol{W_n}) < \rho \qquad \text{Mismatch Condition} \quad \textbf{(3.17)}$$

$$T_n(\boldsymbol{I}) \equiv 0 \qquad \text{Reset Action} \quad \textbf{(3.18)}$$

Stage Eight -- Resonance Condition:

If the match value is greater than or equal to the preset fuzzy valued **vigilance parameter ρ,** then a resonance condition defined by equation 3.19 exists. Resonance stabilizes the F1 output long enough to allow learning to occur through the adjustment of the adaptive weight vector $\boldsymbol{W_n}$ according to the Fuzzy ART learning rule given by equation 3.20 which includes the fuzzy valued **learning parameter β.**

$$M(\boldsymbol{I}, \boldsymbol{W_n}) \geq \rho \qquad \text{Resonance Condition} \quad \textbf{(3.19)}$$

$$\boldsymbol{W_n}^{new} \equiv \beta(\min(\boldsymbol{I}, \boldsymbol{W_n})) + (1-\beta)\boldsymbol{W_n}^{old} \qquad \text{Fuzzy ART Learning Rule} \quad \textbf{(3.20)}$$

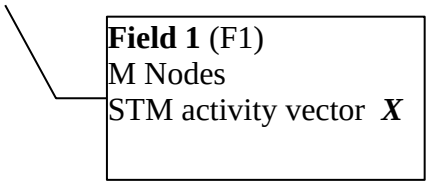
Summary Chart

This ends the Fuzzy ART ANN learning algorithm. A condensed summary of the eight stage process outlined above is provided in table 3.3. This algorithm is important because it is employed in the Fuzzy ART Map ANN discussed in the next section.

Stage	Action	Applicable Equation
1	Input vector I presented at F0 and	

	initial F1 output X is calculated	$\boldsymbol{X} = \boldsymbol{I}$
2	Choice function determines input to F2	$T_n(\boldsymbol{I}) \equiv \frac{\lvert \min(\boldsymbol{I}, \boldsymbol{W_n}) \rvert}{\alpha + \lvert \boldsymbol{W_n} \rvert}$
3	Category choice is made	$T_{max} \equiv \max(T_n) \wedge (\neg\exists\, i \mid T_i = T_{max} \wedge i < n)$
4	F2 output Y is determined	$(Y_n \equiv 1) \wedge (Y_{i \neq n} \equiv 0) \leftrightarrow T_n = T_{max}$
5	F1 output X recalculated	$\boldsymbol{X} = \min(\boldsymbol{I}, \boldsymbol{W_n})$
6	Match function calculated to determine reset or resonance condition	$M(\boldsymbol{I}, \boldsymbol{W_n}) = \lvert \min(\boldsymbol{I}, \boldsymbol{W_n}) \rvert / \lvert \boldsymbol{I} \rvert = \lvert \boldsymbol{X} \rvert / \lvert \boldsymbol{I} \rvert$
7	If mismatch condition, then reset action and go to category choice (stage 3 above).	$M(\boldsymbol{I}, \boldsymbol{W_n}) < \rho \rightarrow T_n(\boldsymbol{I}) \equiv 0$
8	If resonance condition, then apply learning rule	$M(\boldsymbol{I}, \boldsymbol{W_n}) \geq \rho \rightarrow \boldsymbol{W_n}^{new} \equiv \beta(\min(\boldsymbol{I}, \boldsymbol{W_n})) + (1-\beta)\boldsymbol{W_n}^{old}$

Table 3.3 An Eight Stage Fuzzy ART Algorithm


Field 1 (F1)
M Nodes
STM activity vector ***X***

3.11 The Fuzzy ART Map ANN

The Fuzzy ART Map neural network is a network which consists of two Fuzzy ART sub-networks or modules linked by an associative learning network called a Map Field. Figure 3.4 illustrates the Fuzzy ART Map architecture as described in [Car92].

During supervised learning, the ART_A module receives a stream of **training** input vectors $\boldsymbol{a}$, and the ART_B module receives a stream of **target** input vectors $\boldsymbol{b}$. The Fuzzy ART_A module's complement coding preprocesssor transforms the vector $\boldsymbol{a}$ to $\boldsymbol{A}=(\boldsymbol{a},1-\boldsymbol{a})$ at field $F0_A$. The complement coded vector $\boldsymbol{A}$, now twice the dimension of $\boldsymbol{a}$, is the input vector to field $F1_A$. Similarly, the input to field $F1_B$ is the complement coded vector $\boldsymbol{B}=(\boldsymbol{b},1-\boldsymbol{b})$. Both ART_A and ART_B modules activate an output category node in $F2_A$ and $F2_B$ (respectively) in the manner described in section 3.10 above.

The map field F_{AB} is activated whenever one of the ART_A or ART_B categories is active. If node J of $F2_A$ is chosen, then its weights $\boldsymbol{W}j_{AB}$ activate F_{AB}. If node K in $F2_B$ is active, then the node K in F_{AB} is activated by one to one pathways between $F2_B$ and F_{AB}. If both ART_A and ART_B are active, then F_{AB} becomes active only if ART_A predicts the same category as ART_B via the weights $\boldsymbol{W}j_{AB}$. The F_{AB} output vector $\boldsymbol{X}_{AB}$ obeys the following equations:

- if the Jth $F2_A$ node is active and $F2_B$ is active then ..... $\boldsymbol{X}_{AB} = \boldsymbol{Y}_B \wedge \boldsymbol{W}j_{AB}$ (3.21)
- if the Jth $F2_A$ node is active and $F2_B$ is inactive then .... $\boldsymbol{X}_{AB} = \boldsymbol{W}j_{AB}$ (3.22)
- if $F2_A$ is inactive and $F2_B$ is active then ..... $\boldsymbol{X}_{AB} = \boldsymbol{Y}_B$ (3.23)
- if $F2_A$ is inactive and $F2_B$ is inactive then ..... $\boldsymbol{X}_{AB} = 0$ (3.24)

By equation 3.21, $X_{AB} = 0$ if the category selected by ART_A does not match that selected by ART_B. This inhibition of the map field F_{AB} activation induces the **match tracking** process. Match tracking raises the Fuzzy ART_A vigilance (ρ_A) just above the $F1_A$ to $F0_A$ match ratio $|\boldsymbol{X}_A| \div |\boldsymbol{A}|$. This triggers a Fuzzy ART_A search that leads to activation of either a Fuzzy ART_A

category that correctly predicts target input vector $\boldsymbol{b}$ or to a previously uncommitted Fuzzy ART_A category node.

This supervised training method allows each output node of the ART_A network to be assigned a label (by the map field) indicating which class or parent category it represents. When the ART_A network trains on an input vector, it first attempts to recognize it. If it cannot match the input vector to the target output class provided by the ART_B network, then the ART_A network memory is updated. The ART_A network is also updated if it did classify the input vector correctly, but the similarity of the match was below a fixed baseline similarity. This learning method allows the network to discriminate between input vectors which represent new data and those which contain data already learned. As well, the mapping of the output nodes of the ART_A network into different class groups allows the network to be trained to recognize patterns that are not linearly separable, as demonstrated by the two-spiral test in [Car92]. The Fuzzy ART Map network can perform this mapping in one presentation of the training set. These qualities that make this network an attractive candidate for adaptive pattern recognition within a large abstract information database.

Chapter five will describe the implementation of the Fuzzy ART Map architecture in a software ANN program. Since software simulations are not bound by the same physical limits as the systems they simulate, many of the structures shown in figure 3.4 are replaced by functional equivalents to simplify the program design.

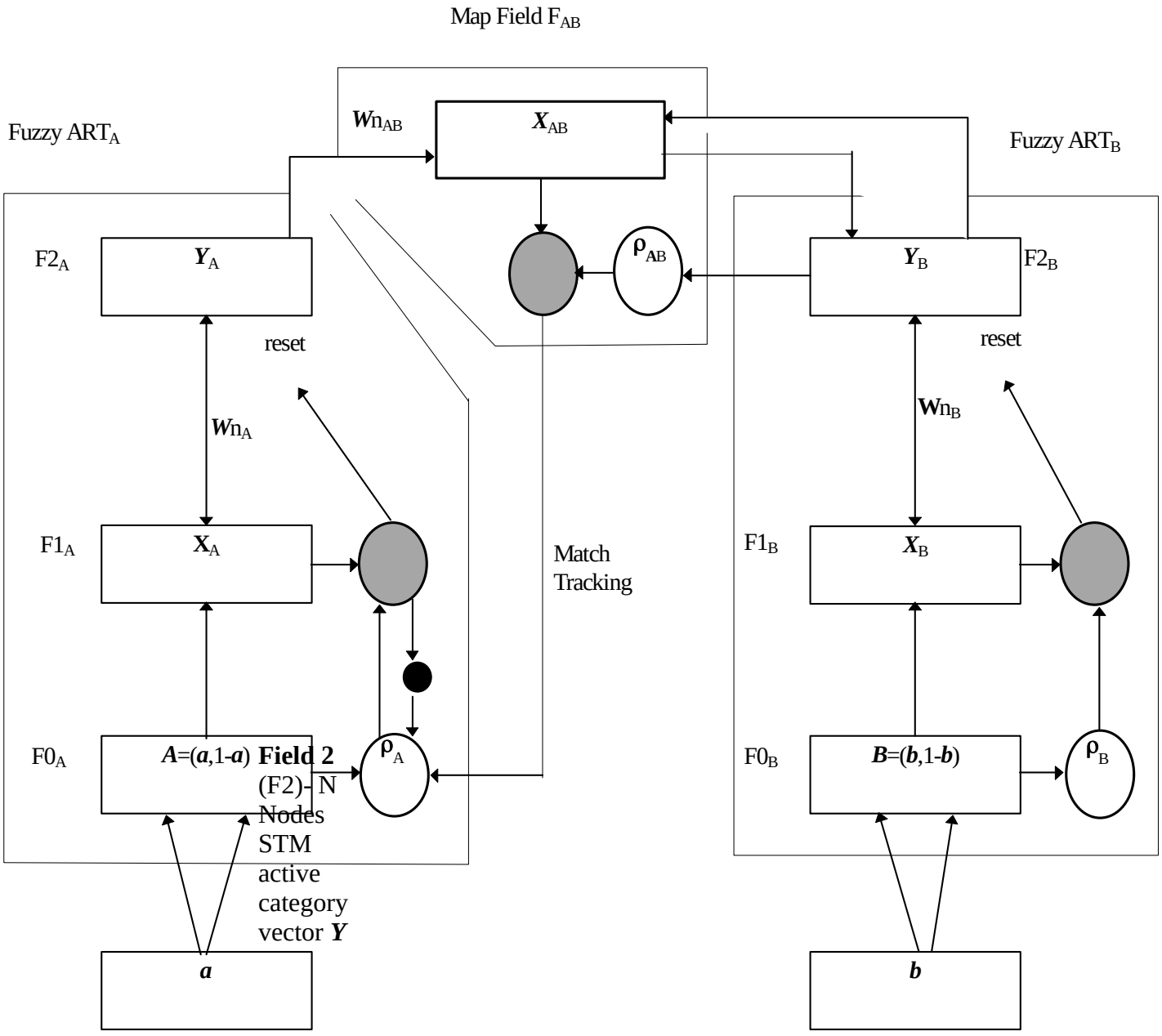


Figure 3.4 Fuzzy ARTMAP architecture [Car92]

3.12 Chapter Summary

This chapter presented three main thrusts -- an overview of Adaptive Resonance Theory, an introduction to Fuzzy Logic, and a description of two types of *Fuzzy* ART ANNs:

- The origin of ART was discussed with respect to its growth from the study of serial list learning error, competitive learning models, and stability issues. In keeping with its roots in the field of psychology, the ART algorithm was described in terms of the general cognitive process of hypothesis selection, test, search, selection, classification, and learning. An explanation of how the theory attained its name and a comparison of ART architectures and other learning schemes was made.

- Fuzzy Logic was introduced through comparisons to binary logic and analogies to Artificial Neural Networks. The raison d'être for this section was to introduce the reader to the concept of the quantified proposition and its role in the "relative sigma count function" distance measure used in Fuzzy ART ANNs.

- The Fuzzy ART ANN learning algorithm was presented which incorporated both the ART learning process and the Fuzzy Logic distance measure. For the sake of clarity, the algorithm was detailed in eight stages using both text and mathematical descriptions. The Fuzzy ART Map ANN was then described as an architecture formed by two communicating Fuzzy ART ANN sub-networks.

This chapter is important to the reader's appreciation of the role that the Fuzzy ART Map ANN plays as the recognizer component in the ASR system described in chapter five. By understanding how the Fuzzy ART Map ANN works, the reader will recognize what portion of the success (or failure) of the ASR system is attributable to the ANN recognizer component. The feature extraction process is as important to the successful performance of an ASR system -- this aspect is covered next in chapter four.

CHAPTER FOUR

AUTOMATIC SPEECH RECOGNITION

4.1 Introduction

Automatic speech recognition refers to the use of a machine to perform the unambiguous identification of linguistic elements present in an utterance of spoken human language. ASR is the first step in a spoken language system that recognizes a person's words, interprets their meaning and provides an appropriate response. Research concerning spoken language systems is important since it will enable people to interact with computers using speech, thus making it easier for people without computer training to access computer-based resources.

This chapter presents material in four main areas. First, it gives a background to ASR systems by discussing their classification and general operating process. Next, some of the ASR research concerns affecting the design of ASR systems are presented. Third, the production and perception of human speech are examined to provide a basic vocabulary of the subject to the reader. The last section in the chapter introduces Formant Ratio Theory, which forms the basis for the signal processing methods used in the ASR system implemented in chapter five.

4.2 ASR System Classification

ASR systems can be classified by their performance regarding the number of speakers they support (speaker dependency), the manner of speech that they accept (discrete or continuous), and the size of their active vocabulary. The following sub-sections describe each

of these categories in more detail and table 4.1 illustrates this classification scheme applied to some of today's commercially available ASR systems:

4.2.1 Speaker Dependency

A speaker dependent system is developed to operate for a single speaker. These systems are usually easier to develop, cheaper to buy and more accurate, but not as flexible as speaker adaptive or speaker independent systems. A speaker independent system is developed to operate for any speaker of a particular type (e.g., American English). These systems are the most difficult to develop, most expensive to buy and accuracy is lower than speaker dependent systems; however, they are more flexible. A speaker adaptive system is developed to adapt its operation to the characteristics of new speakers. Its development effort lies somewhere between speaker independent and speaker dependent systems [CSN95].

4.2.2 Manner of Speech

A discrete speech system operates on a single word at a time and requires a pause between the utterance of each word in a phrase. This is the simplest form of recognition to perform because the ends of a word are easier to find and the pronunciation of a word tends not affect others. Thus, because the occurrences of words are more consistent, they are easier to recognize. A continuous speech system operates on speech in which words are connected (i.e., not separated by a pause). Continuous speech is more difficult to handle because of a variety of effects. First, it is difficult to detect the word boundaries. Another problem is "coarticulation". The production of each phoneme is affected by the production of surrounding phonemes, and similarly the beginning and ending of each word are affected by the words that precede and follow it. The recognition of continuous speech is also affected by the rate of speech, where recognition accuracy tends to decrease as the rate of speech increases.

4.2.3 Active Vocabulary Size

The number of words that a system can recognize at a given instant is referred to as its active vocabulary size. The size of the active vocabulary of a speech recognition system affects the complexity, processing requirements and the accuracy of the system. System accuracy can be adversely influenced if some of the words within the active vocabulary are acoustically similar. This condition becomes more likely as the size of the active vocabulary grows. The recognition of discrete phrases can be used to increase system recognition accuracy since the acoustic dissimilarity between phrases is generally greater than that of words alone. Some ASR systems use **context switching** (fast loading and unloading of active vocabulary modules) to provide a larger virtual active vocabulary. This technique is analogous to a computer operating system's use of hard drive swap space to increase the apparent size RAM available to programs. The size of an ASR system's active vocabulary varies with the task it is required to do; some applications only require a few words (e.g., numbers only) while others require very large dictionaries (e.g., dictation machines).

Product	***Speaker Type***	***Speech Type***	***Active Vocabulary***
Voice Blaster 4.0	dependent	discrete	1024 words
Votan	dependent	discrete or continuous	255 words
VoiceServer	dependent	discrete	1000 words
DragonDictate 3.0	adaptive	discrete	60,000 words
DragonVoiceTools	adaptive	discrete	1,000 words
Kurzweil Voice	adaptive	discrete	60,000 words
PowerSecretary	adaptive	discrete	30,000 words
VoiceType Dictation	independent	discrete	22,000 words
Phonetic Engine 400	independent	continuous	2,000 words
Visus SpeechKit	independent	continuous	1,000 words
DATAVOX -- French	independent	continuous	50 words
ICSS system from IBM	independent	continuous	1,000 words

Table 4.1 Performance Classification Of Some Commercial ASR Systems [CSN95]

4.3 A Four Stage ASR Process

ASR can be described in terms of a four stage process consisting of speech recording, signal processing, feature extraction and utterance classification [Whi76]:

- The first stage is the sound recording stage and deals with the capture of the speech sample. This stage concentrates on the efficient reception and conversion of the acoustic signal from the speaker to an electronic equivalent that is then stored for further processing.

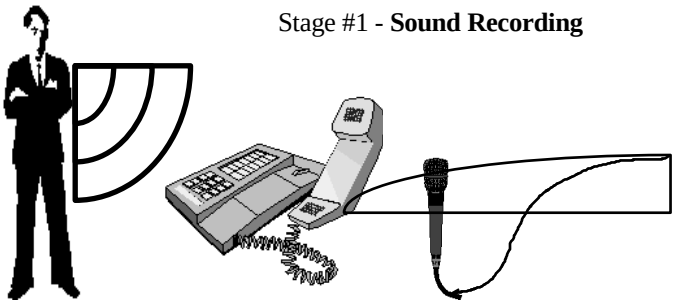
- The next stage is the signal processing stage and is concerned with spectral analysis. This stage concentrates on the initial analysis of the electronic speech signal to produce a parametric representation that encodes spectral information either explicitly or implicitly [Whi76]. This stage can also be referred to as the "front end" or preprocessing stage.

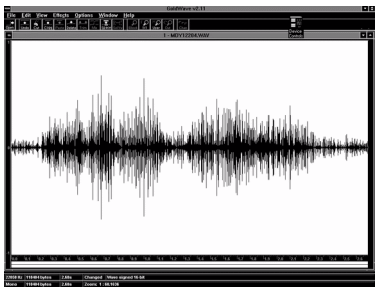
- The third stage is the feature extraction stage and is concerned with the removal of redundant or irrelevant information from the parametric representation produced in stage two. This results in data compression and reduced computational load for all subsequent processing. One example of feature extraction is formant tracking, which is the monitoring of the time evolution of the major peaks of the short-term power spectrum of speech. The formants are produced by resonance in the vocal cavity of the speaker. They provide excellent data compression since *most* of the information content of speech is carried by the time evolution of the first three formants or resonance frequencies [Whi76].

- The last stage is the utterance classification stage and is concerned with identifying what has been said. This stage involves the recognition of phonemes, groups of phonemes, words or phrases, aided by many processes such as DTW (Dynamic Time Warping), HMM (hidden Markov modeling), or ANNs. (Note: more information on

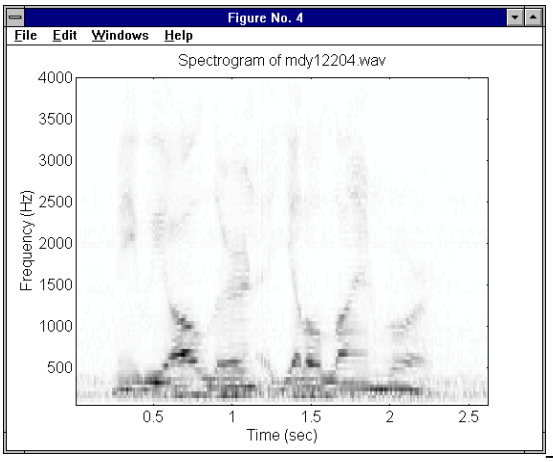
these techniques can be found in [Ain88] and [Fur89].) ASR systems may also use some knowledge of the language to aid the recognition process.

Stage #1 - **Sound Recording**

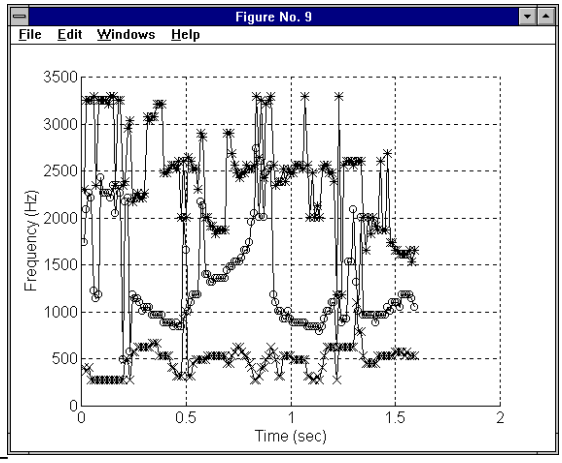




Stage #2 - **Signal Processing**



Stage #3 - **Feature Extraction**



Stage #4 - **Utterance Classification**

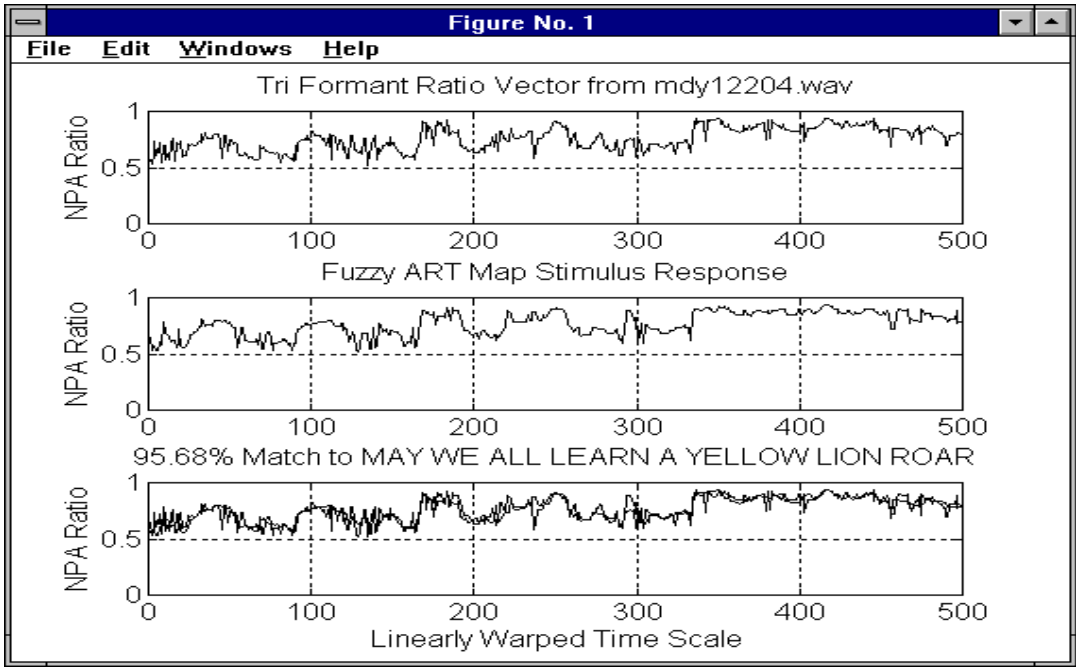


Figure 4.1 A Four Stage ASR Process

4.4 Applicable ASR Research Areas

The ASR system developed as part of this thesis work addresses issues related to two key areas of basic research in spoken language systems as described in [Col95]. These areas of concern include increasing the speed and ease of training ASR systems, and improving the robustness of speech recognition performance when systems are employed in realistic acoustic environments. Both of these problems are described in more detail below:

4.4.1 Rapid Training

Currently, ASR systems must learn large amounts of task-specific data for reasonable performance in a given application. This activity is expensive due to the time required to complete the task and the caliber of personnel required to do the job. Until researchers develop faster and less labor intensive methods of adapting ASR systems to new applications, ASR systems will only be created for those few applications whose high return will offset the system's training costs.

An ASR system capable of rapidly learning new speech data, in a manner independent of speaker type or spoken language, is highly desirable. Ideally, this new speech data should be stored in a modular manner such that subsets of domain knowledge could be combined into packages customized for new applications, without the need for extensive system retraining. These design goals were incorporated in the implementation of the ASR system described in chapter five.

4.4.2 Robust Speech Recognition

Robustness in speech recognition can be defined as **minimal performance degradation due to changes in the input conditions of the acoustic signal** [Col95]. The input conditions of the acoustic signal are the result of factors related to the acoustic

environment, the microphone system, the transmission system, and the variability of human speech. Each of these areas of concern is discussed below with respect to its impact on the design considerations of future ASR systems.

4.4.2.1 The Acoustic Environment

The acoustic environment of practical ASR applications should include those environments in which computers are currently operated today. An ASR system that cannot function in the office environment where a computer is normally located is less than ideal. Thus an ASR system must be able to effectively deal with ambient noise (unwanted signal originating outside the ASR system). Enclosures that isolate the speaker and the ASR system in a controlled acoustical environment have limited applications due to the restricted mobility they impose on the speaker. An example of the appropriate use of an enclosure is the Texas Instruments access control system, described in [Dod76], in which an enclosed 4'x5' foyer is used to improve the acoustic environment as well as imprison persons trying to gain unauthorized access to the computer room.

4.4.2.2 The Microphone System

The microphone system is of primary importance to the effective capture of the speech signal since it acts as the ears of the system. Ideally, speakers should not be restricted by having to use a head-mounted or hand-held microphone when communicating with a machine. Thus it is highly desirable to have a remote microphone attached to the ASR system. An ideal microphone system should perform as well as the human auditory system. With binaural stereo imaging of the auditory scene, the microphone system could spatially pinpoint the speech source and then isolate its signal by ignoring sound emanating from all other points. With a swivel mounted steering system (like that of the human head and neck), the microphone system could effectively track and focus its array on a mobile speech source.

Simpler techniques of noise reduction can be applied to a conventional single microphone system that will improve ASR performance. Spectral measurements of the ambient noise can be used by an ASR system to mask out the effects of steady background noise frequencies below the measured threshold amplitude. As well, a microphone enclosure can be used to attenuate sound emanating from sources outside a given target zone (spatial filtering) including reverberation (sound reflections within a given room).

4.4.2.3 The Transmission Channel

The path traveled by the speech signal from the speaker to the recognition component of the ASR system may introduce spectral distortion and additive noise. Measures must be implemented to ensure that the desired information within the original speech signal is still present after transmission.

Telephone Communications

One of the current demands facing ASR developers is to create an ASR system that performs well over telephone channels. The demand is fueled by economic pressures and the rise in mobile cellular technology, which provides a natural domain for many commercial applications of human-computer interface technology. The use of the telephone channel in ASR systems generally reduces speech recognition accuracy because the transmission path is not consistent but varies depending on the routing of the call through the telephone network. This varying pathway introduces signal distortions such as echoes, cross-talk, phase shift, and spectral attenuation that are difficult to anticipate in advance. One of the most audible effects of the telephone channel is its bandpass filter effect on speech in the frequency range around 300 to 3300 Hertz. Transmission channel effects can be partially alleviated by utilizing information bearing speech features that are not degraded by transmission. The problem could be avoided altogether by processing the speech signal locally (at the hand set) and transmitting information in digital form, but at a greater cost since such a system could not take advantage of the average phone systems already in place. Since over 75% of

Europe's phone systems still operate on Pulse Code signaling, phone companies hope that ASR systems will allow them to sell advanced features to their customers without having to perform costly upgrades to existing phone lines.

4.4.2.4 The Variability of Speech

The human voice can vary the pronunciation of a given phrase in many ways including speaker pitch, intonation, rate of speech, emphasis, and mood. The timbre of one's voice varies from day to day and is also affected by the one's health. These sources of variability combine with the effects of the acoustic environment and transmission channel to greatly vary the speech signal patterns of two instances of the same utterance as viewed by the ASR system. Auditory models of speech processing attempt to emulate the abilities of the human auditory system to perceive the intended linguistic message in a speech signal despite its variability. Understanding human speech perception is an important step in the development of ASR systems. Figure 4.2 below illustrates the hierarchical sequence of the sources of variability affecting the speech signal from the time it is originated by the speaker until it is classified by the ASR system.

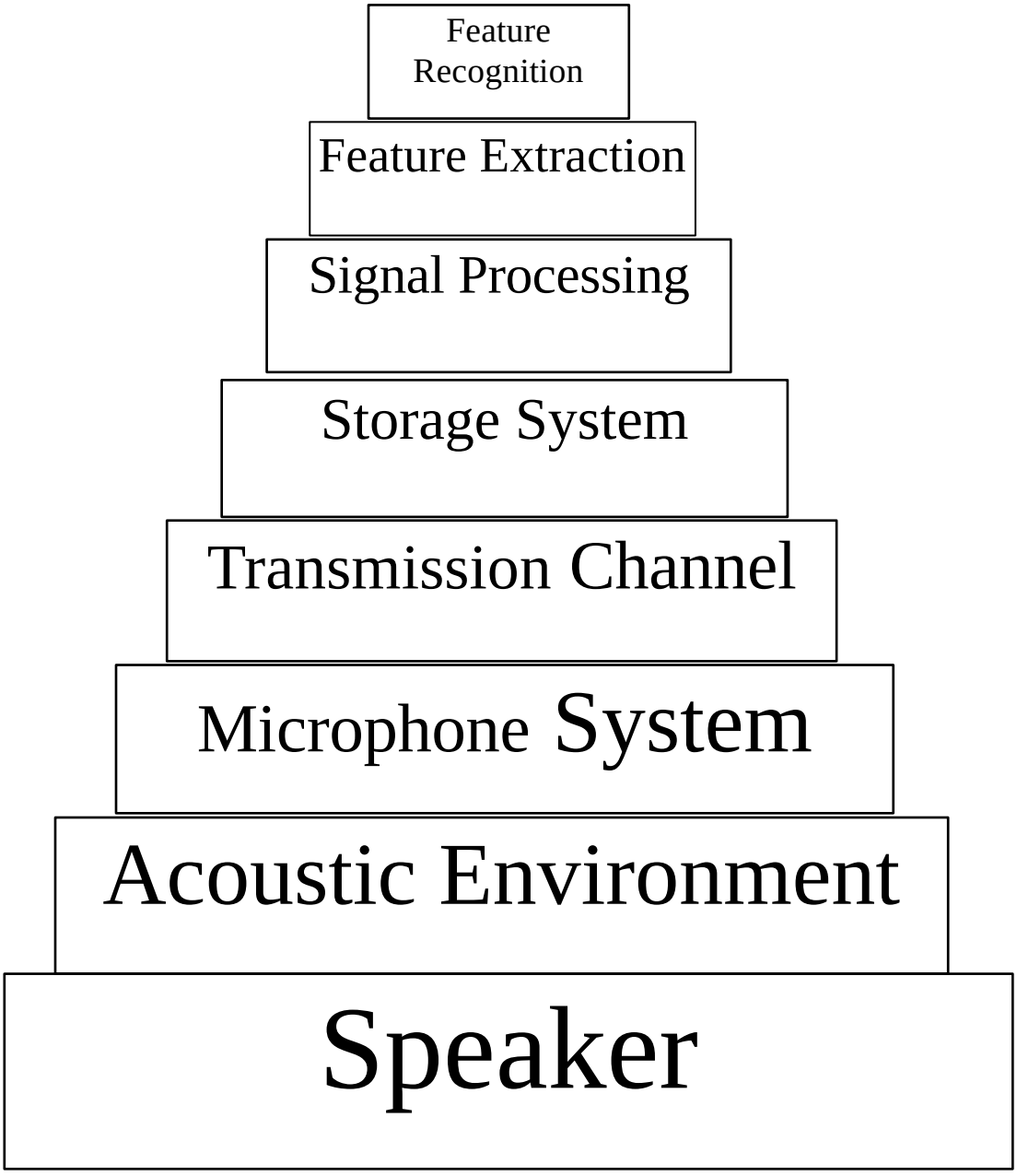


Figure 4.2 A Speech Variability Pyramid

4.5 Human Speech Communication

The study of the production and perception of speech by humans is necessary for researchers who desire to emulate this ability using machines. The entire speech communication process is aimed at transferring a thought from the mind of the speaker to the minds of the audience. Successful communication of ideas using speech depends on both the skill and ability of both parties involved in the transfer process. The following quote from a Communication Speaking class at the Canadian Forces Officer Candidate School illustrates the challenge of spoken communication:

*"You may have **heard** what I actually **said**, but I am not sure you **understood** what I **meant**!"*

It is for this reason that Officer Candidates receive a great deal of training in the communication of instructions to other personnel. Considering the time and effort required by humans to perform effective spoken communication, one can appreciate the challenge of producing a machine that can perform just as well.

4.5.1 Speech Production

The speech production process begins with the conceptualization of an idea that the speaker wants to convey to the audience. He converts this idea into a linguistic structure by selecting the appropriate words to represent it based on his knowledge and that of his audience (if known). His brain then sends signals to the motor-neurons to control the muscles of the vocal tract to articulate an acoustic representation of the linguistic structure [Fur89].

Articulation

Speech is produced by the flow of air through the vocal tract system. The vocal cords can vibrate in the air flow to produce **voiced** sounds at a fundamental frequency ($\mathbf{F_0}$) or voice pitch varied by adjusting the vocal cord tension. $\mathbf{F_0}$ is inversely proportional to the length of

the vocal tract, which varies with the speaker's age and gender. The shape of the vocal tract cavities can also be adjusted to produce specific resonance frequencies or **formants,** which give voiced speech its characteristic sounds. The formants are labeled from lowest to highest frequency using the symbols $\mathbf{F_1}$, $\mathbf{F_2}$, $\mathbf{F_3}$, etc. The shape of the vocal tract is controlled by active **articulators** or moveable speech organs such as the tongue, lips and lower jaw. **Voiceless** sounds are produced without the vibration of the vocal cords. Examples of this are the noise-like **fricative** sounds pronounced in the letters "S" and "F". These sounds are produced by turbulence in the air flow created by the partial closure of the opening of the vocal tract. The **plosive** sounds pronounced in the letters "B", "D", "K", and "P", are produced by the release of built up pressure after a complete closure of the vocal tract.

Phonemes

The smallest set of distinctive sounds for a given language is referred to as the **phonemes** of that language. The phoneme is a linguistic unit defined such that "***... if one phoneme is substituted for another in a word, the meaning of that word may be changed***.**"** [Ain88]. Phonemes can be grouped into classes based on their acoustic characteristics. The two main classes of phonemes are vowels and consonants. **Vowels** are voiced phonemes that are produced with an open vocal tract, that is, a vocal tract with no closure or stricture or close approximation of the articulators. Vowels are easily recognized by their relatively high energy due to the projection of relatively high amplitude resonance frequencies from the open vocal tract. Phonemes lacking this **sonority** or energy quality are called **consonants** and form the other main phoneme category.

4.5.2 Speech Perception

Sound can be defined as "***... the impression received by the brain as a result of the vibration of the ear drum in reaction to changes in air pressure***." [Har72]. This definition points to the difference between the reception and the perception of the speech signal in the human auditory system. The mechanical portion of the auditory system (outer, middle and

inner ear) acts like a microphone. It collects and converts acoustic vibrations to electrical impulses that the neural portion of the auditory system uses to create an impression of the auditory scene in the mind of the listener. The path and the transformation of neural impulses along the auditory nerves from the inner ear to the auditory cortex are complex and not fully understood. How the brain perceives speech from the acoustic vibrations received by the ear is the subject of ongoing research. The following discussion of Formant Ratio Theory offers an explanation for the manner in which we perceive the phonemes that form the building blocks of speech.

4.6 Formant Ratio Theory

This section introduces the concepts of **FRT (Formant Ratio Theory)**. A background knowledge of this theory will allow the reader to appreciate the reason for choosing the feature extraction process implemented in the ASR system described in chapter five. FRT addresses the problem of relating the acoustic waveforms of the vowels uttered by a speaker to the vowels perceived by his or her audience.

According to [Mil89], FRT was formulated by Richard John Lloyd between 1890 and 1892. Lloyd's theory stated that the quality of a vowel depends on the relative frequencies of the formants and not the absolute values. His theory was inspired by several observations, including:

- the role of frequency ratios in music,
- the acoustic differences between the vowels of men, women and children,
- the experiments he carried out involving synthetically produced vowels using glass tubes, and
- the studies of transposition done by varying the playback speed of Ediphone recordings.

Lloyd noticed that similar articulations produce similar perceptions of vowel qualities and similar ratios of the formant frequencies. Today, it is well established that the locations of the

first three formants in the acoustic signal are highly correlated with vowel sound perceived by the human brain [Mil89].

The main strength of FRT is its ability to greatly reduce the differences in the acoustic description of vowels caused by a speaker's age and gender (e.g., generally, younger speakers have higher pitched voices compared to older speakers, while female speakers have higher pitched voices than male speakers). To improve the robustness of ASR, the speech signal must be converted into a set of features in which the variability between speakers is minimal. Such variability is reduced by representing the vowel as a set of ratios of its formant frequencies. In ratio form, the units of frequency cancel out leaving only a measure of the relative difference between the formants, which is highly correlated between different speakers for a given vowel sound.

To dramatically illustrate the effectiveness of formant ratios as a speech identification feature, table 4.2 compares eight American English vowel sounds spoken by men, women and children. The bulk of the formant frequency data was taken from [Bar57] and represents values averaged from 33 men, 28 women and 15 children. The data marked "[Lad75]" was not gender labeled and is included for independent comparison. The vowel symbols used follow the convention in [Mil89] as they lend easily to type written work. Each symbol is immediately followed by a coaching word to aid the reader in identifying the correct vowel sound being described. **It is important to note the correlation of the formant ratio set $\{F_3/F_2, F_2/F_1, F_3/F_1\}$ between speakers for a given vowel as well as between different vowels**.

To better visualize the relationships in the vowel data presented in table 4.2, figures 4.3 and 4.4 contrast the difference between the formant frequency representation and the formant ratio representation of vowels. Figure 4.5 illustrates the **NPA** (Normalized Polar Angle) formant ratio of the vowels from table 4.2. Although not formally introduced until chapter five, the NPA representation is presented here for the reader's convenience to show how the distinction between different vowels can be improved, and the differences due to speaker age and gender for a given vowel can be reduced.

Table 4.2

A

Vowel / ID	No	Speaker	F_3	F_2	F_1	F_3/F_2	F_2/F_1	F_3/F_1
IY (beet)	**1**	Man	3010	2290	270	1.31	8.48	11.15
	2	Woman	3310	2790	310	1.19	9.00	10.68
	3	Child	3730	3200	370	1.17	8.65	10.08
	4	[Lad75]	2890	2250	280	1.28	8.04	10.32
IH (bit)	**5**	Man	2550	1990	390	1.13	5.10	5.77
	6	Woman	3070	2480	430	1.24	5.77	7.14
	7	Child	3600	2730	530	1.32	5.15	6.79
	8	[Lad75]	2560	1920	400	1.33	4.80	6.40
EH (bet)	**9**	Man	2480	1840	530	1.35	3.47	4.68
	10	Woman	2990	2330	610	1.28	3.82	4.90
	11	Child	3570	2610	690	1.37	3.78	5.17
	12	[Lad75]	2490	1770	550	1.41	3.22	4.53
AE (bat)	**13**	Man	2410	1720	660	1.40	2.61	3.65
	14	Woman	2850	2050	860	1.39	2.38	3.31
	15	Child	3320	2320	1010	1.43	2.30	3.29
	16	[Lad75]	2490	1660	690	1.50	2.41	3.61
AA (hot)	**17**	Man	2440	1090	730	2.24	1.49	3.34
	18	Woman	2810	1220	850	2.30	1.44	3.31
	19	Child	3170	1370	1030	2.31	1.33	3.08
	20	[Lad75]	2540	1100	710	2.31	1.55	3.58
	AO	Man	2410	840	570	2.87	1.47	4.23
	22	Woman	2710	920	590	2.95	1.56	4.59
	23	Child	3180	1060	680	3.00	1.56	4.68
	24	[Lad75]	2540	880	590	2.89	1.49	4.31
UH (full)	**25**	Man	2240	1020	440	2.20	2.32	5.09
	26	Woman	2680	1160	470	2.31	2.47	5.70
	27	Child	3310	1410	560	2.35	2.52	5.91
	28	[Lad75]	2380	1030	450	2.31	2.29	5.29
UW (good)	**29**	Man	2240	870	300	2.57	2.90	7.47
	30	Woman	2670	950	370	2.81	2.57	7.22
	31	Child	3260	1170	430	2.79	2.72	7.58
	32	[Lad75]	2250	870	310	2.59	2.81	7.26

Comparison of Vowels by Formant Frequencies and Formant Ratios

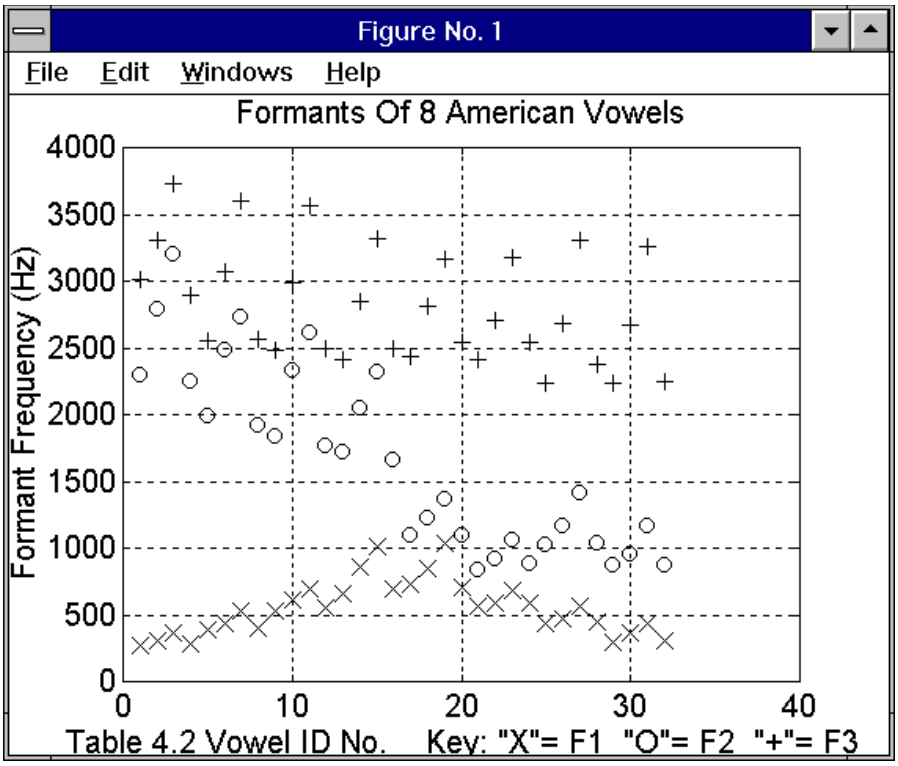


Figure 4.3 Vowel Formant Frequencies from Table 4.2

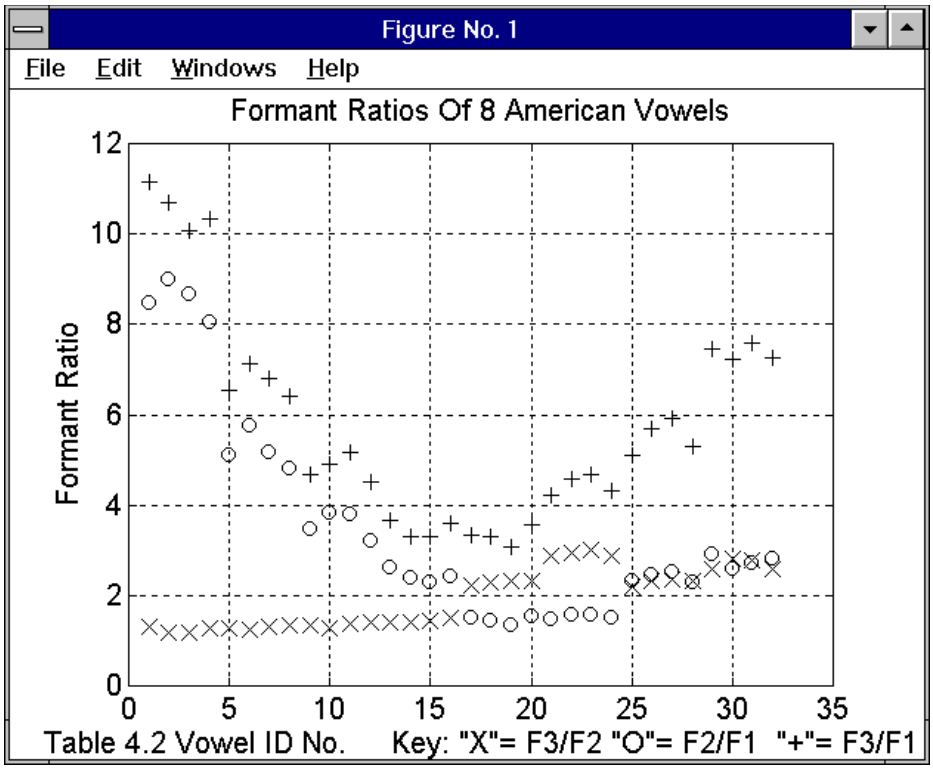


Figure 4.4 Vowel Formant Ratios from Table 4.2

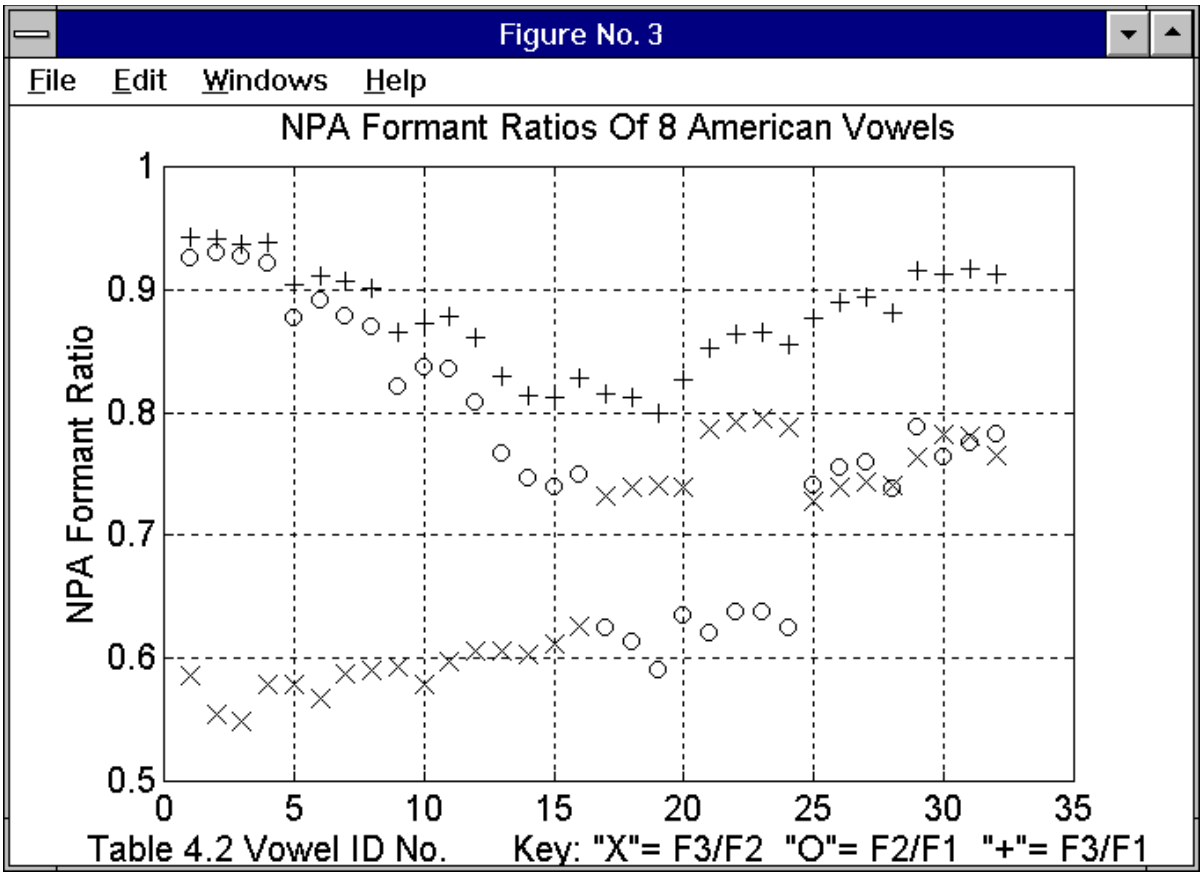


Figure 4.5 Normalized Polar Angle Formant Ratios of Table 4.2 Vowels

4.7 Chapter Summary

This chapter covered four main areas -- ASR system classification and the ASR process, some ASR research concerns, human speech communication, and Formant Ratio Theory:

- The classification of ASR systems was discussed with respect to their speaker dependency, the manner of speech they could recognize, and the size of their active vocabulary. ASR was then described as a four stage process consisting of the sound recording (signal capture), spectral analysis (signal processing), feature extraction (signal compression), and utterance classification (signal identification).

- The ASR research concerns of rapid system training and improved recognition robustness were reviewed. The effects of the acoustic environment, the microphone system, the transmission system, and the variability of speech on ASR system accuracy were discussed.

- The coverage of the production and perception of speech briefly exposed the reader to the process of articulation and the descriptive jargon used to describe speech sounds. For more information on the subject, the reader may wish to read the second chapters of [Ain88] and [Fur89].

    - The Formant Ratio Theory is briefly explained and then illustrated using the formant frequencies of eight American English vowels spoken by several men, women and children. This section shows that vowels can be identified, independently of the speaker's age or gender, based on of the combination of the ratios of the first three formant frequencies present in the acoustic signal.

This chapter is important for the reader's appreciation of the steps taken in the implementation of the ASR system described in chapter five. The reasons for the care taken in capturing the acoustic speech signal, its conversion into a spectral representation, the tracking of formants, and their representation as a set of ratios prior to classification by the Fuzzy ART Map ANN recognizer component, can be found in this chapter.

CHAPTER FIVE

A FUZZY ART MAP ASR SYSTEM

5.1 Introduction

This chapter marks the beginning of the core work of this thesis. The background information given in chapter two (Neural Networks), chapter three (Adaptive Resonance Theory and Fuzzy Logic), and chapter four (Automatic Speech Recognition and Formant Ratio Theory), has been presented in support of the discussion that follows.

This chapter investigates the applicability of a Fuzzy ART Map ANN to the ASR task through the implementation of an ASR system that incorporates a Fuzzy ART Map ANN as its recognition component. The system is capable of performing speaker and language independent discrete phrase recognition over public telephone lines, based on the time series of formant ratios. It is implemented on a 486DX33 desktop computer with sound card and microphone. The *GoldWave 2.11* application is used for on-line speech capture while the *Matlab 4.0* programming environment is used to accomplish the signal processing, feature extraction and ANN simulation. More specific details on the hardware and software used may be found in annex A. The source code for the programs written specifically for this thesis are contained in annexes B through G.

This ASR system is presented within the framework of the four stage ASR process presented earlier in chapter four. First, the **sound recording stage** describes the apparatus and method used to capture the speech signal as well as the creation and storage of the RMC Speech Database that was subsequently used to train and test the ASR system. Second, the **signal processing stage** deals with the spectral analysis of the recorded speech samples and the production of their spectrogram representations. Third, the **feature extraction stage** illustrates the steps required to identify the formants in the spectrogram representations produced in stage two. It also deals with the representation of speech features to the Fuzzy ART Map ANN for recognition. Finally, in the **utterance classification stage,** the Fuzzy

ART Map ANN program is discussed regarding software simplifications, the tuning of the training parameters, and its information retrieval characteristics.

Both microphone and telephone recorded speech are illustrated throughout the chapter to compare the relative effectiveness of the ASR process in both transmission channels. Figure numbers carry the suffix "M" to denote microphone recorded signals while the suffix "T" is used to identify telephone recorded signals.

5.2 Stage #1 -- Speech Recording

The RMC Speech Database

Realistic testing of any speech recognition system precludes the use of voice samples recorded within a narrow time-frame. A database of speech samples recorded over an extended period is needed to capture the daily variations in each of the speaker's voices. Such a database was amassed from September 2 to December 22, 1994. Producing the RMC Speech database was not only more cost-effective than purchasing a commercially available speech corpus, but it had the added advantage that the subjects were locally available to perform live testing of the completed system. Off-line recording of speech samples allowed for the population of the database before the speech recognition system was completed, as well as providing speech samples for design testing. Once the ASR system was completed, on-line automated speech sampling was possible.

The RMC Speech Database is composed of voice samples from 14 volunteers from within the Department of Electrical and Computer Engineering at RMC. The particulars of each speaker are listed below to show the range of age, gender and language experience of the speaker set:

Speaker	***Gender***	***Age***	***Languages Spoken***
Ann Hopkinson	Female	30+	English, French

Bernard Tremblay	Male	30	English, French
Collin Wortley	Male	49	English
Danielle Bossart	Female	54	English, French, German
David Gravel	Male	32	English, French
Don Hamilton	Male	31	English
David Young	Male	31	English
Ian Hawes	Male	26	English
Jean Dolbec	Male	26	French, English
Liam Porter	Male	30	English, French
Larry Reeves	Male	30	English
Murray Johnson	Male	36	English, French
Matthew Oliver	Male	29	English, German
Waldemar Borys	Male	31	English

Table 5.1 RMC Speech Database Speaker Profiles

The speech samples consist of ten phrases recorded on a weekly basis over the four month collection period. These phrases are listed in table 5.2 in the order in which they were recorded within each session. They are numbered zero through nine to match the file naming convention (see figure 5.1) used to store the samples on computer. The first two phrases were included to aid in the registration of speech samples while the last eight phrases were chosen to represent a phonetically diverse range of speech sounds. The third and fourth phrases were taken from the TIMIT speech database, which has been used widely throughout the ASR literature. The remaining six phrases were taken from a published speaker recognition system where they were used to ***"... model the overall distribution of speech sounds that are likely to occur."*** [Sam76]:

Phrase Identifier	***Spoken Phrase***

0	The date is ..... and the time is .....
1	My name is .....
2	She had your dark suit in greasy wash water all year.
3	Don’t make me carry an oily rag like that.
4	May we all learn a yellow lion roar.
5	We were away a year ago.
6	I know when my lawyer is due.
7	Every salt breeze comes from the sea.
8	Our yacht slid around the point into the bay.
9	I was stunned by the beauty of the view.

Table 5.2 RMC Speech Database Phrase List

5.3 Recording Method and Apparatus

To create a robust system, factors concerning the acoustic environment, microphone system, transmission system, and variability of speech were considered in the recording method and apparatus used to capture and preserve the speech signal with a minimum of distortion.

The Acoustic Environment

The acoustic environment of the database recording sessions was chosen to be similar to that in which the finished ASR system was intended to operate, hence, the recording location was chosen in an openly accessible office area with background noise from computer equipment and personnel. The office space was located in room 5018 of the Sawyer Building at RMC. The reverberance in the room was increased by the sound reflecting quality of the painted cinder block walls.

The Microphone System

Since the microphone system is the point of entry for the speech signal to the ASR system, it is important to use a microphone that gives consistent performance over the range of frequencies that are to be recorded. The microphone system should also be able to focus on the speaker and ignore sounds emanating from other sources. The Realistic Super Cardiod microphone was used because it had these attributes. It has a flat frequency response curve over the desired frequency range and provides a 30 degree maximum pick-up cone about the microphone axis that can be used to focus on the speaker. Acoustic signals that have an angle of incidence to the microphone axis greater than 30 degrees are attenuated according to the polar pick-up pattern of the microphone. Generally, attenuation increases with the angle of incidence for a given frequency.

A sound treated microphone enclosure (mic box) was used to shield the microphone from sound sources not directly in front of the microphone and helped reduce the effects of reverberation. The enclosure consisted of a three cubic foot cardboard box lined with egg cartons. The speaker sat in front of the mic box and read the phrase script mounted inside the box. In this manner the speaker's head was always directed towards the microphone during the recording. The microphone placement within the box was dead center of the back wall with the tail handle of the microphone extending through the back wall to support it without the need of a stand.

As an added noise reduction measure, the ambient room noise sampled on several occasions and was later used to construct an active noise mask in the feature extraction stage of the ASR process.

The Transmission System

The path that the speech signal travels from the microphone to the medium where it is digitally stored can be the source of spectral distortions and noise. The opportunity for signal distortion is much less during the on-line recording process compared to the off-line process. During the on-line recording of speech samples, the signal travels directly from the

microphone to the computer sound card where it is digitized. During the off-line process the speech signal travels to the tape deck where it is stored in an analog format on magnetic tape. The signal is then played back from the magnetic tape to the computer sound card for digital storage. To minimize signal distortion during the off-line process, the microphone gain on the tape deck was taped over to prevent random adjustment between speaker sessions. As well, a metal bias cassette was used that provided good frequency response and minimized "tape hiss" noise during playback of the speech signal. No special shielding of the sound card or cables was employed.

Speech Variability

To test an ASR system realistically, it is necessary to record speech samples at various times of the day over an extended period, since the quality of a speaker's voice varies over time. The RMC Speech Database was constructed with this goal in mind. Speakers were also asked to vary their rate of speech and intonation between sessions to provide a wider range of expression on which to test the robustness of the completed system.

5.4 Storage Method

All speech samples were digitized using the computer sound card and then compressed for bulk storage. The analog signals were digitized using a sampling rate of 22050 Hz with 16 bit amplitude resolution and stored in a monaural **PCM (Pulse Code Modulation)** *Windows* "WAV" file format. A filename convention was developed so that the speech sample files could be easily identified. The *MS DOS* operating system restricts the length of filenames to 8 letters, which lead to the adoption of the file naming convention shown in figure 5.1 :

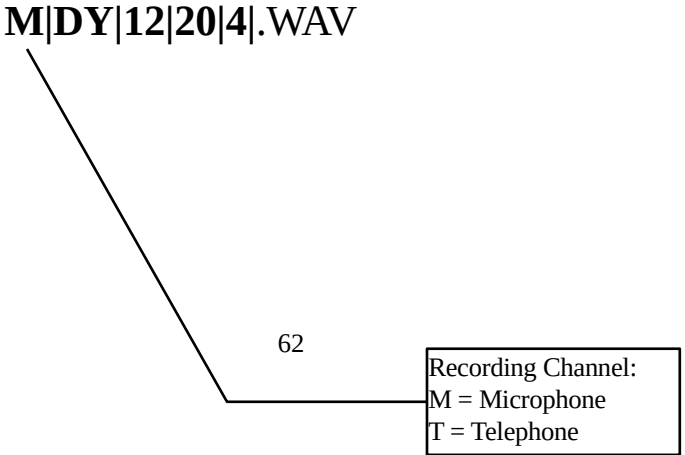


Figure 5.1 RMC Speech Database File Naming Convention

The "WAV" files were subsequently compressed using the *WazWav 0.1* compression program, which achieves a lossy 4:1 compression ratio. Several before and after trials with this program revealed that the frequency loss of a **ComPanded** (compressed then expanded) speech sample occurred above the maximum frequency to be used in the feature extraction process. Thus the loss was an acceptable trade off for the 75% reduction in storage space. The compressed "WAV" files are given a "WAZ" extension on the sample file name.

The database was copied to 3.5 inch high density floppy disks as a backup measure to prevent data loss. A multi-volume archive or the database was created using the *ARJ* (lossless) compression program, which enabled an additional 10% reduction in storage space. To reduce the number of disks required to hold the entire database, the *FORM1968* program was used to reformat the disks from the standard 1.44 MB capacity to a 1.92 MB (1968 KB) capacity.

The gains of compressed data storage are evident when one considers that the RMC Speech Database in uncompressed form consists of 2002 files requiring 218.4 MB of uncompressed disk space. After compression using *WazWav* and *ARJ* and disk reformatting using *FORM1968* , the database occupies only 49.3 MB stored on 26 standard 3.5 inch floppy disks!

ASR System Hardware Components

Figure 5.2 summarizes the communication paths between the various hardware components of the ASR system. The circular shapes denote entities that are external to the ASR system, such as the human speaker and the telephone system. Three speech transmission routes are shown between the speaker and the computer sound card. The “on-line recording” route (directly from the microphone to the sound card) introduced the least amount of signal distortion. The “batch recording” route (used in the creation of the RMC Speech Database) introduced a small amount of spectral attenuation due to the interim analog storage of the speech signal. The “telephone recording” route provided the poorest recording quality of the three paths. The telephone filter effect combined with additive noise from the telephone pickup to produce a signal that represented a realistic challenge to the ASR system. The computer hard drive was used as the working storage area for the digitized signals and removable floppy disks were used to store the permanent copies. During on-line recording, computer speakers allowed people to audition their recordings before submitting them for classification.

Figures 5.3-M and 5.3-T illustrate the time/amplitude representation of the phrase “***May we all learn a yellow lion roar*** ” batch recorded and stored in MDY12204.WAV, then replayed over a local telephone connection and on-line recorded to file TDY12204.WAV.

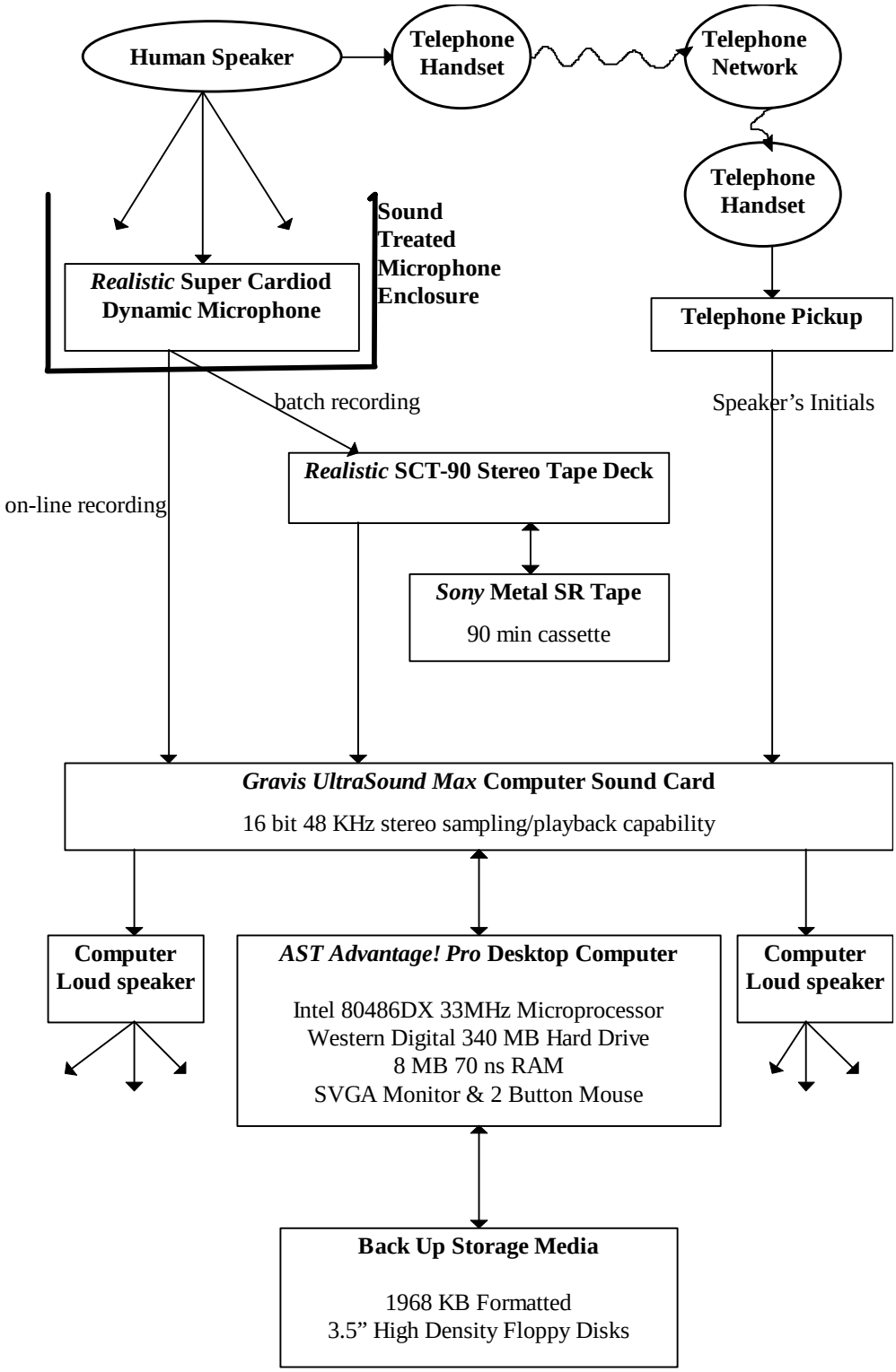


Figure 5-2 ASR System Hardware Components

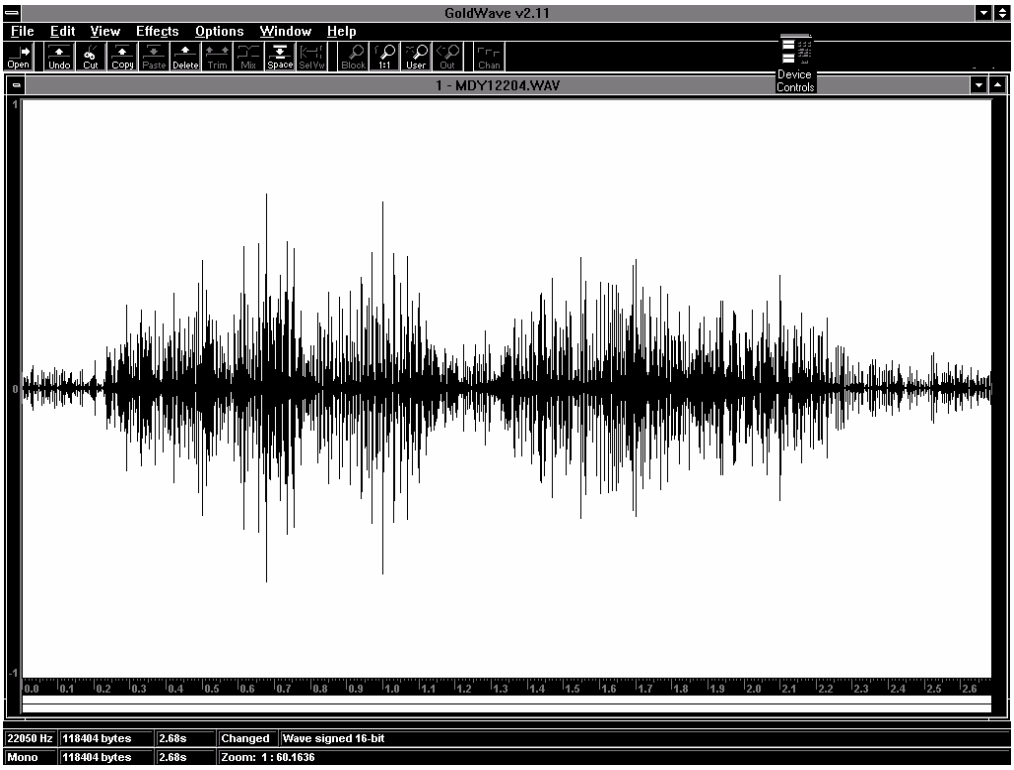


Figure 5.3-M Microphone Speech Signal MDY12204.WAV



Figure 5.3-T Telephone Speech Signal TDY12204.WAV

5.5 Stage #2 -- Signal Processing

In the previous stage, the acoustic speech signals were captured and converted to a digital format. The microphone path speech recordings were sampled at a rate of 22050 Hz and a resolution of 16 bits per sample. Telephone path speech recordings were sampled at a rate of 8000 Hz and a resolution of 16 bits per sample. In this stage, spectral analysis was performed. To accomplish this task, the *Matlab* program "SPECGRAM.M" was invoked using a 512 sample point Hanning window. A 50% window overlap was used for both microphone and telephone recorded speech samples. The resulting temporal resolution of the spectrogram was 11.6 msec for microphone recordings and 32.0 msec for telephone recordings.

The amplitude of the time domain signal was normalized before calculating the spectrogram. All values in the speech sample were adjusted by the ratio of the maximum sample amplitude compared to the maximum amplitude possible in 16 bit representation (2^{15}). Endpoint detection of the spoken phrase was not attempted in the time domain signal. The relative level of background noise varied between speech samples and made the use of a fixed noise threshold prone to error.

The program "MAKESPEC.M" was written to build the spectrogram in a piecemeal fashion in order to circumvent the memory limitations of the ASR system hardware. This was accomplished by segmenting the speech sample into chunks one second long, calculating the spectrogram for each chunk, and then concatenating the resulting spectrograms together. To reduce the storage requirements for each spectrogram matrix, only that portion of the spectrogram representing frequencies below 5000 Hz was stored in the output file. The output filename for each spectrogram file incorporated the name of the sound file used to create it. The first letter in the filename was changed to an "S" to indicate it was a spectrogram file. Examples of the spectrogram produced by the "MAKESPEC.M" program are illustrated in figures 5.4 through 5.7 for both the microphone and telephone recorded speech samples. These spectrograms are used in the next stage of the ASR process.

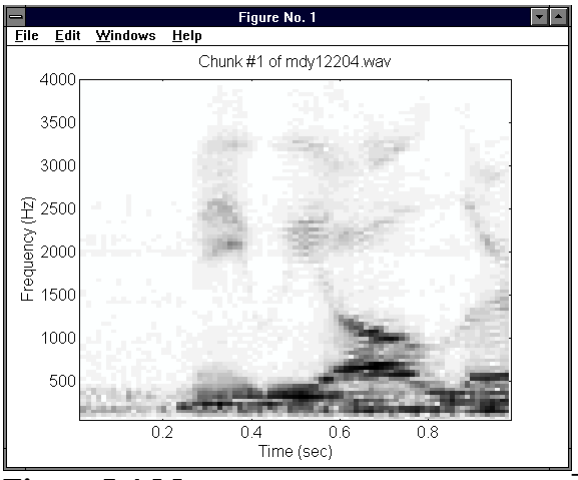


Figure 5.4-M
Chunk #1 of MDY12204.WAV

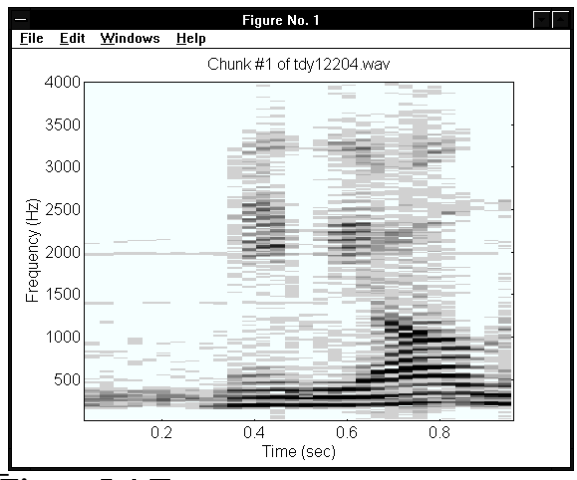


Figure 5.4-T
Chunk #1 of TDY12204.WAV

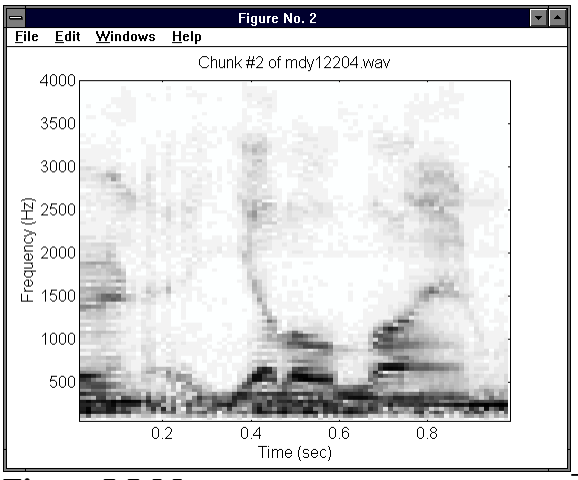


Figure 5.5-M
Chunk #2 of MDY12204.WAV

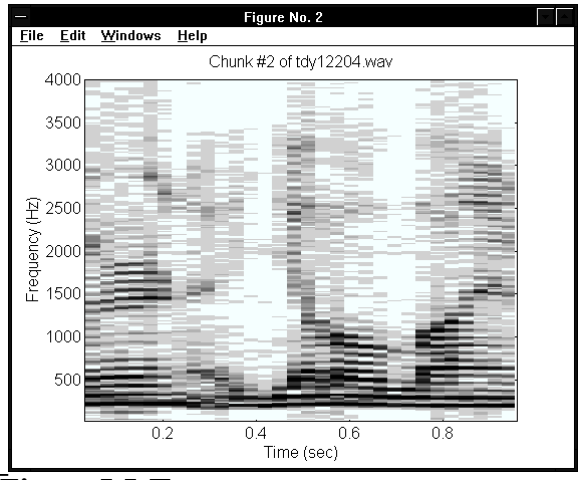


Figure 5.5-T
Chunk #2 of TDY12204.WAV

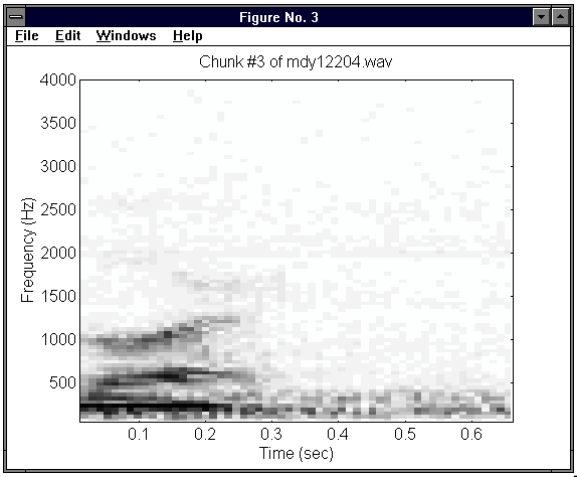


Figure 5.6-M
Chunk #3 of MDY12204.WAV

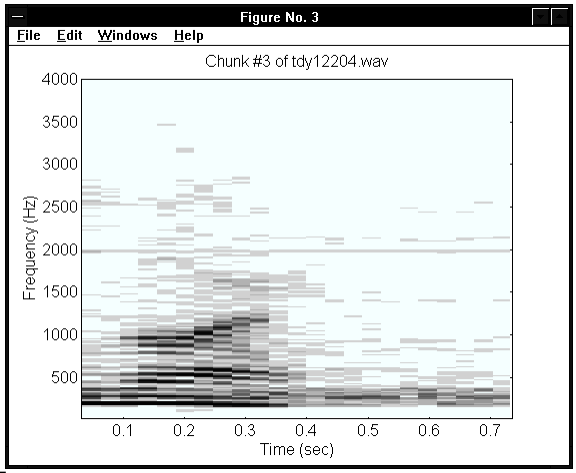


Figure 5.6-T
Chunk #3 of TDY12204.WAV

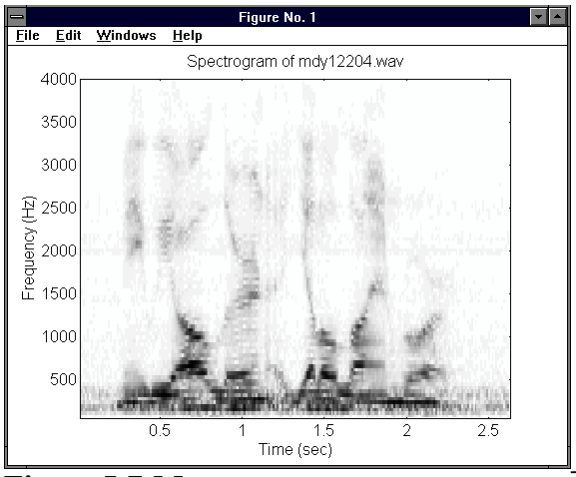


Figure 5.7-M
Spectrogram of MDY12204.WAV

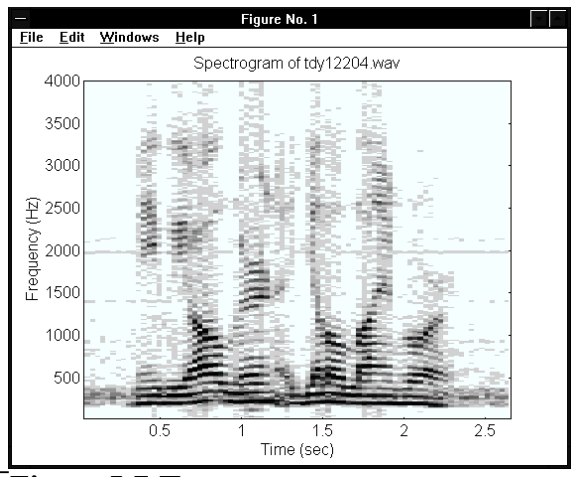


Figure 5.7-T
Spectrogram of TDY12204.WAV

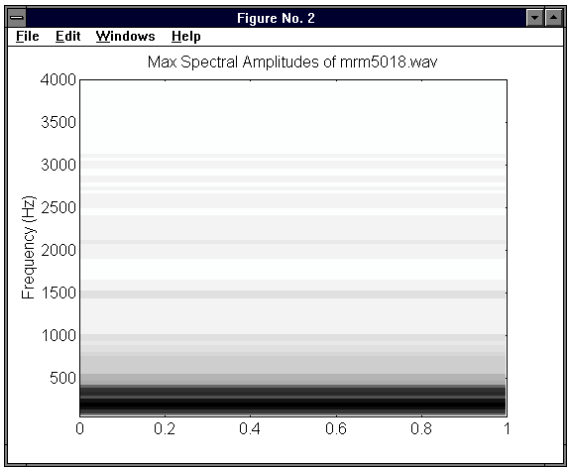


Figure 5.8-M
Microphone Noise Mask

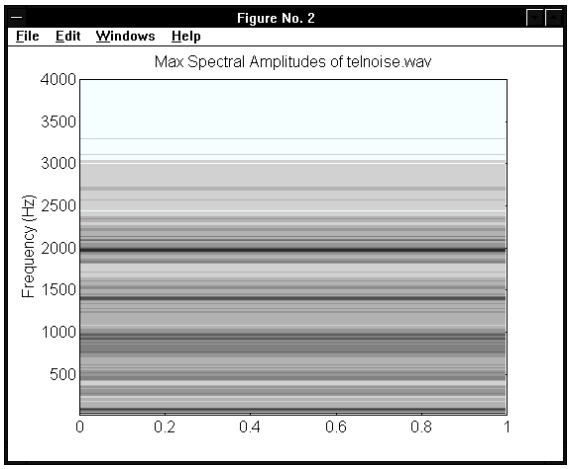


Figure 5.8-T
Telephone Noise Mask

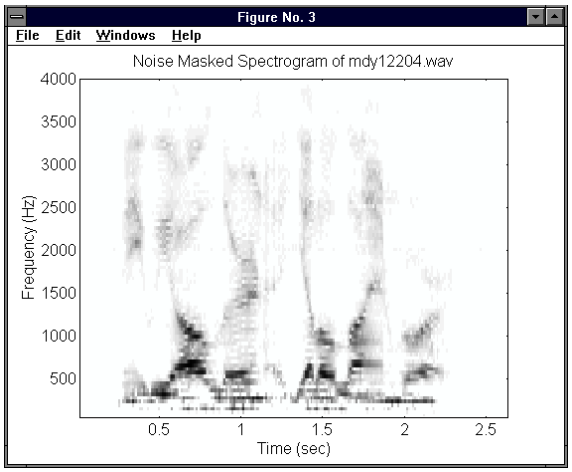


Figure 5.9-M
Noise Masking of MDY12204.WAV

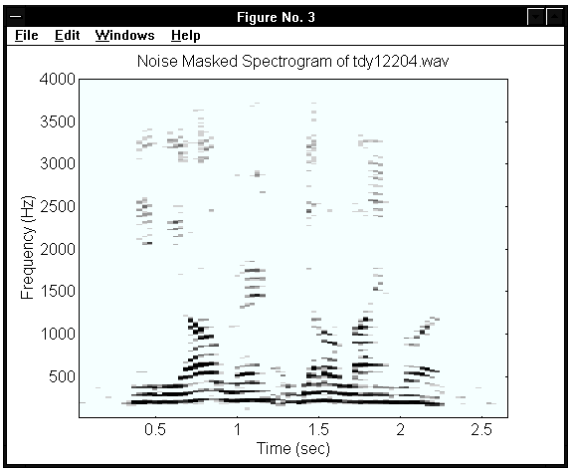


Figure 5.9-T
Noise Masking of TDY12204.WAV

5.6 Stage #3 -- Feature Extraction

This stage is concerned with the isolation of linguistic features within the spectrograms of the speech samples. Non-relevant data is discarded to reduce the load of subsequent calculations. The following text describes the formant extraction process carried out by the "FORMANT3.M" program written as part of this thesis work. The first three formants are used to represent the spoken phrase to the Fuzzy ART Map ANN for identification and classification. Section 5.7 will cover the specifics of the feature representation used to accomplish this task. In order for the reader to gain an appreciation of the inherent difficulty of the ASR task over telephone lines, figures illustrating the processing of both microphone and telephone recorded speech spectrograms are included. Only the portion of the spectrogram corresponding to the telephone bandwidth (300 Hz to 3300 Hz) was considered in the feature extraction process.

5.6.1 Noise Masking

A noise mask was constructed using background noise samples as a means of filtering out the background noise evident in the spectrograms of the speech samples. To build the noise mask, a spectrogram of one second of recorded background noise is calculated. Next, the maximum amplitude of each frequency band in the noise spectrogram is identified and used to produce a masking vector. This spectra of maximum noise amplitudes is subtracted from each time-slice of the speech sample spectrogram calculated above, thus removing any possible contributions of background noise. Any negative values resulting from the previous calculations are reset to a zero value. The use of this noise masking technique improved the identification of the formant frequencies. The masking vectors are shown in figure 5.8-M and figure 5.8-T. The effectiveness of this masking technique can be seen in the noise reduced spectrograms shown in figure 5.9-M and figure 5.9-T.

5.6.2 Silence Reduction

Silence reduction from the speech sample is performed based on a relative **STE (short term energy)** curve derived from the sample's spectrogram. Each point on the curve corresponds to the sum of the frequency amplitudes for a given time-slice of the spectrogram. Since formants appear only in voiced speech it is desirable to isolate only the voiced portions of the speech sample in order to improve formant tracking. Since voiced speech has higher relative energy compared to unvoiced speech, all time-slices of the spectrogram whose STE value falls below a relative threshold (set to a percentage of the maximum STE energy) are discarded. Figures 5.10-M, 5.10-T, 5.11-M and 5.11-T illustrate the spectrograms of a microphone and telephone speech sample before and after the application of this silence reduction technique.

5.6.3 Frequency Amplitude Normalization (Pre-emphasis)

When considering the human voice, there is a general drop in the average amplitude of a frequency as frequency increases. This effect precludes the technique of simply picking formant peaks based on absolute frequency amplitude. In order to counter this effect, the amplitudes of all frequencies are amplified by an amount proportional to their relative frequency octave according to equation 5.1. This formula was derived through experimentation with microphone recorded speech samples. Ideally an adaptive gain function should be used (particularly for use with telephone channel speech) but this item has been left for future work. Figures 5.12-M and 5.12-T illustrate the effect of the octave enhancement technique.

$$\textbf{Octave Gain (SFR)} = 10 \wedge \left(\log_2 (\text{SFR}) / 5 \right) \qquad (5.1)$$

where SFR = Spectrogram Frequency Row number

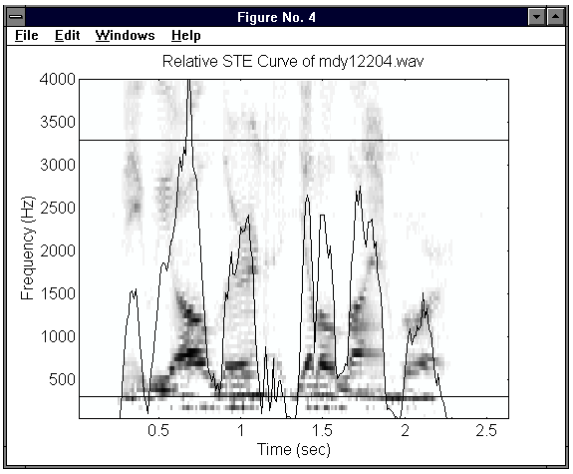


Figure 5.10-M
STE Curve for MDY12204.WAV

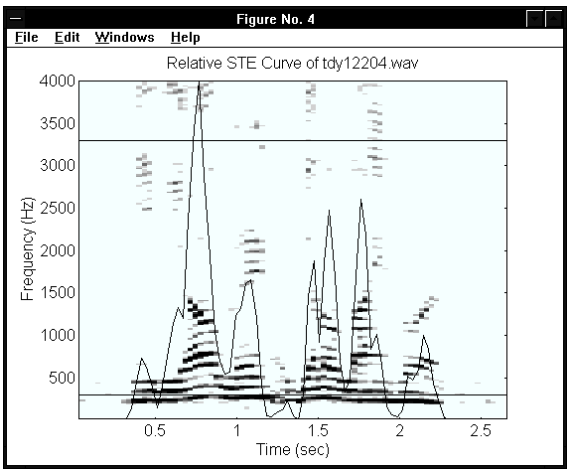


Figure 5.10-T
STE Curve for TDY12204.WAV

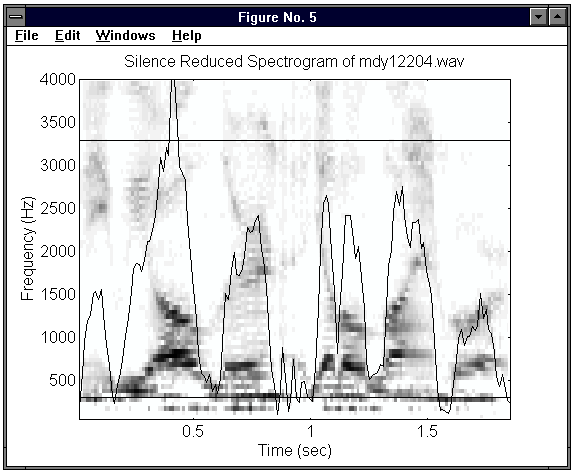


Figure 5.11-M
Silence Reduced MDY12204.WAV

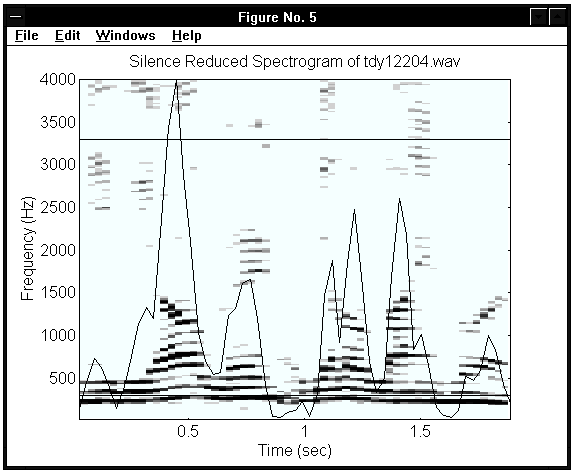


Figure 5.11-T
Silence Reduced TDY12204.WAV

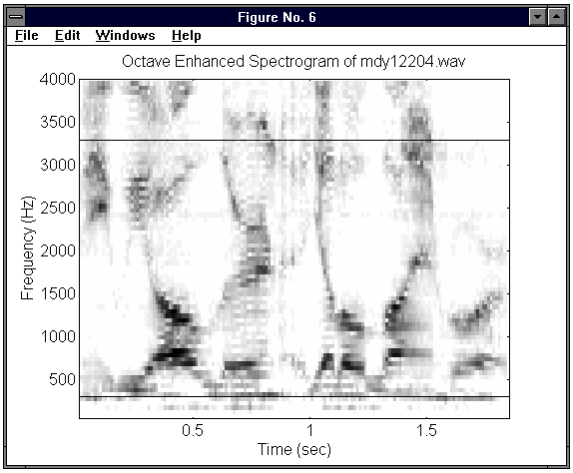


Figure 5.12-M
Octave Enhanced MDY12204.WAV

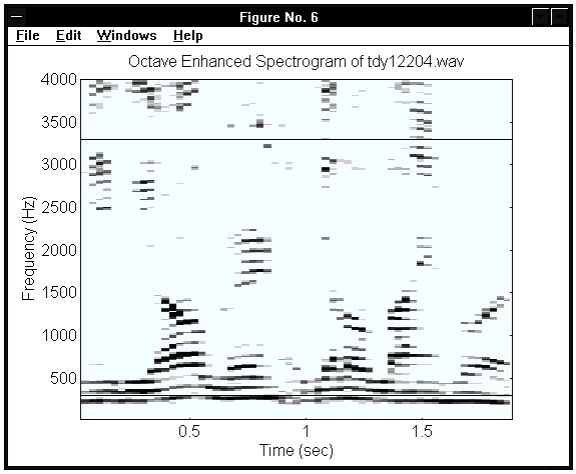


Figure 5.12-T
Octave Enhanced TDY12204.WAV

5.6.4 Peak Picking

For each time-slice within the octave enhanced spectrogram, the local frequency maxima are identified through a “peak picking” process. For each frequency band within a given spectrogram time-slice, the amplitude of the frequency is compared to those of its neighbours within a fixed viewing window, where the frequency band in question is located at the center of the window. If it is the maximum value in the window, then it is a local maximum. Once all the local maxima are identified, only the ones with the greatest amplitudes are retained as candidates for the location of the formant frequencies. The 3 peaks with the largest amplitudes within a given time-slice are used to estimate of the frequency locations of the first three formants. Figures 5.13-M and 5.13-T illustrate the peaks and initial formant estimates resulting from this technique.

5.6.5 Formant Screening

The screening process rejects time-slices from the spectrogram which have erroneous formant frequency estimates. If the first formant frequency is estimated above 1200 Hz or if there are not three formant estimates for a given time-slice, then the time-slice is ignored. What remains is a series of “legal” formant estimates, which are then stored as formant vectors. All formant files are given the initial letter “F” and reflect the name of the spectrogram file from which they were produced (e.g., spectrogram file SDY12204.MAT is used to create formant file FDY122042.MAT). Figures 5.14-M and 5.14-T illustrate the formants remaining after the screening process. Figures 5.15-M and 5.15-T show the final formant vectors that are saved to file.

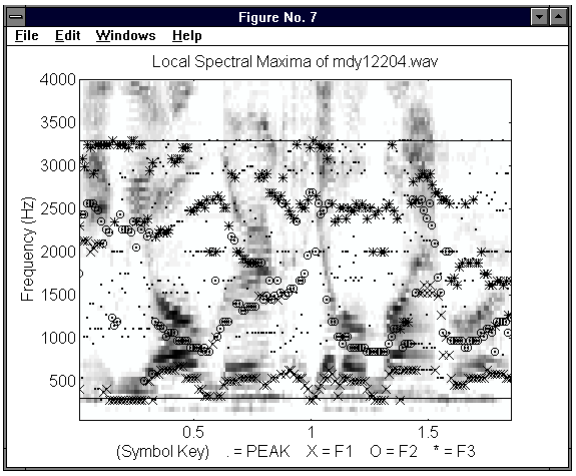


Figure 5-13-M
Local Maxima in MDY12204.WAV

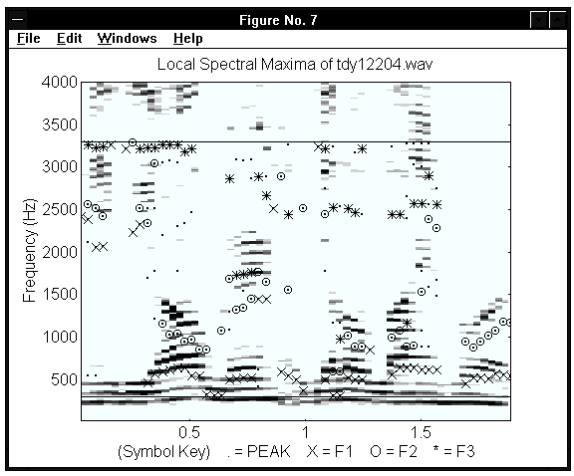


Figure 5.13-T
Local Maxima in TDY12204.WAV

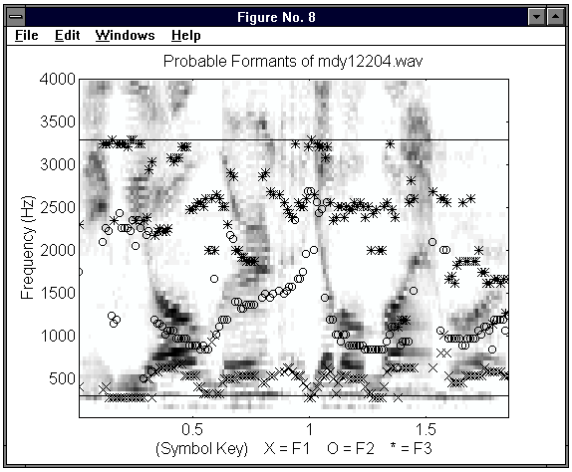


Figure 5.14-M
Formants in MDY12204.WAV

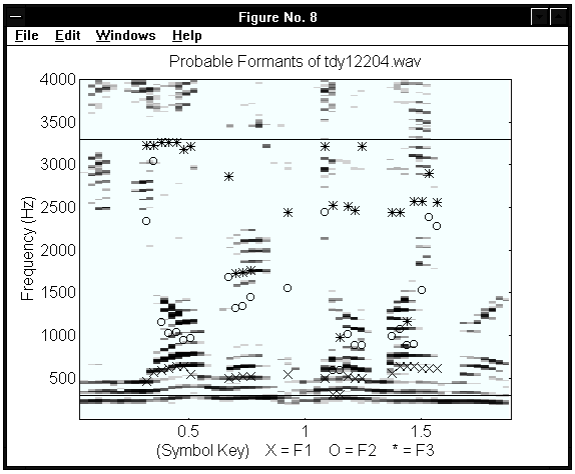


Figure 5.14-T
Formants in TDY12204.WAV

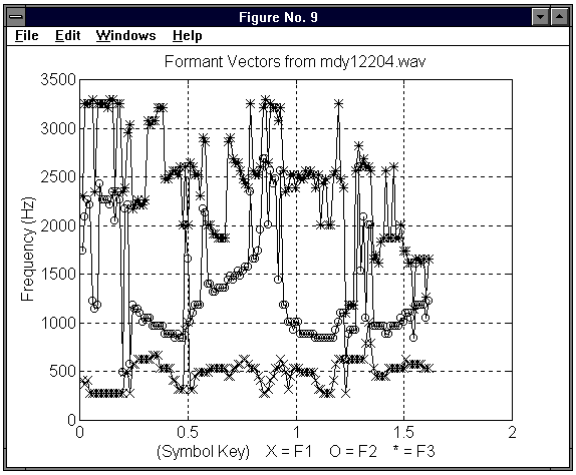


Figure 5.15-M
Formant Vectors in MDY12204.WAV

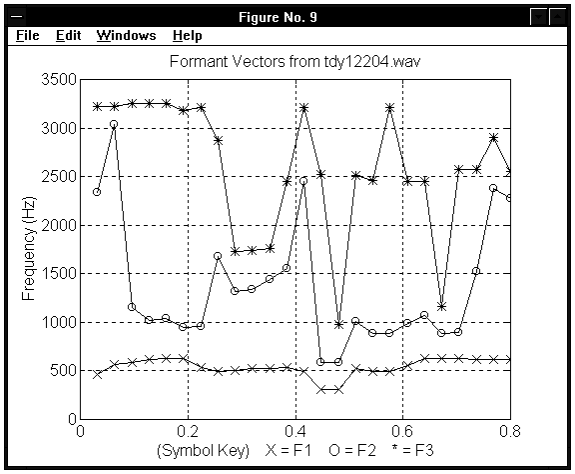


Figure 5.15-T
Formant Vectors in TDY12204.WAV

5.7 Feature Representation

In order to input the extracted formant vectors to the Fuzzy ART Map neural network, they must be combined into a single vector of fixed length, and it must be normalized within the range of fuzzy values. Several representations of the formant vectors were tested, including:

- Tri Formant Vector (TFV),
- Bi Formant Ratio Vector (BFRV),
- Normalized Polar Angle (NPA), and
- Tri Formant Ratio Vector (TFRV).

Each of these feature vector representations is expanded upon in the following discussion. As before, both microphone and telephone speech samples are illustrated to compare the relative effectiveness of each representation technique.

5.7.1 Tri Formant Vector (TFV) Representation

A TFV representation, given by equation 5.2, was created by concatenating the three formant vectors together. The TFV was then normalized by dividing by 3300 Hz (the maximum possible frequency value in the formant extraction process). Finally, **LTW (linearly time warping)** was performed to stretch or compress the vector to a standard length of 500 units. The main disadvantage of this representation is that it is relatively speaker age and gender dependent due to the use of absolute frequency values to represent the phonemes spoken in the speech sample. Figures 5.16-M and 5.16-T illustrate the normalization and LTW of this representation.

$$\mathbf{TFV} = [\, F_3, F_2, F_1 \,] \qquad (5.2)$$

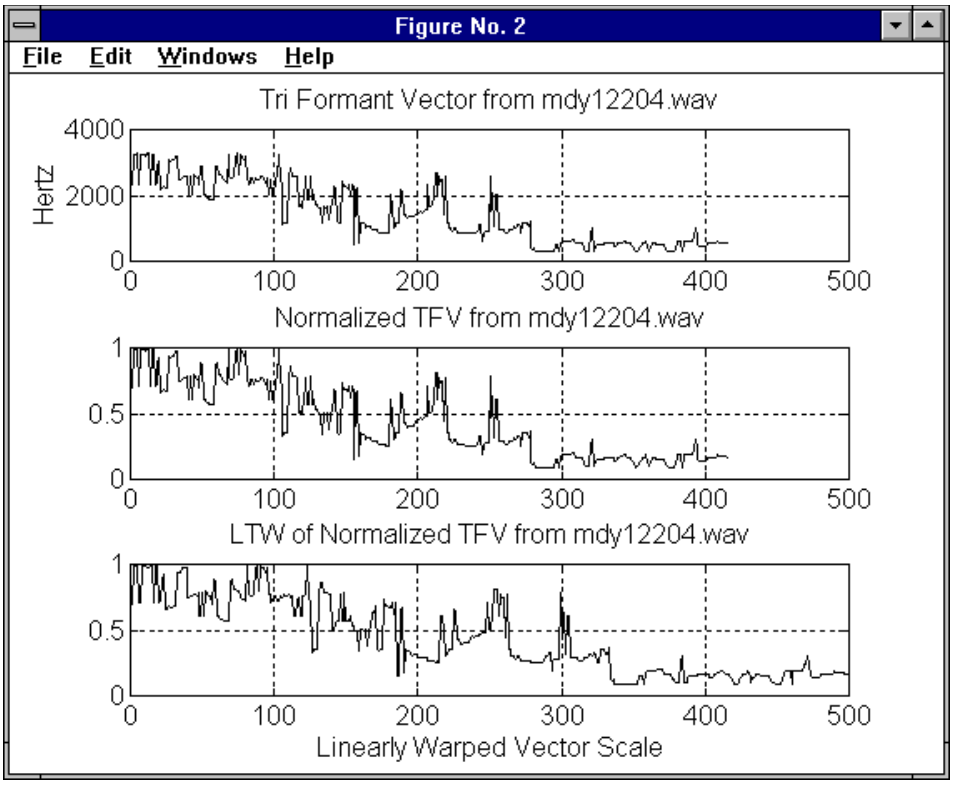


Figure 5.16-M Tri Formant Vector Representation of MDY12204.WAV

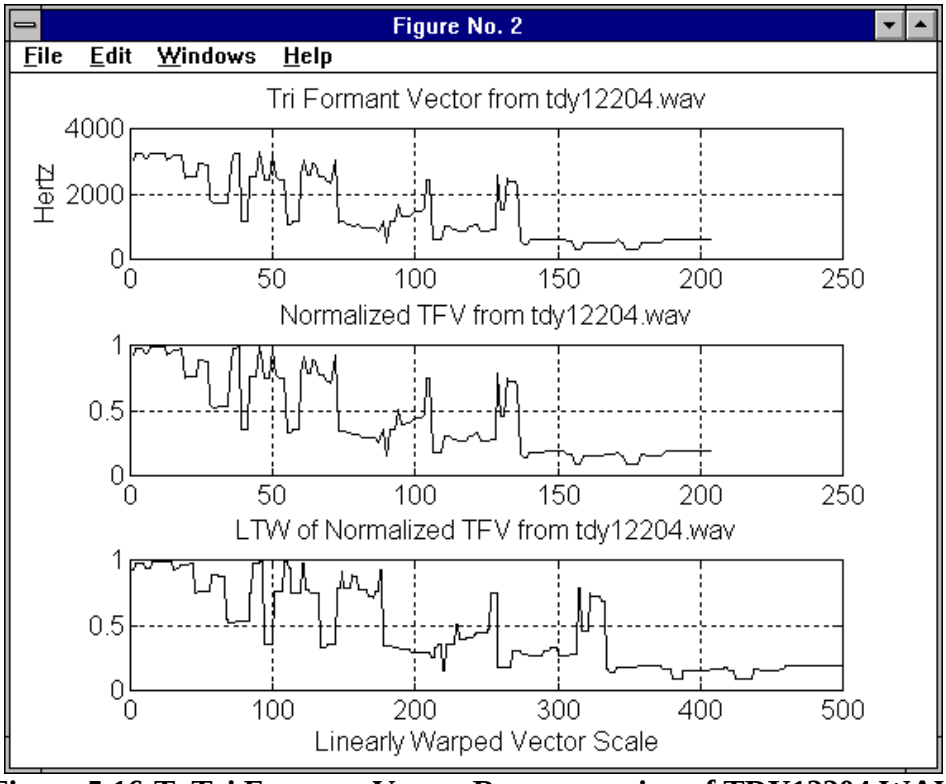


Figure 5.16-T Tri Formant Vector Representation of TDY12204.WAV

5.7.2 Bi Formant Ratio Vector (BFRV) Representation

The BFRV, given by equation 5.3, was constructed from the concatenation of two formant ratio vectors. The standard length used to perform LTW for the BFRV representation was only 300 elements, which substantial reduced memory storage needs compared to that of the TFV representation. As well, the use of formant ratios dramatically reduced speaker dependence. The technique used to normalize the BFRV is discussed in the next section.

$$\mathbf{BFRV} = [\ (F_3/F_2)\ ,\ (F_2/F_1)\ ] \qquad (5.3)$$

5.7.3 Normalized Polar Angle (NPA) Representation

The formant ratios shown above represent the slope of a line between the origin and the intersection point of the numerator and denominator in Cartesian coordinates. Experimentation revealed a non-uniform distribution of the formant ratios slopes numbers for English phonemes. As well, the normalization of such slope numbers to fuzzy range values was difficult (an arbitrarily high value had to be chosen). These problems were not evident when the formant ratios were expressed with polar coordinates. Separation between the formant ratios of different phonemes was improved and the maximum polar angle (90 degrees or 0.5π radians) made normalization easier. The conversion of the BFRV from Cartesian to NPA form is given by equation 5.4. The effect this conversion has on the BFRV representation can be seen in figures 15.17-M and 5.17-T.

$$\mathbf{NPA(BFRV) = \arctan(BFRV) / 0.5\pi} \qquad (5.4)$$

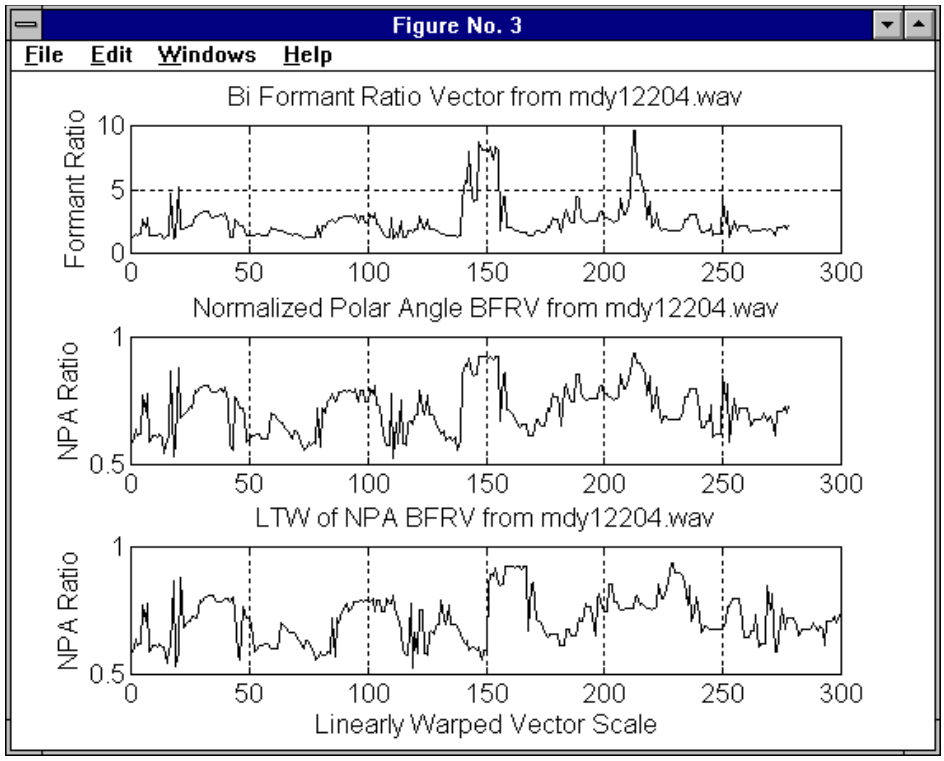


Figure 5.17-M Bi Formant Ratio Vector Representation of MDY12204.WAV

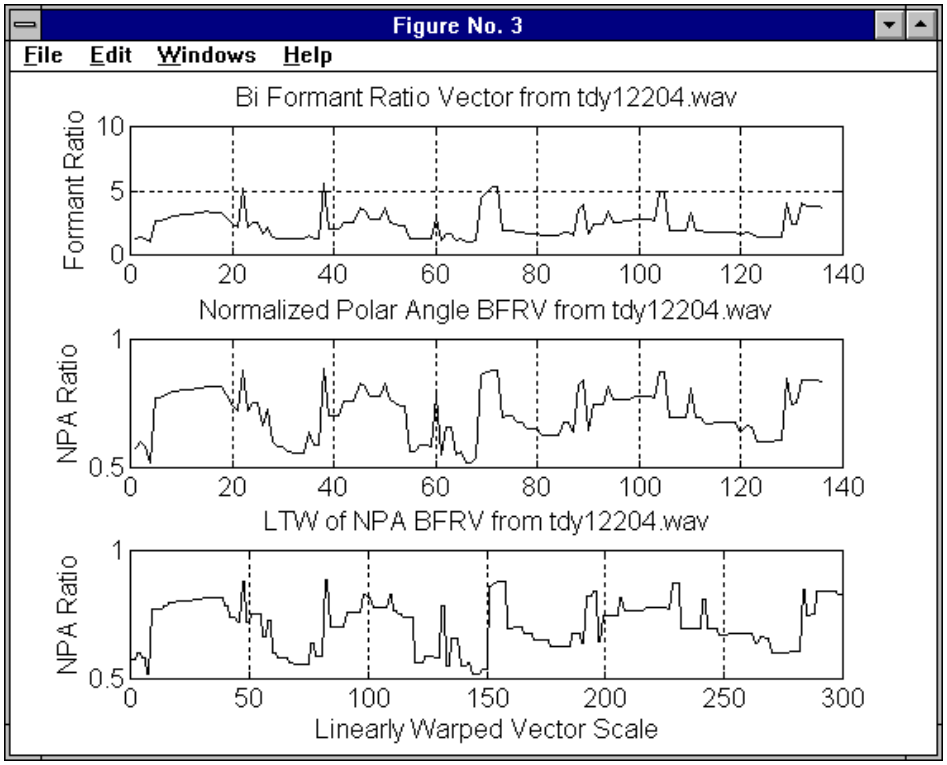


Figure 5.17-T Bi Formant Ratio Vector Representation of TDY12204.WAV

5.7.4 Tri Formant Ratio Vector (TFRV) Representation

The addition of a third formant ratio [F_3 /F_1] vector to the BFRV (equation 5.3) produces the TFRV representation given by equation 5.5. This extension to the BFRV improves the distinction between feature vectors representing similar sounding vowels. An example given in [Mil89] is the vowel pair AA (hot) and AO (caught). Another example is UH (full) and UW (good). Figure 4.2 illustrates how the $\mathbf{F_3/F_1}$ ratio helps discriminate these vowels compared to the use of only the $\mathbf{F_3/F_2}$ and $\mathbf{F_2/F_1}$ ratios (as in the BFRV). The TFRV and its NPA representation are shown in figures 5.18-M and 5.18-T.

$$\mathbf{TFRV = [\ (F_3/F_2),\ (F_2/F_1),\ (F_3/F_1)\ ]} \qquad \mathbf{(5.5)}$$

5.8 Stage #4 -- Utterance Classification

In this stage, a Fuzzy ART Map ANN is used to classify the vectors representing the speech samples that were produced in stage #3. Figures 5.19-M and 5.19-T present the classification process in three plots. The first plot shows the TFRV representation of the speech sample that is presented to the network. The second plot shows the matching vector retrieved by the Fuzzy ART Map ANN in response to the network input. The last plot in each figure illustrates an overlay comparison of the two matching vectors for inspection.

Figure 5.20 summarizes the ASR system software components responsible for carrying out the four stages of the ASR process. The program code for the *Stdemo* script files is included in annex B. The program code for the *Matlab* script files can be found in annexes C, D, and E.

The remainder of this section describes the design of software that simulates the neural network and the settings of the network parameters used to achieve various results. Network modifications, training parameters, and information retrieval are discussed.

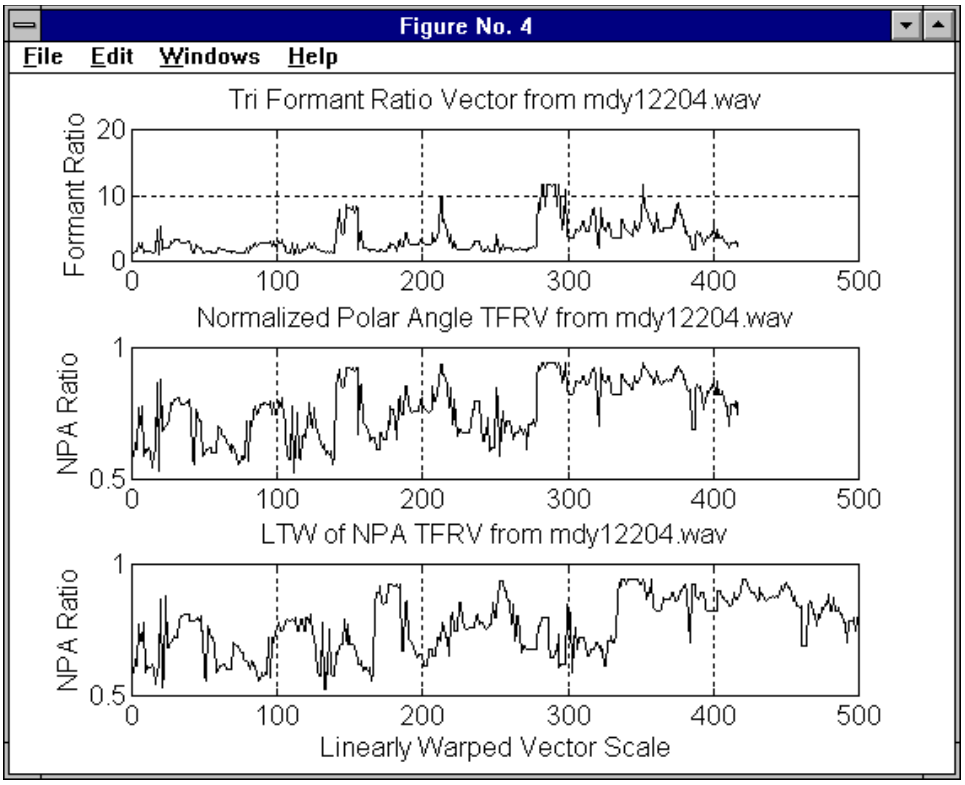


Figure 5.18-M Tri Formant Ratio Vector Representation of MDY12204.WAV

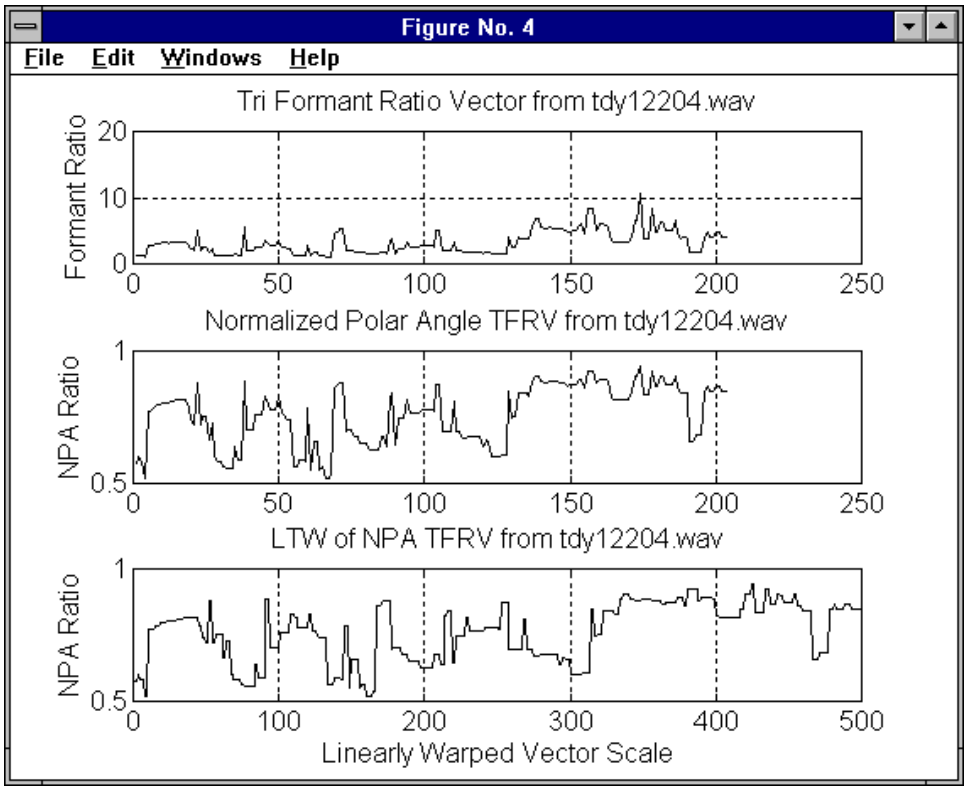


Figure 5.18-T Tri Formant Ratio Vector Representation of TDY12204.WAV

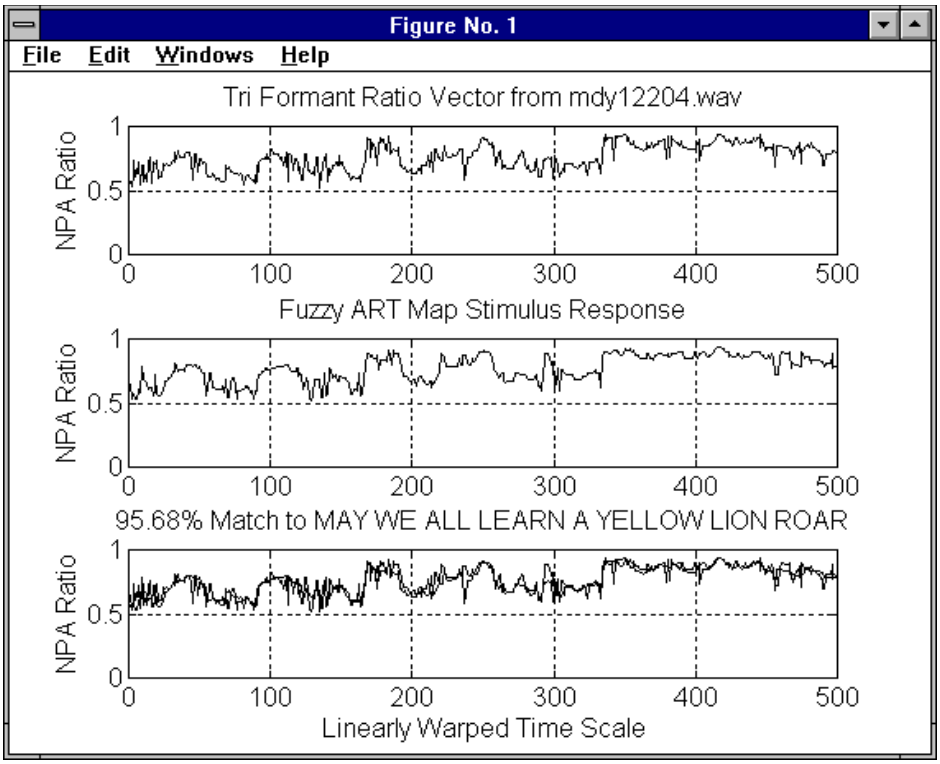


Figure 5.19-M Fuzzy ART Map Recognition of MDY12204.WAV

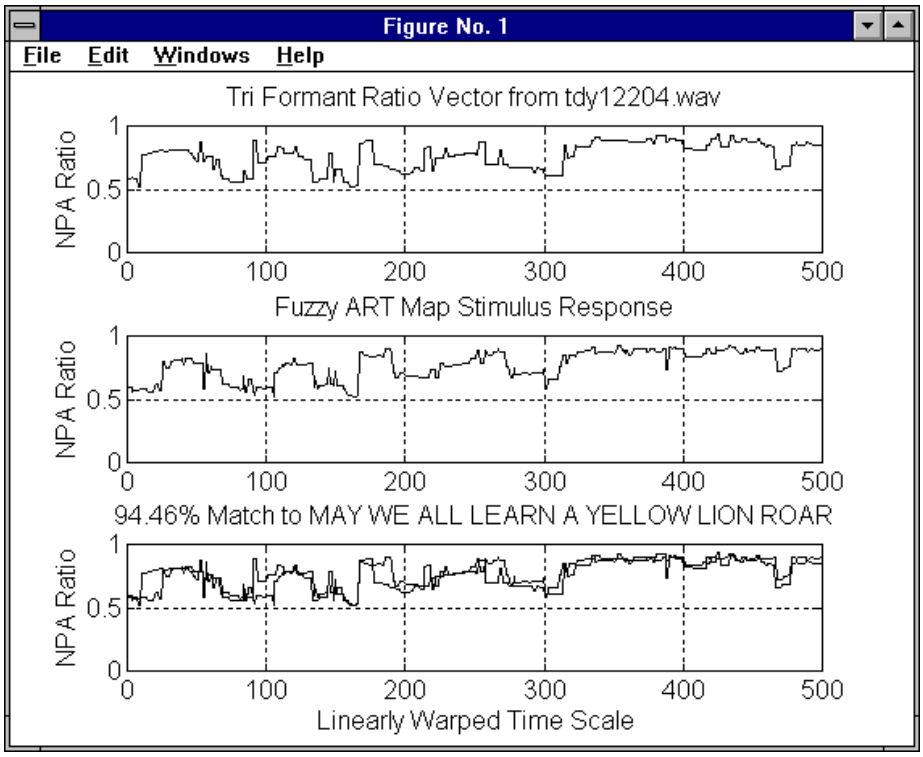


Figure 5.19-T Fuzzy ART Map Recognition of TDY12204.WAV

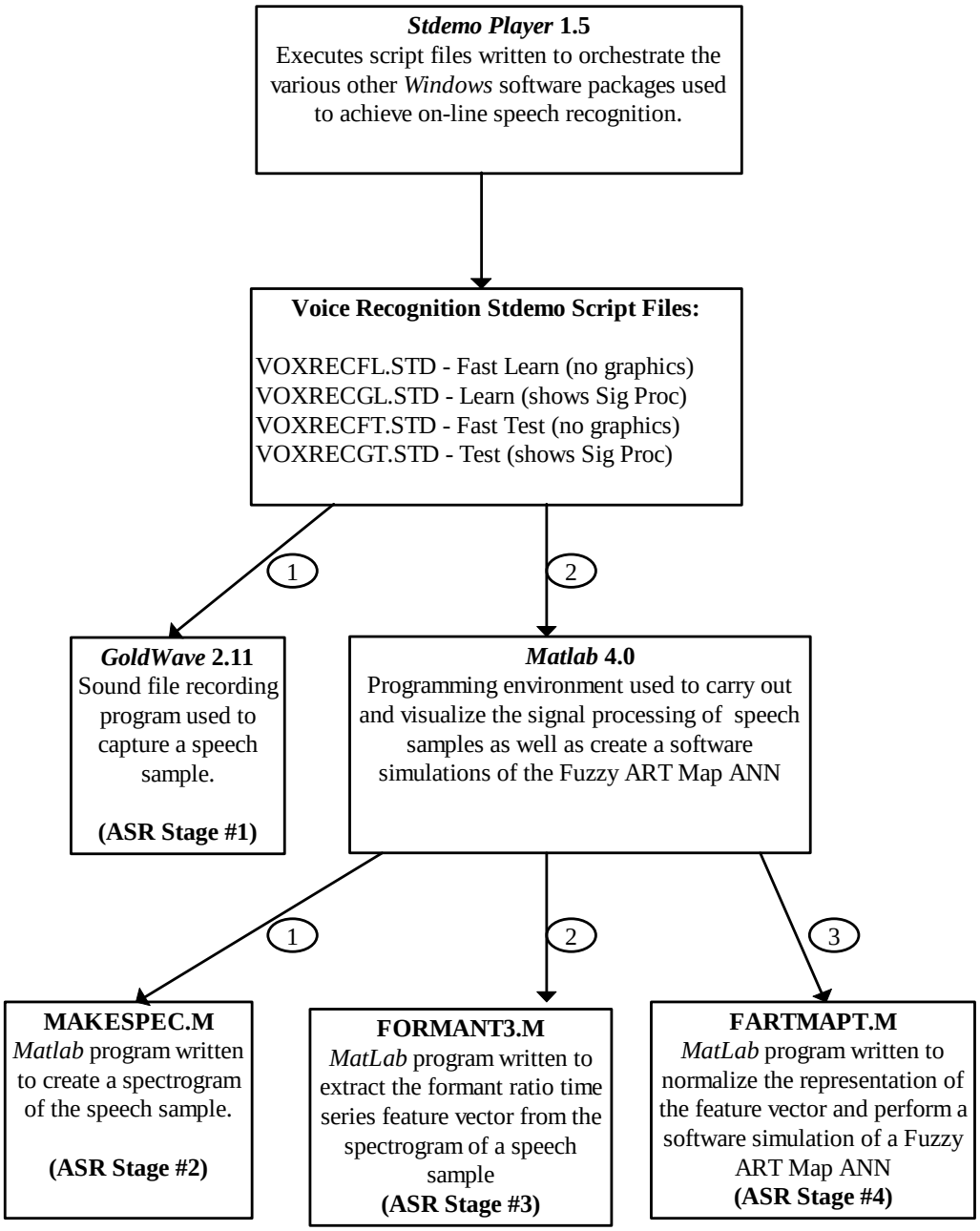


Figure 5.20 ASR System Software Components

5.8.1 Modifications to the Fuzzy ART Map ANN

The feature classifier is implemented in the program FARTMAPT.M. It is based on the description of the Fuzzy ART Map neural network in [Car92]. Since the architecture is being simulated in software, certain simplifications of the processing algorithm were possible. The first modification involves the "supervising" Fuzzy ART_B module which produces the target output for the Fuzzy ART_A module. It has been replaced by a mapping matrix which serves to label the output nodes of the Fuzzy ART_A module with the corresponding desired target vector $\boldsymbol{b}$. Figure 5.21 is a reproduction of figure 3.4 from [Car92] which has been modified to reflect this simplification.

In figure 5.21, the vector $\boldsymbol{a}$ represents the speech feature vector being classified by the Fuzzy ART module while vector $\boldsymbol{b}$ represents the correct classification label of vector a . When vector $\boldsymbol{a}$ causes an output node to become committed, the corresponding row of the mapping matrix is initialized with the value of vector $\boldsymbol{b}$, thus effectively labeling the output node of the Fuzzy ART module.

A second modification addresses the problem of identifying adequate network memory capacity when initializing the size of the network weight matrix. Due to the local manner in which the LTM traces are stored in the network memory , the dynamic expansion of the network memory during training is possible. The ANN program was modified to append 10 new columns to the network weight matrix when it began to reach full capacity.

5.8.2 Network Training

The simplification described above is possible because of the Fuzzy ART modules parameters are set for fast learning and no recoding of output categories. This means that the centroids of new clusters are defined by the vector that caused its creation and that the LTM weight remain fixed thereafter. The stability that results enables the rapid single epoch learning of training sets, since there is no adaptation or "drift" of the cluster centroid with the

exposure to subsequent input. In light of its fixed nature in this implementation, a more apt description for the cluster centroid would be a **classification landmark**, since it does not reflect the center of a cluster of points in feature space. The landmark vector defines the center of a classification zone whose radius is defined by the vigilance parameter distance measure.

is

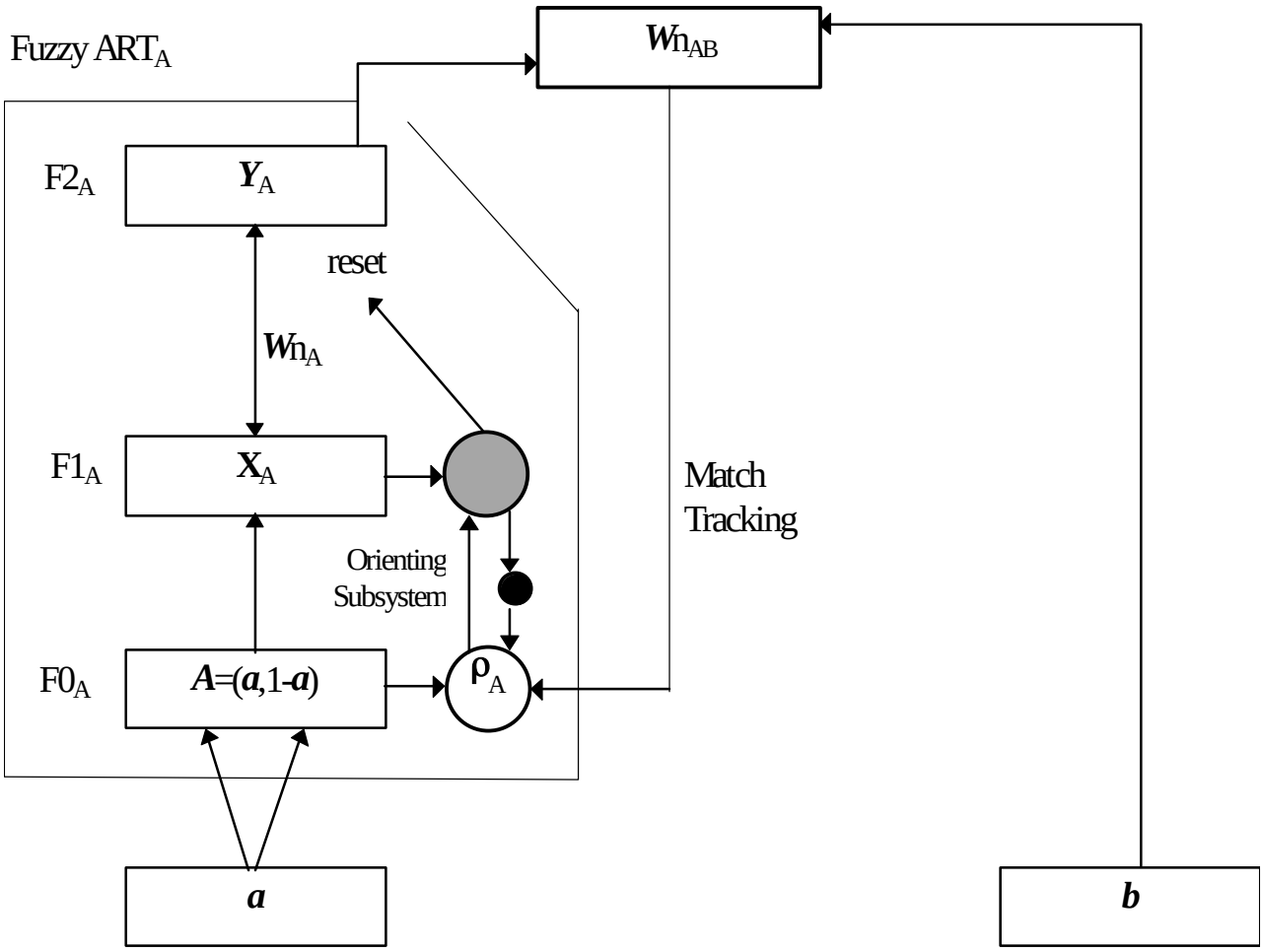


Figure 5.21 Modified Fuzzy ART Map ANN

automatically considered a new landmark based solely on its distance from all other

landmarks. In the second scenario, a new feature vector defines a point within a conflicting classification zone or several overlapping zones (where the zone size is defined by the baseline value of the vigilance parameter). In the second case, the network automatically begins incrementing the baseline vigilance parameter in a **match tracking** process that identifies and compares the classification of the closest landmark to that of the new feature vector. A classification mismatch results in the formation of a new landmark. This learning process is efficient in that it only stores those vectors required to define the boundary areas between the different classifications in feature space. The number of landmarks required to map out these boundaries is directly proportional to the similarity of the classifications to be learned.

5.8.3 Information Retrieval

The network recognizes feature vectors based on the nearest landmark defined in the networks memory. Since the network must compare the feature vector under consideration to all of the landmark vectors stored in its memory map, the speed of information retrieval is decreased as the size of the network memory grows. Due to the trade-off of speed and accuracy that comes with increased network training and increased memory size, it is important that the feature vectors be represented to the network in a manner that minimizes vector size and maximizes inter-classification distances. The aim is to require fewer vectors to define the landmarks of the classification boundaries in feature space, and to reduce the memory required to store each vector. The judicious selection of speech features that can efficiently represent the linguistic message is important here. The use of the TFRV to identify phoneme content is an example of such a representation.

5.9 Chapter Summary

This chapter investigates the applicability of a Fuzzy ART Map ANN to the ASR task through the implementation of an ASR system that incorporates a Fuzzy ART Map ANN as its recognition component. The system is capable of performing speaker and language

independent discrete phrase recognition over public telephone lines, based on the time series of formant ratios. This ASR system is presented within the framework of the four stage ASR process presented earlier in chapter four:

- First, the **sound recording stage** describes the apparatus and method used to capture the speech signal as well as the creation and storage of the RMC Speech Database that was subsequently used to train and test the ASR system;

- Second, the **signal processing stage** deals with the spectral analysis of the recorded speech samples and the production of their spectrogram representations;

- Third, the **feature extraction stage** illustrates the steps required to identify the formants in the spectrogram representations produced in stage two and also deals with the representation of these features to the Fuzzy ART Map ANN for recognition. Both microphone and telephone recorded speech are shown to compare the effectiveness of the techniques used in both transmission channels; and

- Fourth, the **utterance classification stage** discusses the Fuzzy ART Map ANN program regarding software simplifications, the tuning of the training parameters, and its information retrieval characteristics.

This chapter represents the core work of this thesis. It explains the rationale for the design decisions incorporated in the Fuzzy ART Map ASR system (the source code for the software components is contained separately in the annexes). The effectiveness of this system is tested next in chapter six.

CHAPTER SIX

EXPERIMENTAL RESULTS

6.1 Introduction

This chapter deals with the testing and evaluation of the ASR System. It begins with the testing goals and the control measures used to achieve them. This is followed by a listing of the experimental test results. The confusion matrix format is used to present the recognition rate for each phrase as well as a combined average for all eight phrases. An analysis of the recognition errors is provided in the final section, which evaluates the test results in light of the testing goals.

6.2 Testing Goals

One of the goals of testing was to measure the recognition accuracy of the ASR system and determine the how well it performed over a range of conditions, including:

- Speaker dependency (speaker dependent vs. speaker independent),
- Transmission channel (microphone recordings vs. telephone recordings), and
    - Feature representation (Bi Formant Ratio Vector vs. Tri Formant Ratio Vector).

It was also desirable to be able to directly assess the contribution of that the Fuzzy ART Map ANN had as the recognition component of the ASR system. To accomplish this, a well-known ANN would be used as a benchmark recognizer and substituted into the ASR system. It would be tested under the same conditions as the ANN it replaced. Finally, an analysis of the recognition errors was needed to identify the strengths and weaknesses of the present system and possibly isolate sources of error to improve system performance in future

work. The following paragraphs describe the control measures and test methods used to achieve these goals.

6.2.1 Testing Control Measures

To increase confidence in the test results, the tests were designed to be consistent, impartial and as realistic as possible. The following control measures were observed for all tests:

- Only the RMC Speech Database was used as speech material;
- The same training and test sets were used in all tests;
- No overlap of (identical) phrases occurred between the training and test sets;
- Set phrases were chosen chronologically (e.g., training set phrases predate those in test set);
- No phrases were thrown out to improve results; and
    - Only the speakers who had recorded a minimum of 40 phrase samples were used for testing.

6.2.2 Testing Methods

To satisfy the test goals, the recognition accuracy of the ASR system was tested under every permutation of the three (binary) conditions of interest; namely, speaker dependency, transmission channel and feature representation. This produced a total of eight tests. The benchmark testing (described below) was carried out separately using only microphone recorded speech converted to the BFRV representation. The speaker dependency testing of the benchmark ANN produced an additional two tests. The results of all ten test sessions are presented in section 6.3, after the following description of testing methods.

Speaker Dependency Testing

In speaker dependent testing, only the speech samples from the speaker under test were used to train and test the ASR system. Half of the total recorded samples from the test speaker were used to train the network, while the other half was used to test the network. Because two speakers did not record an even number of sessions, the numbers of test phrases are not uniformly distributed amongst each phrase type. To provide statistically meaningful performance measurements, the test results from each of the eleven speakers have been combined and are presented in one confusion matrix.

In speaker independent testing, each of the speakers contributed five tokens of the eight phrases (numbered 2 through 9). The ASR system was trained on the speech samples from the first ten speakers, for a total of 400 training samples. The 40 phrases from the eleventh speaker were used to test the system. Eleven such tests were conducted by rotating each of the speakers through the test speaker position and training the ASR system using the other ten.

Transmission Channel Testing

The RMC Speech Database, which was batch recorded using a high quality microphone at a sampling rate of 22050 Hz with 16 bit resolution, was used for all microphone channel speech samples.

To assess the ability of the ASR system to perform phrase recognition over real public phone lines, the RMC Speech Database was played into a telephone handset and digitally recorded (on-line) at the other end of a local telephone call using an electromagnetic telephone pickup. A sampling rate of 8000 Hz with 16 bit resolution was used to lower disk space requirements while maintaining a frequency resolution greater than 3300 Hz (the upper limit of the telephone channel). This **telephone filtered** version of the RMC Speech Database was then used for all telephone channel speech samples.

Feature Representation Testing

The effectiveness of the BFRV (Bi Formant Ratio Vector) representation of speech samples is compared to that of the TFRV (Tri Formant Ratio Vector) representation developed in chapter five. This latter representation was discovered late in the testing schedule. Due to time limitations, benchmark testing (described below) using the TFRV representation was left to future work.

Benchmark Testing

A backpropagation trained neural network was selected as the benchmark recognizer used to replace the Fuzzy ART Map component of the ASR system. More precisely, the network was a multi-layer feedforward ANN utilizing the backpropagation learning algorithm augmented by a momentum term and an adaptive learning rate. The network consisted of an input layer of 300 nodes (the size of the BFRV input vector), a hidden layer of 50 nodes (found to provide optimal performance) with a tan-sigmoid activation function, and an output layer of 8 nodes (one for each phrase to be recognized) with a linear activation function.

Nguyen-Widrow initial conditions were used to provide the best hope of achieving a good set of starting weights for the network. Several sets of initial conditions were tested and the set that produced the quickest convergence was kept and used to initialize the network for all training. Due to time constraints, each training session was limited to 1000 cycles (epochs) of the training set. The source code for the ANN programs used to simulate this network is listed in Annex F (BPMAPPER.M) and Annex G (BPTESTER.M).

6.3 Test Results

This section presents the experimental results of ten test sessions. The benchmark testing results are presented first, and compare the speaker dependency performance of the backpropagation ANN using only microphone speech samples and the BFRV representation. Table 6.1 gives the speaker dependent results and table 6.2 lists those for the speaker independent test case.

The remaining eight test sessions are performed on the ASR system with the Fuzzy ART Map ANN as the recognizer component. For these last eight sessions, the transmission channel testing results (microphone vs. telephone) are presented side by side to carry on the precedent set in chapter five. Again, the suffix “M” and “T” are used to denote microphone and telephone speech samples respectively. The tables are organized with the BFRV representation results presented first, and the more accurate TFRV representation last, as shown below:

- Tables 6.3-M and 6.3-T BFRV Representation / Speaker Dependent
- Tables 6.4-M and 6.4-T BFRV Representation / Speaker Independent
- Tables 6.5-M and 6.5-T TFRV Representation / Speaker Dependent
    - Tables 6.6-M and 6.6-T TFRV Representation / Speaker Independent

The recognition rates for each session are summarized in table 6.7 at the end of the section.

Recognized → ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**69**		**2**	**1**		**5**			**69 / 77**
3		**76**		**1**					**76 / 77**
4			**74**	**1**	**1**	**1**			**74 / 77**
5			**1**	**72**	**1**	**1**		**2**	**72 / 77**
6	**2**		**3**		**59**	**2**	**5**	**8**	**59 / 79**
7	**2**			**1**		**76**			**76 / 79**
8						**2**	**77**		**77 / 79**
9				**6**	**3**		**1**	**69**	**69 / 79**
Total Output	**73**	**76**	**80**	**82**	**64**	**87**	**83**	**79**	**572 = 91.7% 624**

Table 6.1 Benchmark ANN / BFRV / Microphone / Speaker Dependent

Recognized→ ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**51**		**2**			**1**	**1**		**51 / 55**
3	**1**	**52**					**1**	**1**	**52 / 55**
4		**1**	**52**	**2**					**52 / 55**
5			**1**	**54**					**54 / 55**
6	**1**		**1**		**47**		**3**	**3**	**47 / 55**
7	**2**	**2**				**48**	**2**	**1**	**48 / 55**
8		**1**			**5**	**1**	**46**	**2**	**46 / 55**
9		**4**		**1**	**4**	**3**	**2**	**41**	**41 / 55**
Total Output	**55**	**60**	**56**	**57**	**56**	**53**	**55**	**48**	**391 = 88.7 % 440**

Table 6.2 Benchmark ANN / BFRV / Microphone / Speaker Independent

Recognized → ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**76**		**1**						**76 / 77**
3		**76**	**1**						**76 / 77**
4			**77**						**77 / 77**
5				**77**					**77 / 77**
6					**69**		**4**	**6**	**69 / 79**
7		**2**		**1**		**75**	**1**		**75 / 79**
8							**79**		**79 / 79**
9				**2**				**77**	**77 / 79**
Total Output	**76**	**78**	**79**	**80**	**69**	**75**	**84**	**83**	**606** = **97.1%** **624**

Table 6.3-M Fuzzy ART Map / BFRV / Speaker Dependent / Microphone

Recognized → ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**71**		**1**			**3**	**1**	**1**	**71/77**
3	**1**	**73**	**1**	**1**				**1**	**73/77**
4			**75**		**1**			**1**	**75/77**
5		**2**		**74**	**1**				**74/77**
6	**1**		**1**	**2**	**59**		**12**	**4**	**59/79**
7	**2**	**2**		**3**	**2**	**64**	**1**	**5**	**64/79**
8	**1**				**2**	**1**	**75**		**75/79**
9	**1**	**4**		**3**	**5**		**8**	**58**	**58/79**
Total Output	**77**	**81**	**78**	**83**	**70**	**68**	**97**	**70**	**549** = **88.0%** **624**

Table 6.3-T Fuzzy ART Map / BFRV / Speaker Dependent / Telephone

Recognized→ ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**54**							**1**	**54 / 55**
3	**1**	**54**							**54 / 55**
4			**53**	**2**					**53 / 55**
5				**55**					**55 / 55**
6				**2**	**47**		**4**	**2**	**47 / 55**
7					**1**	**48**	**2**	**4**	**48 / 55**
8							**51**	**4**	**51 / 55**
9					**1**	**1**	**2**	**51**	**51 / 55**
Total Output	**55**	**54**	**53**	**59**	**49**	**49**	**60**	**61**	**414 = 94.1%** **440**

Table 6.4-M Fuzzy ART Map / BFRV / Speaker Independent / Microphone

Recognized→ ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**46**		**1**			**6**	**2**		**46 / 55**
3		**52**		**2**				**1**	**52 / 55**
4	**2**		**49**	**1**		**3**			**49 / 55**
5			**1**	**53**				**1**	**53 / 55**
6				**1**	**34**	**3**	**12**	**5**	**34 / 55**
7	**3**	**1**			**1**	**45**	**2**	**3**	**45 / 55**
8			**1**		**1**		**52**	**1**	**52 / 55**
9		**1**		**3**	**2**	**1**	**4**	**44**	**44 / 55**
Total Output	**51**	**54**	**52**	**60**	**38**	**58**	**72**	**55**	**375 = 85.2%** **440**

Table 6.4-T Fuzzy ART Map / BFRV / Speaker Independent / Telephone

Recognized → ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**77**								**77 / 77**
3		**76**	**1**						**76 / 77**
4			**77**						**77 / 77**
5				**77**					**77 / 77**
6					**74**		**3**	**2**	**74 / 79**
7	**1**	**1**		**1**		**75**		**1**	**75 / 79**
8							**79**		**79 / 79**
9				**1**				**78**	**78 / 79**
Total Output	**78**	**77**	**78**	**79**	**74**	**75**	**82**	**81**	**613 = 98.2%** **624**

Table 6.5-M Fuzzy ART Map / TFRV / Speaker Dependent / Microphone

Recognized → ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**73**		**1**			**2**		**1**	**73 / 77**
3		**75**	**1**	**1**					**75 / 77**
4		**1**	**75**	**1**					**75 / 77**
5		**1**		**76**					**76 / 77**
6	**1**			**1**	**66**		**6**	**5**	**66 / 79**
7	**1**	**2**		**4**	**1**	**67**	**1**	**3**	**67 / 79**
8		**1**			**1**	**2**	**75**		**75 / 79**
9	**1**	**3**		**1**	**5**		**5**	**64**	**64 / 79**
Total Output	**76**	**83**	**77**	**84**	**73**	**71**	**87**	**73**	**571 = 91.5%** **624**

Table 6.5-T Fuzzy ART Map / TFRV / Speaker Dependent / Telephone

Recognized→ ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**52**	**2**						**1**	**52 / 55**
3	**1**	**54**							**54 / 55**
4			**53**	**2**					**53 / 55**
5				**55**					**55 / 55**
6					**49**	**1**	**2**	**3**	**49 / 55**
7					**1**	**48**	**1**	**5**	**48 / 55**
8					**1**		**51**	**3**	**51 / 55**
9					**1**		**2**	**52**	**52 / 55**
Total Output	**53**	**56**	**53**	**57**	**52**	**49**	**56**	**64**	**414 = 94.1%** **440**

Table 6.6-M Fuzzy ART Map / TFRV / Speaker Independent / Microphone

Recognized→ ↓Spoken	2	3	4	5	6	7	8	9	Recognition Rate
2	**43**		**1**			**9**	**2**		**43 / 55**
3		**53**		**2**					**53 / 55**
4			**54**			**1**			**54 / 55**
5			**2**	**52**				**1**	**52 / 55**
6				**1**	**39**	**1**	**7**	**7**	**39 / 55**
7	**3**				**3**	**45**		**4**	**45 / 55**
8		**1**			**1**		**50**	**3**	**50 / 55**
9				**1**	**2**	**2**	**3**	**47**	**47 / 55**
Total Output	**46**	**54**	**57**	**56**	**45**	**58**	**62**	**62**	**383 = 87.0%** **440**

Table 6.6-T Fuzzy ART Map / TFRV / Speaker Independent / Telephone

		BFRV Representation		TFRV Representation	
		Speaker Dependent	Speaker Independent	Speaker Dependent	Speaker Independent
Benchmark Recognizer	Microphone Recording	**91.7%** (572/624)	**88.7%** (391/440)	**not tested**	**not tested**
Fuzzy ART Map Recognizer	Microphone Recording	**97.1%** (606/624)	**94.1%** (414/440)	**98.2%** (613/624)	**94.1%** (414/440)
	Telephone Recording	**88.0%** (549/624)	**85.2%** (375/440)	**91.5%** (571/624)	**87.0%** (383/440)

Table 6.7 ASR System Testing Summary

6.4 Evaluation Of Results

This section provides and evaluation of the results of the ten test sessions presented in the previous section. Observations are made with respect to the points of concern outlined in the testing goals, restated here for the reader's convenience:

- Benchmark comparison (Fuzzy ART Map ANN vs. Backpropagation Network),
- Speaker dependency (speaker dependent vs. speaker independent),
- Transmission channel (microphone vs. telephone),
- Feature representation (BFRV vs. TFRV), and
- Error Analysis (Why does network fail - explain cause of errors).

6.4.1 Benchmark ANN Performance Comparison

Compared to the backpropagation ANN recognizer, the Fuzzy ART Map ANN recognizer required less time to train was more accurate at recognizing phrases. The combined recognition rate (two tests: speaker dependent & independent) for the benchmark network was only 90.5%, while the Fuzzy ART Map achieved a combined rate of 95.9% (an increase of 5.4%).

The size of the training set directly affected the training time for both ANN recognizers. The worst case was the speaker independent training sets, which involved 400 training samples each. Running on the hardware described in chapter 5, the backpropagation ANN recognizer required nine hours to complete 1000 epochs. In comparison, the Fuzzy ART Map recognizer required only 15 minutes to learn the same 400 patterns. Figures 6.1 and 6.2 illustrate a sample speaker independent training session using the backpropagation ANN recognizer.

6.4.2 Speaker Dependency Performance Comparison

The speaker dependent recognition performance was superior to speaker independent performance in all test sessions. The combined recognition rate (four tests: BFRV & TFRV, microphone & telephone) for the speaker dependent testing was 93.7% while that of the speaker independent testing was 89.8% (a 3.9% drop).

6.4.3 Transmission Channel Performance Comparison

The microphone channel recognition performance was better than that of the telephone channel in all tests. The difference in recognition rates between the two channels was, on average, 2% less using the TFRV representation compared to the BFRV representation. The combined recognition rate (four tests: speaker dependent & independent,

BFRV & TFRV) for the microphone tests was 96.2% compared to 88.3% for the telephone tests (a drop of 7.9%).

6.4.4 Feature Representation Performance Comparison

The TRFV representation consistently improved the performance of the ASR system compared to the use of the BFRV representation, with one exception. The recognition rate for speaker independent microphone recorded speech was the same (94.1%). The combined recognition rate (four tests: Speaker dependent & Independent, microphone & telephone speech) using the BFRV representation of the speech signal was 91.4% while the use of the TFRV representation improved this to 93.1% (up 1.7%)

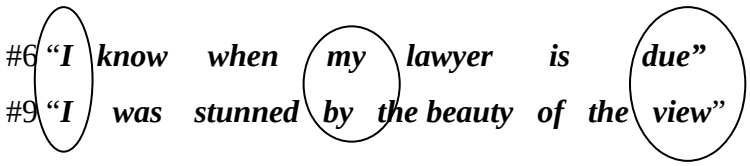
6.4.5 Error Analysis

The largest single drop in performance occurred between the microphone and telephone channel. This can be attributed to the octave gain formula (equation 5.1) that was developed experimentally using only microphone speech. Without proper frequency amplitude normalization, the efficiency of the remainder of the formant extraction process is seriously degraded. The development of a new normalization technique appropriate to the telephone channel would improve the telephone channel recognition rate.

The confusion matrices point out higher error rates between phases that contain similar vowel sounds in relatively the same positions in the sentence. For example, phrase number 6 is often confused with phrase number nine:

#6 “***I know when my lawyer is due”***
#9 “***I was stunned by the beauty of the view***”

Since phrase comparisons are based on a linearly time warped representation of voiced phonemes extracted from each phase, the phrases may be transformed to look something like this:



The time alignments of similar sounds in the two phrases are circled. The variability of each individuals speech phrases contributes to this error. It was found that the error rates were lower for speakers who spoke in a consistent manner (e.g., constant tempo and pronunciation).

The last major source of error was the use of one standard noise mask for all speech samples. Keeping in mind the RMC database was recorded over a four month period, the background noise on each sample could vary considerably. The same is true for the telephone filtered version of the speech files. This practice was carried out only to simplify test processing using batch recordings. The on-line demo version of the ASR system captures a background noise sample for each recording making the noise masking technique far more effective. The testing of the ASR system in an on-line manner is left as an item for future work.

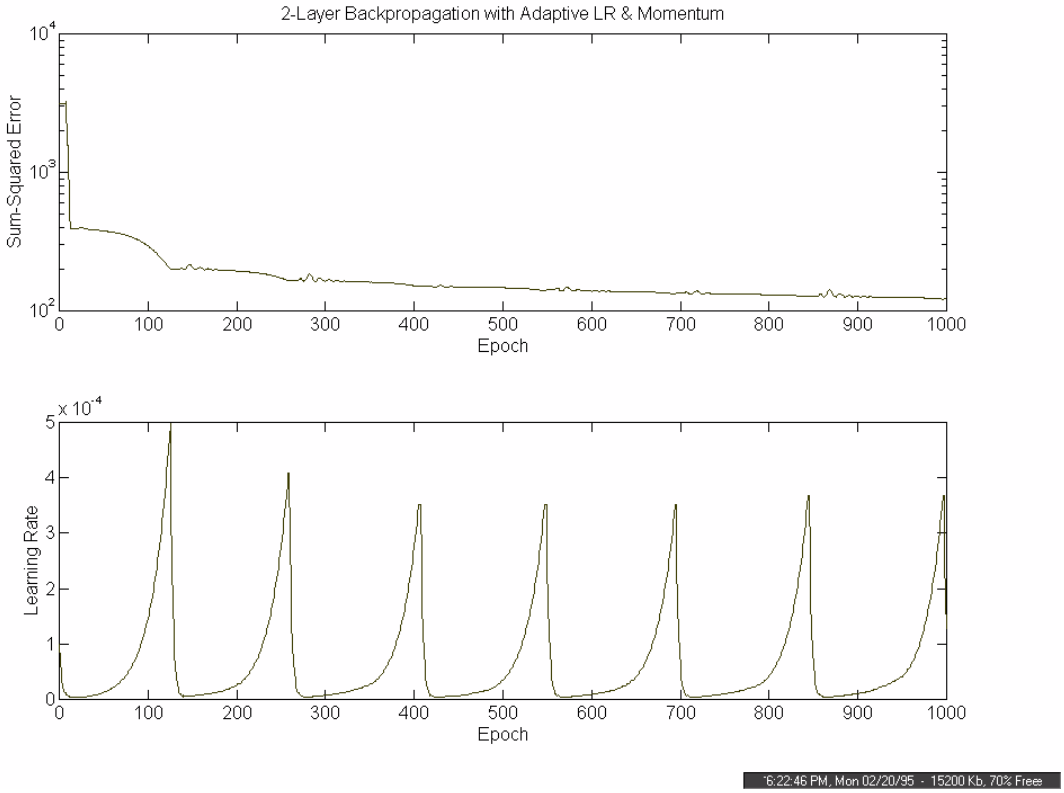


Figure 6.1 Benchmark ANN -- Sample Training Progress Record

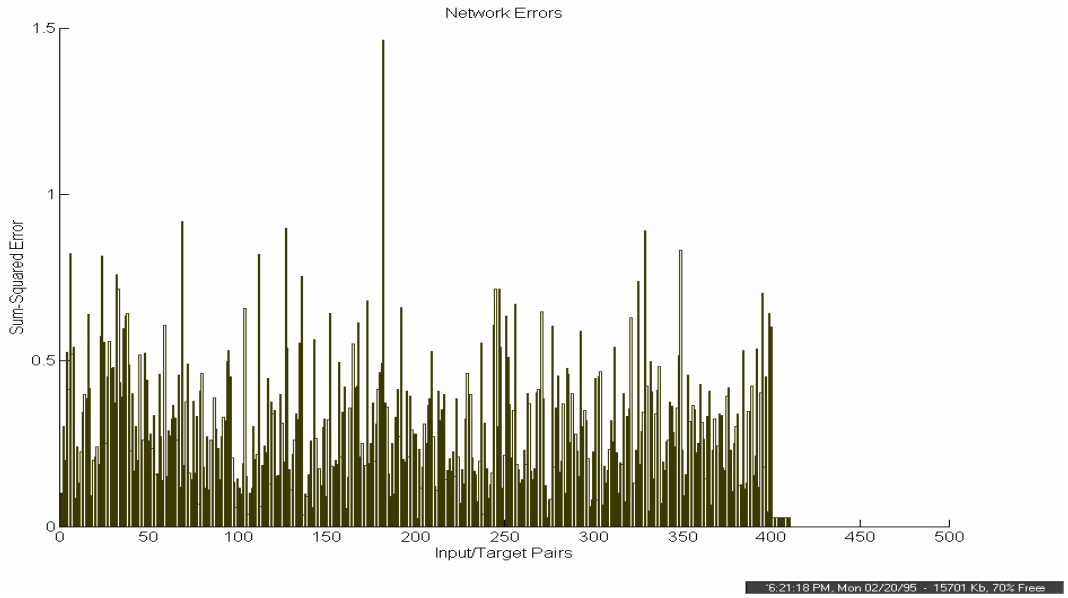


Figure 6.2 Benchmark ANN -- Sample Training Error Summary

6.5 Chapter Summary

This chapter dealt with the testing and evaluation of the ASR System in three main sections, which are recapped below:

- The first section covered the testing goals, testing control measures and a description of the various tests that were performed.

- The second section contained the test results illustrating the ASR systems performance concerning speaker dependency (speaker dependent vs. speaker independent), recording channel (microphone vs. telephone), and feature representation method (BFRV vs. TFRV). A benchmark comparison test was also carried out using a multi-layered feedforward ANN trained using the backpropagation algorithm.

- The last section provided an analysis of the ASR system performance. The main sources of error were identified and possible solutions were proposed. These included the use of an octave enhancement technique specific to the telephone channel, and on-line testing to take advantage of noise masks tailored to each speech sample.

		BFRV Representation		TFRV Representation	
		Speaker Dependent	Speaker Independent	Speaker Dependent	Speaker Independent
Benchmark Recognizer	Microphone Recording	**91.7%** (572/624)	**88.7%** (391/440)	**not tested**	**not tested**
Fuzzy ART Map Recognizer	Microphone Recording	**97.1%** (606/624)	**94.1%** (414/440)	**98.2%** (613/624)	**94.1%** (414/440)
	Telephone Recording	**88.0%** (549/624)	**85.2%** (375/440)	**91.5%** (571/624)	**87.0%** (383/440)

Table 6.7 ASR System Testing Summary (reprinted for reader's convenience)

CHAPTER SEVEN

CONCLUDING REMARKS

7.1 Thesis Summary

An ASR (automatic speech recognition) system was developed which incorporated a Fuzzy ART (Adaptive Resonance Theory) Map ANN (artificial neural network) as the component that classifies and recognizes speech samples presented to the system. The process of converting the spoken acoustic signal into a representation suitable for input to the recognizer component was described in terms of a four stage ASR process which included sound recording, spectral analysis, feature extraction and representation, and utterance classification.

Both microphone and telephone were used in the sound recording stage in order to measure and compare the system's performance on both transmission channels. The acoustic signals were digitized and converted into spectrogram form. Noise reduction, silence reduction and frequency amplitude normalization were carried out prior to tracking the first three speech formants. These formant were subsequently combined in a TFRV (tri formant ratio vector) representation that dramatically reduced the differences between the representation of the speech phrase due to speaker age and gender. The TFRV was then normalize to the range of fuzzy values (from zero to one) and LTW (linearly time warping) was performed to stretch or compress the TFRV to the size of the input layer of the Fuzzy ART Map ANN recognizer.

The combination of the TFRV speech signal representation and the Fuzzy ART Map ANN recognizer produced an ASR system capable of a speaker dependent recognition rate of 98.2% for microphone recorded speech and 91.5% for the same speech samples played over a noisy telephone line. The speaker independent rates were about 4% lower. Testing was

performed using the RMC Speech Database, which contained eight test phrases chosen from the literature for their diverse phonetic representation. The voices of the ten males and one female used in the testing, were recorded in a relatively noisy computer room over a period of four months in order to capture the variability of human speech.

A multi-layer feedforward ANN trained with the backpropagation algorithm (aided by a momentum term and an adaptive learning rate) was used as a benchmark in side by side comparison tests with the Fuzzy ART Map ANN. Its performace was approximately 5% lower than that of the Fuzzy ART Map ANN while the time required to train it was considerably longer.

7.2 Conclusion

The goal of this research was to investigate the applicability of the Fuzzy ART Map ANN to the task of Automatic Speech Recognition. **The development of the ASR system described in Chapter Five demonstrates that the Fuzzy ART Map ANN can be used to perform speech recognition, while the testing documented in Chapter Six establishes that the Fuzzy ART Map ANN can perform ASR more effectively than the ANN used in the benchmark tests** in the following ways:

- Its rapid learning capability **significantly reduced system training time** and produced **more efficient speech recognition**;

- Its localized (vector) data storage s**upports modular packaging and reconfiguration of active vocabulary**;

- Its use of a unit (ones) matrix to initialize its network memory **eliminates the uncertainty associated with random selection of starting weights** required by the benchmark ANN; and

- Unlike the benchmark ANN, whose fixed network size limited its storage capacity, the Fuzzy ART Map ANN **network expands in response to the need to learn "new" data** (i.e., significantly different from that which it already has stored in its memory).

7.3 Future Work

Improvements to the ASR system developed in this thesis stem from the error analysis in section 6.4.5. The development of an frequency amplitude normalization technique that adapts to the spectral attenuation of a given telephone connection would greatly improve the system's recognition performance in that area. Test scores may be improved by using on-line testing to take advantage of noise masks tailored to each speech sample. For telephone speech samples, temporal resolution could be improved to detect rapid changes in the speech signal through the use of a higher sampling rate, similar to that used for microphone recorded speech. Finally, the system could be made to execute faster if it was compiled as an executable program.

REFERENCES

[Ain88] W.A. Ainsworth, Speech Recognition By Machine, Peter Peregrinus Ltd, London, United Kingdom, 1988.

[Alw85] R.G. Alward, A Speech Recognizer Based on Energy and Zero Crossing, M.Eng. Thesis, Royal Military College of Canada, Jun 1985.

[Bar57] H.L. Barney and K.K. Dunn, “Speech Analysis”, pp 180-201 in Manual Of Phonetics , L. Kaiser (ed), North-Holland Publishing Co, Amsterdam, 1957.

[Car87a] Gail A. Carpenter and Stephen Grossberg, "A massively parallel architecture for a self-organizing neural pattern recognition machine”, Computer Vision Graphics and Image Processing, Vol.37, 1987, pp. 54-115.

[Car87b] Gail A. Carpenter and Stephen Grossberg, "ART 2 : Self-organization of stable category recognition codes for analog input patterns",Applied Optics, Dec. 1, 1987,
pp. 4919-4930

[Car88] Gail A. Carpenter and Stephen Grossberg, "The ART of Adaptive Pattern Recognition by a Self-Organizing Neural Network", IEEE Computer, pp 77-88, Mar 1988.

[Car90] Gail A. Carpenter and Stephen Grossberg,”ART3: Hierarchical search using chemical transmitters in self-organizing pattern recognition architectures”, Neural Networks, vol. 3, pp. 129-152, 1990

[Car91] Gail A. Carpenter, Stephen Grossberg, D.B. Rosen, “Fuzzy ART: Fast stable learning and categorization of analog patterns by an adaptive resonance system”, Neural Networks, vol. 4, pp. 759-771, 1991.

[Car92] Gail A. Carpenter, Stephen Grossberg, John H. Reynolds, Natalya Markuzon and David B. Rosen, "Fuzzy ARTMAP: A Neural Network Architecture for Incremental Supervised Learning of Analog Multidimensional Maps", IEEE Trans. on Neural Networks, vol.3, pp 698-713, Sep 1992.

[Col95] Ron Cole et al.,"The Challenge of Spoken Language Systems: Research Directions for the Nineties", IEEE Trans. on Speech and Audio Processing, vol 3, Jan 1995

[CSN95] Andrew Hunt, Barry Arons, Joe Campbell, Oliver Jakobs, Sonja Kowalewski, Tony Robinson, "Frequently Asked Questions", Comp.Speech Newsgroup on Usenet, available on Internet via anonymous FTP from rtfm.mit.edu:/pub/usenet/news.answers/comp-speech-faq

[Dix79] N. Rex Dixon and Thomas B. Martin (eds), Automatic Speech & Speaker Recognition, IEEE Press, New York, New York, 1979.

[Dod76] George R. Doddington, "Personal Identity Verification Using Voice", pp 398-402, in [Dix79]

[Fur89] Sadaoki Furui, Digital Speech Processing, Synthesis, and Recognition, Marcel Dekker, New York, New York, 1989.

[Gro76] Stephen Grossberg, "Adaptive pattern classification and universal recoding, II: Feedback, expectation, oflaction, and illusions", Biological Cybernetics, No. 23, 1976, pp. 187-202

[Gro82] Stephen Grossberg, "Associative and Competitive Principles of Learning and Development - The Temporal Unfolding and Stability of STM and LTM Patterns", in Competition and Cooperation in Neural Nets, Springer-Verlag, New York, 1982, pp295 - 341.

[Har72] R.R.K. Hartmann and F.C. Stork, Dictionary Of Language And Linguistics, Applied Science Publishers Ltd, London, England, 1972.

[Lad75] Peter Ladefoged, A Course in Phonetics, Harcourt Brace Jovanovich Inc., New York, New York, 1975.

[Mar90] Alianna Maren, Craig Harston, and Robert Pap, Handbook of Neural Computing Applications, Academic Press, San Diego, California, 1990.

[Mil89] James D. Miller,"Auditory-perceptual interpretation of the vowel", J. Acoust. Soc. Am. 85(5), May 1989, pp 2114-2133

[Mur73] Gerald M. Murch, Visual And Auditory Perception, The Bobbs-Merrill Company, Indianapolis, Indiana, 1973, pp 105-110

[Sam76] Marvin R. Sambur, "Speaker Recognition Using Orthogonal Linear Prediction", section IV, page 407 in [Dix79]

[Whi76] George M. White, "Speech Recognition: A Tutorial Overview", pp 87-99 in [Dix79].

[You85] N.R. Young, A Speech Recognizer Based on Energy Zero Crossing and Linear Prediction, M.Eng. Thesis, Royal Military College of Canada, Dec 1985.

[Zad88] Lotfi A. Zadeh, "Fuzzy Logic", IEEE Computer, pp 83-93, Apr 1988.

[Zur92] Jacek M. Zurada, Introduction to Artificial Neural Systems, West Publishing, St. Paul, MN, 1992.

REFERENCES

[Ain88] W.A. Ainsworth, Speech Recognition By Machine, Peter Peregrinus Ltd, London, United Kingdom, 1988.

[Alw85] R.G. Alward, A Speech Recognizer Based on Energy and Zero Crossing, M.Eng. Thesis, Royal Military College of Canada, Jun 1985.

[Bar57] H.L. Barney and K.K. Dunn, “Speech Analysis”, pp 180-201 in Manual Of Phonetics , L. Kaiser (ed), North-Holland Publishing Co, Amsterdam, 1957.

[Car87a] Gail A. Carpenter and Stephen Grossberg, "A massively parallel architecture for a self-organizing neural pattern recognition machine”, Computer Vision Graphics and Image Processing, Vol.37, 1987, pp. 54-115.

[Car87b] Gail A. Carpenter and Stephen Grossberg, "ART 2 : Self-organization of stable category recognition codes for analog input patterns",Applied Optics, Dec. 1, 1987,
pp. 4919-4930

[Car88] Gail A. Carpenter and Stephen Grossberg, "The ART of Adaptive Pattern Recognition by a Self-Organizing Neural Network", IEEE Computer, pp 77-88, Mar 1988.

[Car90] Gail A. Carpenter and Stephen Grossberg,”ART3: Hierarchical search using chemical transmitters in self-organizing pattern recognition architectures”, Neural Networks, vol. 3, pp. 129-152, 1990

[Car91] Gail A. Carpenter, Stephen Grossberg, D.B. Rosen, “Fuzzy ART: Fast stable learning and categorization of analog patterns by an adaptive resonance system”, Neural Networks, vol. 4,
pp. 759-771, 1991.

[Car92] Gail A. Carpenter, Stephen Grossberg, John H. Reynolds, Natalya Markuzon and David B. Rosen, "Fuzzy ARTMAP: A Neural Network Architecture for Incremental Supervised Learning of Analog Multidimensional Maps", IEEE Trans. on Neural Networks, vol.3, pp 698-713, Sep 1992.

[Col95] Ron Cole et al.,"The Challenge of Spoken Language Systems: Research Directions for the Nineties", IEEE Trans. on Speech and Audio Processing, vol 3, Jan 1995

[CSN95] Andrew Hunt, Barry Arons, Joe Campbell, Oliver Jakobs, Sonja Kowalewski, Tony Robinson, "Frequently Asked Questions", Comp.Speech Newsgroup on Usenet, available on Internet via anonymous FTP from rtfm.mit.edu:/pub/usenet/news.answers/comp-speech-faq

[Dix79] N. Rex Dixon and Thomas B. Martin (eds), Automatic Speech & Speaker Recognition, IEEE Press, New York, New York, 1979.

[Dod76] George R. Doddington, "Personal Identity Verification Using Voice", pp 398-402, in [Dix79]

[Fur89] Sadaoki Furui, Digital Speech Processing, Synthesis, and Recognition, Marcel Dekker, New York, New York, 1989.

[Gro76] Stephen Grossberg, "Adaptive pattern classification and universal recoding, II: Feedback, expectation, oflaction, and illusions", Biological Cybernetics, No. 23, 1976, pp. 187-202

[Gro82] Stephen Grossberg, "Associative and Competitive Principles of Learning and Development - The Temporal Unfolding and Stability of STM and LTM Patterns", in Competition and Cooperation in Neural Nets, Springer-Verlag, New York, 1982, pp295 - 341.

[Har72] R.R.K. Hartmann and F.C. Stork, Dictionary Of Language And Linguistics, Applied Science Publishers Ltd, London, England, 1972.

[Lad75] Peter Ladefoged, A Course in Phonetics, Harcourt Brace Jovanovich Inc., New York, New York, 1975.

[Mar90] Alianna Maren, Craig Harston, and Robert Pap, Handbook of Neural Computing Applications, Academic Press, San Diego, California, 1990.

[Mil89] James D. Miller,"Auditory-perceptual interpretation of the vowel", J. Acoust. Soc. Am. 85(5), May 1989, pp 2114-2133

[Mur73] Gerald M. Murch, Visual And Auditory Perception, The Bobbs-Merrill Company, Indianapolis, Indiana, 1973, pp 105-110

[Sam76] Marvin R. Sambur, "Speaker Recognition Using Orthogonal Linear Prediction", section IV, page 407 in [Dix79]

[Whi76] George M. White, "Speech Recognition: A Tutorial Overview", pp 87-99 in [Dix79].

[You85] N.R. Young, A Speech Recognizer Based on Energy Zero Crossing and Linear Prediction, M.Eng. Thesis, Royal Military College of Canada, Dec 1985.

[Zad88] Lotfi A. Zadeh, "Fuzzy Logic", IEEE Computer, pp 83-93, Apr 1988.

[Zur92] Jacek M. Zurada, Introduction to Artificial Neural Systems, West Publishing, St. Paul, MN, 1992.

ANNEX A

ASR System Hardware and Software

The hardware consists of a desktop personal computer (AST Advantage! Pro with a 80486DX 33MHz processor, 430 MByte hard drive and 8MByte RAM), a sound card (Gravis Ultrasound Max capable of 16 bit 48 KHz stereo recording and playback), a high quality microphone (Realistic Super Cardiod Dynamic Microphone with 80 to 15000 Hz frequency response), a stereo tape deck (Realistic SCT-90 with metal bias recording option), a metal bias cassette tape (Sony Digital Excellence Metal SR type IV metal bias 70microsec EQ), and a home made microphone enclosure consisting of 3 cubic foot cardboard box lined with egg cartons.

The software used runs within MicroSoft's Windows For Workgroups version 3.11 running on top of MicroSoft's DOS version 6.2. Mathwork's Matlab for Windows version 4.0 was used to perform the speech signal feature extraction and simulation of the fuzzy ART Map neural network. Automated speech recording is performed using GoldWave version 2.11 controlled by a Windows script file program called Stdemo Player version 1.5. The compression of digitized speech files is accomplished using WazWave version 0.1. The ARJ compression program was used to make a backup achive copy of the database. The FORM1968 program was used to reformat floppy disks to a 1968KByte capacity.

Annex B

ASR Demo program code (*Stdemo Player 1.5*)

```
// ############################################################
// PROGRAM NOTES
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Program Name: VOXRECFL.STD
//
// Called By: STDEMO.EXE
//
// Purpose: STDemo script language file to control execution of
//              Windows applications for speech recognition. This
//            program performs FAST LEARNING (ie. no graphics are
//            displayed)
//
//
// Author: Captain David Peter Young
//
// ############################################################
// PROGRAM STARTUP ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// DISABLE LOCKOUTS - show cursor permanently and enable windows
// input permanently, without these two line the cursor is locked
// within the demo window.
:C11
:I11
//
// SPEECH RECOGNITION INTRO - play intro welcome and display graphics
:pwelcome.wav
//
//
// INPUT PHRASE INFO
:%SCC
Type in the phrase using CAPITAL LETTERS ...
1
%
// ############################################################
// VOICE SAMPLING ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Launch Gold Wave
```

```
:(goldwave.exe
//
// Remove help window with alt-f x
@fx
//
// ******* RECORD A SAMPLE OF THE BACKGROUND NOISE *****************
//
// Open a new file to record
@fn
// Select 16 bit MONO @ 22050 Hz for 1 sec
// Down arrow - Tab - Tab - Tab - Up Arrow - Up Arrow - Return
]|]~]~]~]^]^]!
//
// Ensure the "Device Controls" window window is active
:<Device Controls|0
//
// Notify user of noise sampling
// Hide STOP and CONTINUE buttons
:X0
:Y0
//
// Set I/O box timeout (time in seconds)
:+2
//
// Enlarge default font size to make message stand out
:d36
//
// Display Text
:#WCC
   SAMPLING BACKGROUND NOISE ...
#
//
// Return default setting
:X2
:Y2
:+0
:d12//
// Start recording
// two tabs are required for the first recording made after opening GoldWave
// Tab - Tab - Return
]~]~]!
//
// Save Noise Sample to NOISE.WAV
// Ensure the "GoldWave v2.11" window is active
:<GoldWave v2.11|0
//
```

```
// Save file in standard WAV format
// ALT-F S
@f
S
NOISE.WAV
]!
//
// If file already exists, confirm overwrite
// check for overwrite dialog window then answer yes
:?Save Sound File|AnswerYes
:GAnswerNA
:LAnswerYes
y
:LAnswerNA
//
// ****** RECORD VOICE SAMPLE **************
//
// Ensure focus on GoldWave window
:<GoldWave v2.11|0
//
// Open a new file to record
@f
n
// Select 16 bit MONO @ 22050 Hz for 5 sec
// Tab - Tab - Tab - Down Arrow - Return
]~]~]~]|]!
//
// Ensure the "Device Controls" window window is active
:<Device Controls|0
//
// Prompt speaker to give voice sample
:#WCC
After you press continue, recording will begin.
.N
You will have 5 seconds to state your phase.
#
//
// Start recording
// Tab - Tab - ShiftTab - Return
]~]~#]~]!
//
// Wait 5 seconds while recording is made
// required for synchronizing script to application
:W5
//
// Start playback - must alternate active hotspot with tab key
```

```
// to get return key to start player
// Tab - ShiftTab - Return
]~#]~]!
//
// Prompt speaker to resample again or continue with recognition
:LIsItGood
:*MRD
Is this the sample you want to submit for recognition?
1Resample|NO ...I want to make another recording.
2SaveSample|YES ..Recognize this voice sample.
*
//
// Resample voice
:LResample
// Prompt the speaker to give voice sample
:#WCC
RESAMPLING ...
.S
After you press continue, recording will begin.
.N
You will have 5 seconds to state your phase.
#
//
// Ensure the "Device Controls" window is active
:<Device Controls|0
//
// Start recording - only one tab required to change hotspot
// on second and subsequent recordings
// Tab - Return
]~]!
//
// Wait 5 seconds will recording is made
:W5
//
// Start playback - must alternate active hotspot with tab key
// to get return key to start player
// Tab - ShiftTab - Return
]~#]~]!
//
// Prompt speaker to accept sample
:GIsItGood
//
// Save the current sample in the buffer to file
:LSaveSample
//
// Ensure the "GoldWave v2.11" window is active
```

```
:<GoldWave v2.11|0
//
// Save file in standard WAV format
// ALT-F S
@f
S
DEMOSAMP.WAV
]!
//
// If file already exists, confirm overwrite
// check for overwrite dialog window then answer yes
:?Save Sound File|OverWrite
:GNoConfirm
:LOverWrite
y
:LNoConfirm
//
// DEMO SAMPLE IS NOW SAVED FOR ANALYSIS
// Close GoldWave Application
// Alt-F - x
@fx
//
// remove application from STDemo stack
:)
// ############################################################
// DEMO SAMPLE FEATURE EXTRACTION ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// LAUNCH MATLAB - version 4.0 for windows with Signal Processing,
// Neural Networks, and SPC Tools ToolBoxes
// Assume default MatLab directory
:(matlab.exe
//
// Start MatLab script file to extract features from
// speaker's voice sample (graphics off)
makespec('demosamp.wav',0); clear ]!
formant3('semosamp',0,1,'noise.wav'); clear ]!
//
// Wait 30 seconds for feature extraction
:W30
// Start MatLab script file to analyze features
fartmapb('femosamp',1, '$1' ,0); clear ]!
// Hold Prompt
:#WRD
Holding ... for results
#
```

```
//
// Close Matlab Window
:<MATLAB Command Window|0
@fx
//
// close MatLab on STDemo stack
:)
// ############################################################
//
// CLOSE PROGRAM ROUTINE
//
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Stop Label - this is jumped anytime STOP button is pushed
:L__STOP__
//
//
// GOODBYE MESSAGE - Display end of program message with
// short fuse timeout (ie. no button click required)
//
// Hide STOP and CONTINUE buttons
:X0
:Y0
//
// Set I/O box timeout (time in seconds)
:+5
//
// Enlarge default font size to make message stand out
:d36
//
// Play Good Bye Audio
:pGOODBYE.WAV
//
// Display Good Bye Text
:#WCC
END OF VOICE RECOGNITION
#
// Terminate script execution to invoke auto closure of open windows (if any)
:Z
// ==============
// END OF PROGRAM
// ==============
```

```
// ############################################################
// PROGRAM NOTES
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Program Name: VOXRECFT.STD
//
// Called By: STDEMO.EXE
//
// Purpose: STDemo script language file to control execution of
//              Windows applications for speech recognition. This
//            program performs FAST TESTING (ie. no graphics are
//            displayed)
//
//
// Author: Captain David Peter Young
//
// ############################################################
// PROGRAM STARTUP ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// DISABLE LOCKOUTS - show cursor permanently and enable windows
// input permanently, without these two line the cursor is locked
// within the demo window.
:C11
:I11
//
// SPEECH RECOGNITION INTRO - play intro welcome and display graphics
:pwelcome.wav
//
// ############################################################
// VOICE SAMPLING ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Launch Gold Wave
:(goldwave.exe
//
// Remove help window with alt-f x
@fx
//
// ******* RECORD A SAMPLE OF THE BACKGROUND NOISE **********
//
// Open a new file to record
@fn
// Select 16 bit MONO @ 22050 Hz for 1 sec
// Down arrow - Tab - Tab - Tab - Up Arrow - Up Arrow - Return
]|]~]~]~]^]^]!
```

```
//
// Ensure the "Device Controls" window window is active
:<Device Controls|0
//
// Notify user of noise sampling
// Hide STOP and CONTINUE buttons
:X0
:Y0
//
// Set I/O box timeout (time in seconds)
:+2
//
// Enlarge default font size to make message stand out
:d36
//
// Display Text
:#WCC
   SAMPLING BACKGROUND NOISE ...
#
//
// Return default setting
:X2
:Y2
:+0
:d12//
// Start recording
// two tabs are required for the first recording made after opening GoldWave
// Tab - Tab - Return
]~]~]!
//
// Save Noise Sample to NOISE.WAV
// Ensure the "GoldWave v2.11" window is active
:<GoldWave v2.11|0
//
// Save file in standard WAV format
// ALT-F S
@f
S
NOISE.WAV
]!
//
// If file already exists, confirm overwrite
// check for overwrite dialog window then answer yes
:?Save Sound File|AnswerYes
:GAnswerNA
:LAnswerYes
```

```
y
:LAnswerNA
//
// ****** RECORD VOICE SAMPLE **************
//
// Ensure focus on GoldWave window
:<GoldWave v2.11|0
//
// Open a new file to record
@f
n
// Select 16 bit MONO @ 22050 Hz for 5 sec
// Tab - Tab - Tab - Down Arrow - Return
]~]~]~]|]!
//
// Ensure the "Device Controls" window window is active
:<Device Controls|0
//
// Prompt speaker to give voice sample
:#WCC
After you press continue, recording will begin.
.N
You will have 5 seconds to state your phase.
#
//
// Start recording
// Tab - Tab - ShiftTab - Return
]~]~#]~]!
//
// Wait 5 seconds while recording is made
// required for synchronizing script to application
:W5
//
// Start playback - must alternate active hotspot with tab key
// to get return key to start player
// Tab - ShiftTab - Return
]~#]~]!
//
// Prompt speaker to resample again or continue with recognition
:LIsItGood
:*MRD
Is this the sample you want to submit for recognition?
1Resample|NO ...I want to make another recording.
2SaveSample|YES ..Recognize this voice sample.
*
//
```

```
// Resample voice
:LResample
// Prompt the speaker to give voice sample
:#WCC
RESAMPLING ...
.S
After you press continue, recording will begin.
.N
You will have 5 seconds to state your phase.
#
//
// Ensure the "Device Controls" window is active
:<Device Controls|0
//
// Start recording - only one tab required to change hotspot
// on second and subsequent recordings
// Tab - Return
]~]!
//
// Wait 5 seconds will recording is made
:W5
//
// Start playback - must alternate active hotspot with tab key
// to get return key to start player
// Tab - ShiftTab - Return
]~#]~]!
//
// Prompt speaker to accept sample
:GIsItGood
//
// Save the current sample in the buffer to file
:LSaveSample
//
// Ensure the "GoldWave v2.11" window is active
:<GoldWave v2.11|0
//
// Save file in standard WAV format
// ALT-F S
@f
S
DEMOSAMP.WAV
]!
//
// If file already exists, confirm overwrite
// check for overwrite dialog window then answer yes
:?Save Sound File|OverWrite
```

```
:GNoConfirm
:LOverWrite
y
:LNoConfirm
//
// DEMO SAMPLE IS NOW SAVED FOR ANALYSIS
// Close GoldWave Application
// Alt-F - x
@fx
//
// remove application from STDemo stack
:)
// ############################################################
// DEMO SAMPLE FEATURE EXTRACTION ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// LAUNCH MATLAB - version 4.0 for windows with Signal Processing,
// Neural Networks, and SPC Tools ToolBoxes
// Assume default MatLab directory
:(matlab.exe
//
// Start MatLab script file to extract features from
// speaker's voice sample (graphics off)
makespec('demosamp.wav',0); clear ]!
formant3('semosamp',0,1,'noise.wav'); clear ]!
//
// Wait 30 seconds for feature extraction
:W30
// Start MatLab script file to analyze features
fartmapb('femosamp',2,'test',0); clear ]!
// Hold Prompt
:#WRD
Holding ... for results
#
//
// Close Matlab Window
:<MATLAB Command Window|0
@fx
//
// close MatLab on STDemo stack
:)
// ############################################################
// CLOSE PROGRAM ROUTINE
// ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
//
// Stop Label - this is jumped anytime STOP button is pushed
```

```
:L__STOP__
//
//
// GOODBYE MESSAGE - Display end of program message with
// short fuse timeout (ie. no button click required)
//
// Hide STOP and CONTINUE buttons
:X0
:Y0
//
// Set I/O box timeout (time in seconds)
:+5
//
// Enlarge default font size to make message stand out
:d36
//
// Play Good Bye Audio
:pGOODBYE.WAV
//
// Display Good Bye Text
:#WCC
END OF VOICE RECOGNITION
#
// Terminate script execution to invoke auto closure of open windows (if any)
:Z
// ==============
// END OF PROGRAM
// ==============
```

Annex C

MAKESPEC.M Program Code (*Matlab 4.0*)

```
function makespec(WaveFile,PlotFlag)

% MAKE ME A SPECTROGRAM!!
%
% Usage: makespec('Wave.File',PlotFlag) [PlotFlag=1 displays specgram chunks]
%
% Note- This algorithm builds a spectrogram by segmenting the sound file into manageable
%       lengths and processing each piece sequentially. Each chunk of the spectrogram is
%       concatenated together to form one big spectrogram representing the whole sound file.
%       In this manner, long sound files can be processed with limited RAM.

% SET PLOTS TO WHITE BACKGROUND FOR PRINTING SCREEN SHOTS
close;close;close;close;close;close;close;close;close;close;close;
whitebg;

%LOAD WAVE DATA
[WaveData,Fs]=loadwav(WaveFile);

% CONSTANTS
NFFT=512;                   % Number of samples in FFT window frame
% TimeRes=0.0116;           % Time resolution (in seconds) of each spectrogram timeslice
% NOverLap=(NFFT-round(Fs*TimeRes)); % Number of samples in frame overlap required
to produce desired time resolution
NOverLap=0.5*NFFT;
```

```
FreqRes=Fs/NFFT;            % Bandwidth resolution of FFT frequency bands (specgram
rows)
MaxFreq=4000;               % Ignore frequencies above this level
HighBand=ceil(MaxFreq/FreqRes); % The specgram row representing the maximum
frequency


% MAXIMIZE AMPLITUDE OF WAVE FILE
% 16 bit resolution gives 2^16 amplitude levels, 2^15 positive and 2^15 negative
Boost_Ratio = 2^15/max(abs(WaveData));
WaveData = WaveData * Boost_Ratio;

% DEFINE NUMBER OF SAMPLES TO BE PROCESSED IN EACH CHUNK
% the memory constant in the equation should be adjusted according to how much matlab
%memory is available
%MemoryConstant=100;
%ChunkSize=round(Fs*TimeRes*MemoryConstant);
ChunkSize=Fs; % 1 second worth of samples

% DETERMINE HOW MANY CHUNKS THERE ARE IN THE SOUND FILE
NumberChunks = ceil(length(WaveData)/ChunkSize);


%#########################################################################
%
% BIG LOOP - begin producing spectrograms one chunk at a time
%
%~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

for chunk=1:NumberChunks  %
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
```

```
CHUNK=int2str(chunk);

% LOAD NEXT 1 SECOND OF SPEAKER'S VOICE SAMPLE
if chunk < NumberChunks
      ChunkData=WaveData((1+(chunk-1)*ChunkSize):(chunk*ChunkSize));
else
      ChunkData=WaveData((1+(chunk-1)*ChunkSize):length(WaveData));
      % If last chunk then load to End Of File

   % check length of last chunk to prevent a crash - if chunk is too small skip it
   if length(ChunkData) < NFFT
     break;
   end;
end; % of if chunk


% CALCULATE REAL PART OF VOICE CHUNK SPECGRAM
ChunkSpec=abs(specgram(ChunkData,NFFT,NOverLap));

% KEEP ONLY THE BOTTOM HALF OF THE SPECTROGRAM UP TO MAX
FREQUENCY
ChunkSpec=ChunkSpec(1:HighBand,:);

if PlotFlag==1
      % PLOT SPECGRAM CHUNK
      figure(chunk);
      clf reset;
      [SpecRows,SpecCols]=size(ChunkSpec);
      TimeIndex=(1:SpecCols)*NOverLap/Fs;
      FreqIndex=(1:SpecRows)*Fs/NFFT;
      imagesc(TimeIndex,FreqIndex,ChunkSpec); axis('xy')
```

```
        title(['Chunk #',CHUNK,' of ',WaveFile]);
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');

% CONVERT SCALED COLOUR IMAGE TO INVERTED GRAYSCALE COLOUR %
MAPPING TO ACCOMODATE PRINT OUTS
        colormap(gray);
        graymap=colormap;
        colormap(1-graymap);
        pause(3);

   end;

   % SAVE SPECGRAM CHUNK TO FILE
   % If this is the first chunk, then create a new file otherwise append it to other chunks in
   % existing file
   if chunk==1
        Spec = ChunkSpec;
        save tempfile.mat Spec ;
   else
        load tempfile.mat;
        Spec = [Spec,ChunkSpec];
        save tempfile.mat Spec ;
   end; % if chunk==1

end; % for chunk=1:NumberChunks
<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<

if PlotFlag == 1
        figure(chunk+1)
        clf reset;
```

```
        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec); axis('xy')
        title(['Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');

% CONVERT SCALED COLOUR IMAGE TO INVERTED GRAYSCALE COLOUR
% MAPPING TO ACCOMODATE PRINT OUTS
        colormap(gray);
        graymap=colormap;
        colormap(1-graymap);
        pause(3);

end;

% SAVE SPECTROGRAM TO "S" FILE
save tempfile Spec NFFT Fs WaveFile Boost_Ratio
clear
load tempfile
save(['s',WaveFile(2:length(WaveFile)-4),'.mat'])
fprintf('\nSpectrogram saved to %s \n',['s',WaveFile(2:length(WaveFile)-4),'.mat'])
```

Annex D

FORMANT3.M Program Code (*Matlab 4.0*)

```
function formant3(SpecFile,PlotFlag,NRFlag,NoiseFile)

% YOUNG FORMANT EXTRACTOR
%
% PROGRAM OVERVIEW:
%
% USAGE: formant3(SpecFile,PlotFlag,NRFlag,NoiseFile)
%
% Plot Flag is used to control the level of screen "visualization"
% PlotFlag==2 will plot 9 figures
% PlotFlag==1 will plot 1 figure (the most important) of filtered speech formants
% PlotFlag==0 plots nothing (fastest execution)
%
% NRFlag==1 performs noise reduction of the spectrogram using the wav file "NoiseFile"
%       and is highly recommended for steady state background noise (ie. hard disk RPMs
etc)

% TURN ON WHITE BACKGROUND FOR SCREEN SHOTS
% ensure all figures are closed first
close; close; close; close; close; close; close; close; close; close; close;
whitebg;

%##########################################################################
%       PART 1 - LOAD SPECTROGRAM
%##########################################################################
```

```
load(SpecFile);
[SpecRows,SpecCols]=size(Spec);

%~~~~~~~~~~~~~~ GLOBAL CONSTANTS ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

NFFT=512;                   % Number of samples in FFT window frame
NOverLap=0.5*NFFT;
FreqRes=Fs/NFFT;    % Bandwidth resolution of FFT frequency bands (specgram rows)
MinFreq=300;                % Ignore frequencies below this level
MaxFreq=3300;               % Ignore frequencies above this level
                            % Eg. Telephone bandwidth 300-3300 Hz
% The specgram row representing the miminum frequency
LowBand=ceil(MinFreq/FreqRes);
% The specgram row representing the maximum frequency
HighBand=ceil(MaxFreq/FreqRes);
% The number of formants to be extracted from the signal
MaxFormants=3;
% The Half Formant Bandwidth in frequency (Hz)
HFBFreq=150;
% The Half Formant Bandwidth in specgram rows
HFBRows=ceil(HFBFreq/FreqRes);

%~~~~~~~~~~~~~~ GLOBAL CONSTANTS ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

% PLOT THE SPECTROGRAM OF JUST LOADED
if PlotFlag > 1
        figure(1)
        clf reset
        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
```

```
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec); axis('xy')
        title(['Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');
        colormap(1-gray);
      pause(3);
end;


if NRFlag > 0
  % PRODUCE A NOISE REDUCTION MASK
  [NOISE]=loadwav(NoiseFile); % background noise recorded from live session
  MaxNoise=max(abs(specgram(NOISE*Boost_Ratio,NFFT))');
  % max of transpose specgram matrix gives freq row maximums
  MaxNoise=MaxNoise'; % convert to a column vector
  MaxNoise=MaxNoise(1:SpecRows,1); % crop to same number of rows as spectrogram
  if PlotFlag > 1
        % PLOT MAXNOISE
        figure(2)
        clf reset;
        TimeIndex=[0 1];
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,MaxNoise); axis('xy')
        title(['Max Spectral Amplitudes of ',NoiseFile]);
        ylabel('Frequency (Hz)');
        colormap(1-gray);
        pause(3);
  end; % if PlotFlag

  for TimeIndex=1:SpecCols
```

```
        Spec(:,TimeIndex)=Spec(:,TimeIndex)-MaxNoise;
  end;

  % ELIMINATE NEGATIVE NUMBERS FROM SPECTROGRAM
  Spec=(Spec+abs(Spec))/2;

  if PlotFlag > 1
        figure(3)
        clf reset;
        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec); axis('xy')
        title(['Noise Masked Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');
        colormap(1-gray);
        pause(3);
  end;
end; % if NoiseReduction

%##########################################################################
%
% PART 2 - PROCESS SPECTROGRAM FOR VOICED PHONEMES IN UTTERANCE
%
%##########################################################################

% DETERMINE SHORT TERM ENERGY OF SPEC COLUMNS WITHIN THE GIVEN
% FREQUENCY RANGE
STEnergy=(sum(Spec(LowBand:HighBand,:)));
```

```
% PLOT SHORT TERM ENERGY CURVE ON SPECTROGRAM (SCALED TO MAX
% FREQUENCY)
if PlotFlag > 1
        figure(4)
        clf reset;

        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec(1:HighBand,:)); axis('xy')
        title(['Relative STE Curve of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');
        colormap(1-gray);

        % Plot STE curve overlay
        hold on;
        plot(TimeIndex,max(FreqIndex)*STEnergy/max(STEnergy),'k')
        hold off

        % mark the minimum and maximum frequency band stops
        hold on;
        plot(TimeIndex,MinFreq*ones(1,SpecCols),'k');
        plot(TimeIndex,MaxFreq*ones(1,SpecCols),'k');
        hold off;
        pause(3);
end;

% REMOVE PORTIONS OF LOW ENERGY (SILENCE AND FRICATIVES)
% using 2nd frequency row as a reference
Index=find(HighBand*STEnergy/max(STEnergy) > 2);
```

```
% keep only the high energy portions
Spec=Spec(:,Index);
% determine new size of spectrogram
[SpecRows,SpecCols]=size(Spec);
% keep only the high energy portions
STEnergy=STEnergy(Index);
LengthSTE=length(STEnergy);

% PLOT SPECTROGRAM WITH LOW ENERGY SECTIONS REMOVED
if PlotFlag > 1
        figure(5)
        clf reset;
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec(1:HighBand,:)); axis('xy')
        title(['Silence Reduced Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');
        colormap(1-gray);

        % Plot STE curve overlay
        hold on;
        plot(TimeIndex,max(FreqIndex)*STEnergy/max(STEnergy),'k')
        hold off

        % mark the minimum and maximum frequency band stops
        hold on;
        plot(TimeIndex,MinFreq*ones(1,SpecCols),'k');
        plot(TimeIndex,MaxFreq*ones(1,SpecCols),'k');
        hold off;
```

```
        pause(3);
end;


% APPLY OCTAVE GAIN TO EMPHASIZE HIGHER FREQUENCY BANDS
for row=1:SpecRows
        Gain(row,1)=10^(log2(row)/5); %
end;
for TimeIndex=1:SpecCols
        Spec(:,TimeIndex)=Spec(:,TimeIndex).*Gain;
end;

if PlotFlag > 1
        % PLOT SPECTROGRAM TO SCREEN
        figure(6);
        clf reset;

        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec(1:HighBand,:)); axis('xy')
        title(['Octave Enhanced Spectrogram of ',WaveFile]) ;
        xlabel('Time (sec)');
        ylabel('Frequency (Hz)');
        colormap(1-gray);

        % mark the minimum and maximum frequency band stops
        hold on;
        plot(TimeIndex,MinFreq*ones(1,SpecCols),'k');
        plot(TimeIndex,MaxFreq*ones(1,SpecCols),'k');
        hold off;
```

```
        pause(3);
end;
%##########################################################################
%
%       PART 3 - EXTRACT FORMANTS
%
%##########################################################################

% PERFORM INITIALIZATIONS BASED ON SIZE OF SPECTROGRAM
Peak=zeros(SpecRows,SpecCols);
Formant=zeros(SpecRows,SpecCols);


% DETERMINE SPECTRAL PEAKS FOR EACH TIME SLICE

for TimeIndex=1:SpecCols

  PeakNumber=0;

  % SAVE FREQUENCY BIN NUMBER OF LOCAL PEAKS
  % (max values within Formant Bandwidth)
  for FreqBand=LowBand:HighBand;
    LowFB=FreqBand-HFBRows;
    if LowFB < 1
        LowFB=1;
    end;

    HighFB=FreqBand+HFBRows;
    if HighFB > SpecRows
        HighFB=SpecCols;
```

```
    end;

    FBMax=max(Spec(LowFB:HighFB,TimeIndex)); % the max value within the window

    if Spec(FreqBand,TimeIndex) == FBMax & FBMax ~= 0
        PeakNumber=PeakNumber+1;
        % record the frequency band of the local peak
        PeakFrequency(PeakNumber,TimeIndex)=FreqBand;
        % record its amplitude
        PeakAmplitude(PeakNumber,TimeIndex)=Spec(FreqBand,TimeIndex);
        % keep track of the total number of peaks in a given time slice
        NumberOfPeaks(TimeIndex)=PeakNumber;
    end;
  end; % for FreqBand

  % SINCE ARRAY WAS NOT INITIALIZED, TIME INDEX LENGTH IS VARIABLE
  % SINCE PEAKS MAY NOT BE PRESENT AT THE  END OF THE
 % SPECTROGRAM. IN THIS CASE, INSERT DUMMY VALUES AS PLACE
  % HOLDERS TO PROVIDE UNIFORM ARRAY LENGTHS
  if PeakNumber==0
        PeakFrequency(1,TimeIndex)=1;
        PeakAmplitude(1,TimeIndex)=1;
        NumberOfPeaks(TimeIndex)=0;
  end;
end; % for TimeIndex


% DEFINE FORMANTS AS PEAKS WITH GREATEST AMPLITUDE / NUMBERED
% FROM LOW TO HIGH FREQUENCY
for TimeIndex=1:SpecCols
        % SORT AMPLITUDES (ascending order)
```

```
        [SortedAmplitudes,PeakNumbers]=sort(PeakAmplitude(:,TimeIndex));
        % change the index to descending order
        PeakNumbers=flipud(PeakNumbers);

        % SELECT THE ONES WITH GREATEST AMPLITUDES
        if NumberOfPeaks(TimeIndex) < MaxFormants
                % pick only non-zero elements
                MaxPeaks=PeakNumbers(1:NumberOfPeaks(TimeIndex));
        else
                % pick the biggest ones in the index
                MaxPeaks=PeakNumbers(1:MaxFormants);
        end; % if

% LABEL THE FORMANTS IN ORDER OF PEAK NUMBER (LOWEST TO HIGEST
% FREQUENCY BAND)
        SortedMaxPeaks=sort(MaxPeaks);
        for FormantNumber=1:length(MaxPeaks)
        Formant(FormantNumber,TimeIndex)=
                PeakFrequency(SortedMaxPeaks(FormantNumber),TimeIndex);
        end;

end; % for TimeIndex

if PlotFlag > 1
        % PLOT SPECTROGRAM TO SCREEN
        figure(7);
        clf reset;

        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
```

```
        imagesc(TimeIndex,FreqIndex,Spec(1:HighBand,:)); axis('xy')
        title(['Local Spectral Maxima of ',WaveFile]) ;
        xlabel('(Symbol Key)    . = PEAK    X = F1    O = F2    * = F3');
        ylabel('Frequency (Hz)');
        colormap(1-gray);

        % mark the minimum and maximum frequency band stops
        hold on;
        plot(TimeIndex,MinFreq*ones(1,SpecCols),'k');
        plot(TimeIndex,MaxFreq*ones(1,SpecCols),'k');

        % PLOT PEAKS AND ROUGH DRAFT OF FORMANTS ON SPECTROGAM
        plot(TimeIndex,PeakFrequency'*FreqRes-(FreqRes/2),'.k')
        plot(TimeIndex,Formant(1,:)'*FreqRes-(FreqRes/2),'xk')
        plot(TimeIndex,Formant(2,:)'*FreqRes-(FreqRes/2),'ok')
        plot(TimeIndex,Formant(3,:)'*FreqRes-(FreqRes/2),'*k')

        hold off;
        pause(3);
end;


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%   FILTER OUT SECTIONS WITH HIGH FIRST FORMANT
%   OR NO THIRD FORMANT
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% FIND ALL ZERO VALUED THIRD FORMANTS AND SET ALL THE FORMANTS
% FOR THAT TIME SLICE TO ZERO
ZeroIndex=find(Formant(3,:) == 0);
for TimeIndex=1:length(ZeroIndex)
```

```
   for FormantNumber=1:MaxFormants
        Formant(FormantNumber,ZeroIndex(TimeIndex))=0;
   end;
end;

% FIND ALL FIRST FORMANTS GREATER THAN 1200 HZ AND ZERO OUT THE
TIMESLICES
ZeroIndex=find(Formant(1,:) > ceil(1200/FreqRes));
for TimeIndex=1:length(ZeroIndex)
   for FormantNumber=1:MaxFormants
        Formant(FormantNumber,ZeroIndex(TimeIndex))=0;
   end;
end;


%%%%%%%%%%%%%%%%% PLOT FILTERED FORMANTS
if PlotFlag > 1
        figure(8);
        clf reset;
        [SpecRows,SpecCols]=size(Spec);
        TimeIndex=(1:SpecCols)*NOverLap/Fs;
        FreqIndex=(1:SpecRows)*Fs/NFFT;
        imagesc(TimeIndex,FreqIndex,Spec(1:HighBand,:)); axis('xy')
        title(['Probable Formants of ',WaveFile]) ;
        xlabel('(Symbol Key)    X = F1    O = F2    * = F3');
        ylabel('Frequency (Hz)');
        colormap(1-gray);
        % mark the minimum and maximum frequency band stops
        hold on;
        plot(TimeIndex,MinFreq*ones(1,SpecCols),'k');
        plot(TimeIndex,MaxFreq*ones(1,SpecCols),'k');
```

```
        % plot filtered formants
        plot(TimeIndex,Formant(1,:)'*FreqRes-(FreqRes/2),'xk')
        plot(TimeIndex,Formant(2,:)'*FreqRes-(FreqRes/2),'ok')
        plot(TimeIndex,Formant(3,:)'*FreqRes-(FreqRes/2),'*k')

        hold off;
        pause(3);
end;

%%%% KEEP ONLY THOSE FORMANT TIMESLICES WITH NON-ZERO VALUES
Formant=Formant(1:MaxFormants,find(Formant(3,:) > 0));
TimeIndex=(1:length(Formant))*NOverLap/Fs;


%%%%%%%%%%%%%% CONVERT FORMANT VALUES TO FREQUENCY
Formant=Formant*FreqRes-(FreqRes/2);

if PlotFlag > 0
        % PLOT FORMANTS ON FREQUENCY SCALE FOR INSPECTION
        figure(9)
        clf reset;
        plot(TimeIndex,Formant(1,:)','k');
        title(['Formant Vectors from ',WaveFile]);
        xlabel('(Symbol Key)    X = F1    O = F2    * = F3');
        ylabel('Frequency (Hz)');
        hold on;
        plot(TimeIndex,Formant(2,:)','k');
        plot(TimeIndex,Formant(3,:)','k');
        plot(TimeIndex,Formant(1,:)','kx');
        plot(TimeIndex,Formant(2,:)','ko');
```

```
        plot(TimeIndex,Formant(3,:)','k*');
        hold off;
        grid on;

        pause(3);
end;

%##########################################################################
%
%       PART 5 - SAVE RESULTS TO FILE
%
%##########################################################################

% SAVE FEATURE VECTOR IN FORMANT "F" FILE

save tempfile Formant WaveFile
clear
load tempfile
save(['f',WaveFile(2:length(WaveFile)-4),'.mat'])
fprintf('Processing Completed !! \nResults stored in %s \n',
['f',WaveFile(2:length(WaveFile)-4),'.mat']);
```

Annex E

FARTMAPT.M Program Code (*Matlab 4.0*)

```
function fartmapt(FormFile,MODE,SIV,PlotFlag)

%          A FUZZY ART MAP NEURAL NETWORK
%               SPEECH RECOGNITION SYSTEM
%                         BASED ON
%                    FORMANT RATIOS
%
%                          by
%         Capt D.P. Young, CD, B.Sc., RRMC
%                   Canadian Forces
%
% (c) Copyrigth by D.P. Young 1995
% This program may be used within the Department of National Defence
% but copyright for open use remains the property of the author.
%
%
% Usage fartmapt(FormFile,MODE,SIV,PlotFlag)
%
%   FormFile = string name of file containing extracted formant vectors
%MODE = 1 for training
%          = 2 for testing
%     SIV = Signal Identification Vector (used to map fuzzy ART categories to a given
phrase)
%           = 999 for autodetect from filename
% PlotFlag = 0 for no graphics
```

```
%            = 1 for graphics

% TURN ON WHITE BACKGROUND FOR SCREEN SHOTS
% ensure all figures are closed first
close; close; close; close; close; close; close; close; close; close; close;
whitebg;

%~~~~~~~~~~~~~~~~~~~~~~~~~ GLOBAL CONSTANTS ~~~~~~~~~~~~~~~~~~~~~~~~~
if MODE==1
  vigilance=1.00; % This is the baseline training vigilance
else
  vigilance=0.50; % This is the baseline testing vigilance
end;
% The choice function uses this non-zero positive parameter in its denominator to
% break the tie between competitive nodes
Choice_Parameter = 0.001;


NewClusterFlag = 0; % This flag is used to signal the creation of a new cluster

% Code number used to label unrecognized input vectors. A positive number is used
UNRECOGNIZED=999;
 % since it must serve later as an vector index (no negative indexes allowed)
 % Warning: Ensure this number is not overrun by a real cluster number due to
 % cluster expansion(explosion?).

StandardSIVLength=80; % Max length of SIV string, used to pad with blanks f
%~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
% BATCH MODE SIV ASSIGNMENT BY FILE NAME
%~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
```

```
% The SIV arguement is set to 999 when the program is to automatically construct a SIV
code
% based on the file name of the formant file.

if SIV(1) == 999
      BatchFlag=1;
         PHRASE=FormFile(8);
         if PHRASE=='0' SIV='DATE AND TIME';
         elseif PHRASE=='1' SIV='NAME';
         elseif PHRASE=='2' SIV='SHE HAD YOUR DARK SUIT IN GREASY WASH
WATER ALL YEAR';
         elseif PHRASE=='3' SIV='DON^T MAKE ME CARRY AN OILY RAG LIKE
THAT';
         elseif PHRASE=='4' SIV='MAY WE ALL LEARN A YELLOW LION ROAR';
         elseif PHRASE=='5' SIV='WE WERE AWAY A YEAR AGO';
         elseif PHRASE=='6' SIV='I KNOW WHEN MY LAWYER IS DUE';
         elseif PHRASE=='7' SIV='EVERY SALT BREEZE COMES FROM THE SEA';
         elseif PHRASE=='8' SIV='OUR YACHT SLID AROUND THE POINT INTO THE
BAY';
         elseif PHRASE=='9' SIV='I WAS STUNNED BY THE BEAUTY OF THE VIEW';
         else SIV='BATCH FILE NAME ERROR';
         end; %if PHRASE
  else
      BatchFlag=0;
end; %if SIV

SIV=[SIV,blanks(StandardSIVLength-length(SIV))];
% pad SIV string to standard SIV length with blanks

FileNameSIV=SIV;
% store phrase identifier from filename for automatic error detection using database files
```

```
%#########################################################################
% PERFORM FEATURE VECTOR REPRESENTATION PRIOR TO NETWORK INPUT
%#########################################################################
% Load formant vectors from FormFile

load(FormFile);

% Calculate the Tri Formant Ratio Vector (TFRV) representation

F3F2=(Formant(3,:)./Formant(2,:));
F2F1=(Formant(2,:)./Formant(1,:));
F3F1=(Formant(3,:)./Formant(1,:));
TFRV=[F3F2,F2F1,F3F1];

% Convert the TFRV to Normalized Polar Angle (NPA) form

NPA_TFRV=atan(TFRV)/(0.5*pi);

% Linear Time Warp (LTW) the Normalized TFRV to standard network input length

StandardLength=500;
LengthTFRV=length(TFRV);

for Index=1:StandardLength-1
   LTW_NPA_TFRV(Index)=NPA_TFRV(ceil(LengthTFRV/StandardLength*Index));
end;

LTW_NPA_TFRV(StandardLength)=NPA_TFRV(LengthTFRV);
% this is a work around a bug in the ceil function
```

```
if PlotFlag > 0
        % Plot the feature vector
        figure(1);
        subplot(3,1,1);
        plot(LTW_NPA_TFRV,'k');
        axis([0 StandardLength 0 1]); % a standard axis aids in visual comparison
        title(['Tri Formant Ratio Vector from ',WaveFile]);
        ylabel('NPA Ratio');
        grid on;
end;

% Initialize starting weights:
% Load Network Weight Matrix file if it exists or initialize from scratch
% NOTE: floppy drives can be used for removeable media

FID = fopen('fartmapt.mat'); % (FID = -1 if it doesn't exist)
if FID == -1
   fprintf(' \n INITIALIZING Fuzzy ART Map Network Memory Matrix ! \n');
   M=2*StandardLength;  % The input dimension of the network
% It is TWICE THE SIZE OF THE FEATURE VECTOR due to complement coding
   N=10;
% N is the number of outputs from the network.
% It defines the number of classifications defineable - matrix is initialized with 10 outputs
% MORE OUTPUTS ARE ADDED AS DEMANDED BY PROGRAM NEEDS
   W = ones(N,M);
% W is the feedforward/feedback weight matrix between the input and output layers
   max_cluster_number = 0;
% keeps track of the highest committed cluster
   SIM=ones(N,length(SIV));
% Signal Identification Matrix with length(SIV) outputs and N category inputs
```

```
% NOTE** This convention is currently the reverse of that used in W weight matrix
% Recode the W matix to (input#,output#) standard at a later date (reduce code)
else
   fclose(FID);
   load('fartmapt.mat');
   [N,M]=size(W);
end;

%#########################################################################
% FUZZY ART SELF ORGANIZATION
%#########################################################################

% Calculate complement coding input vector..Note that the dimension has doubled.
I = [LTW_NPA_TFRV,1-LTW_NPA_TFRV];
% it is assumed that the feature vector is a row vector

% Calculate choice function Tj(I) for each output node
% To save time, PERFORM CHOICE FUNCTION ONLY FOR CATEGORY NODES
% AND ONE UNCOMMITTED NODE
for j = 1:(max_cluster_number+1)
   T(j) =  sum( min(W(j,:),I) ) / ( Choice_Parameter + sum(W(j,:)) );
end; % for j

% Perform "Hypothesis" testing, in descending order of Choice Function output magnitude
% until a match is made that meets the required vigilance, and meets the Match Tracking
% requirements

active_nodes = (max_cluster_number+1);
% identify number of nodes in competition (other outputs are spares)

MatchTracking=1; % set flag
```

```
while MatchTracking==1

 cluster_search = 1; % turn flag on
 while cluster_search == 1,

   % find node with maximum output
   % IF TWO NODES BOTH HAVE MAX OUTPUTS THEN THE NODE WITH THE
   % LOWEST INDEX GETS CHOSEN , max function does this automatically
   [maxnum,max_index] = max(T);

   % Vigilance (similarity) test

   similarity = sum( min(W(max_index,:),I) ) / sum(I);

   if (similarity < vigilance) & (active_nodes > 1)

      % RESET: Disable Node and look for next T(j)max

      T(max_index) = 0; % deactivate winning node by setting its choice function to zero
      active_nodes = active_nodes - 1; % decrement active node counter

    else % RESONANCE: stop search and record cluster number

        cluster=max_index;
      cluster_search = 0; % set flag to end cluster search, exit from while loop

  end; % if similarity
end; % while cluster_search

if MODE==1
    % >>>>>>>>> LEARNING MODE - MATCH TRACKING PROCESS <<<<<<<<
```

```
    % if the cluster output achieved by the resonance (above) is not mapped to the desired
    % SIV code in the Signal Identification Matrix (SIM) then the vigilance parameter is
    % increase from the baseline value to a value just above the similarity ratio (in order to
    % induce reset) and the "Hypothesis" testing is restarted from where it left off

    if SIM(cluster,:)==SIV    % Check for SIV match and break out of while loop
        MatchTracking=0;
    elseif max_index > max_cluster_number  % Check for new cluster, and assign SIV to
SIM
      SIM(cluster,:)=SIV;
      MatchTracking=0; % Break out of while loop
    else
        % Boost vigilance just above similarity
        vigilance = 1.00000000000001*similarity;
       % Matlab displays accuracy to 14 decimal places
    end; %if SIM
  else
    MatchTracking=0; % Only perform Match Tracking in learning mode
  end; % if MODE==1

end; % While MatchTracking==1


% LEARNING PHASE:     FAST LEARNING = NO RECODING OF CATEGORY
% WEIGHTS AFTER INITIAL FORMATION ... A new category is formed solely around a
% new input vector which did not fit in any other category

if max_index > max_cluster_number  % current max_index is a new cluster
   if MODE ==1
        % Update flags, counters and weights
        NewClusterFlag = 1;
```

```
        max_cluster_number = max_index;
        fprintf('\nNew Cluster Formed = %i \n',max_cluster_number);
      W(max_index,:) = I; % Because the learning rate is 1 all new clusters are centered on the founding vector
    else
      % MODE 2 RECOGNITION ONLY - NO LEARNING ALLOWED
      fprintf('\n UNRECOGNIZED FEATURE !!! \n');
      cluster=UNRECOGNIZED; % assign the feature vector to the "UNRECOGNIZED" cluster instead of creating a new cluster
    end; % if MODE
end; % if max_index

% If in test mode, then Identify the voice sample
if max_index <= max_cluster_number
  if MODE==2
      SIV=SIM(cluster,:);

% AUTO ERROR DETECTION FOR DATBASE FILES -
% compare SIV of matching pattern to that of input file name
      if (BatchFlag == 1) & (strcmp(SIV,FileNameSIV)==0)
            fprintf('WRONG PHRASE IDENTIFIED || ');
        end;

        % PRINT OUT MATCHING VECTOR INFO
      fprintf('%5.2f Percent || %s \n',100*similarity,SIV);

        % PLOT THE MATCHING MEMORY VECTOR FOR COMPARISON
        if PlotFlag > 0
              figure(1);
              subplot(3,1,2);
              plot(W(cluster,:),'k');
```

```
            axis([0 StandardLength 0 1]); % a standard axis aids in visual comparison
            title('Fuzzy ART Map Stimulus Response');
            ylabel('NPA Ratio');
            grid on;


            % OVERLAP THE TWO VECTORS
            subplot(3,1,3)
            plot(LTW_NPA_TFRV,'k')
            axis([0 StandardLength 0 1]); % a standard axis aids in visual comparison
            title([num2str(100*similarity),'% Match to ',deblank(SIV)]);
            xlabel('Linearly Warped Time Scale')
            ylabel('NPA Ratio');
            hold on
            plot(W(cluster,:),'k');
            hold off
            grid on;
        end;

   end; % if MODE==2
end; % if max_index

%#########################################################################
% NEW IDEA -
% allow weight matrices to grow when it gets low on uncommitted output classes
% ADD WEIGHTS FOR TWENTY MORE CATEGORIES
  if (N - max_cluster_number) < 2
     N=N+20 ;
     fprintf('\n >>>>>>>>>>> Caution - Network Memory is GROWING!!! <<<<<<<<');
     W=[W;ones(20,M)];
     % Expand Signal Identification Matrix to accomodate 20 more input categories
```

```
    SIM=[SIM;ones(20,length(SIV))];
  end;
%
%#########################################################################

% PLOT FINAL WEIGHTS IF THEY HAVE CHANGED DUE TO NEW
% CLUSTER FORMATION
if NewClusterFlag == 1 & PlotFlag > 0
   figure(2);
   imagesc(W');
   axis('xy');
   title('Fuzzy ART Map Memory');
   xlabel('Long Term Memory (LTM) Patterns');
end;

% SAVE NETWORK WEIGHTS AND NUMBER OF LAST COMMITTED OUTPUT
% NODE
if NewClusterFlag==1
        save fartmapt.mat W max_cluster_number SIM
end;

% PRINT END OF PROGRAM
fprintf('=========================================================\n');
```

Annex F

BPMAPPER.M Program Code (*Matlab 4.0*)

```
function bpmapper(FARTMAPFile)
% BPMapper - Back Propogation Neural Network
%
% This program inputs the Fuzzy ART Map LTM traces (W Matrix) as training vectors and
% the Signal Identification Matix (SIM) as target vectors.
% This network utilizes Nguyen-Widrow initial conditions, adaptive learning rates and
% momentum with the back-propogation learning rule in an attempt to map the inputs to the
% desired outputs.

help bpmapper

% Load input and target vectors
load(FARTMAPFile);
% where W is the weight file created by FARTMAPT.M which is already normalized to
% the fuzzy values and includes ones complement coding and where SIM is the Signal
% Identification Matrix created by FARTMAPT.M


% remove the ones complement component to reduce input vector length
[Wrows,Wcols]=size(W);
P=W(:,1:Wcols/2);
%convert to column vectors for compatibility with the BP algorithm below
P=P';

% Convert target vectors from text to numerical format (local binary representation)
[SIMrows,SIMcols]=size(SIM);
```

```
T=zeros(SIMrows,8);

for rownum=1:SIMrows

        PHRASE=(SIM(rownum,1:5));

           if PHRASE=='SHE H' T(rownum,:)=[1 0 0 0 0 0 0 0];
        elseif PHRASE=='DON^T' T(rownum,:)=[0 1 0 0 0 0 0 0];
        elseif PHRASE=='MAY W' T(rownum,:)=[0 0 1 0 0 0 0 0];
        elseif PHRASE=='WE WE' T(rownum,:)=[0 0 0 1 0 0 0 0];
        elseif PHRASE=='I KNO' T(rownum,:)=[0 0 0 0 1 0 0 0];
        elseif PHRASE=='EVERY' T(rownum,:)=[0 0 0 0 0 1 0 0];
        elseif PHRASE=='OUR Y' T(rownum,:)=[0 0 0 0 0 0 1 0];
        elseif PHRASE=='I WAS' T(rownum,:)=[0 0 0 0 0 0 0 1];
        else   T(rownum,:)=[0 0 0 0 0 0 0 0]; % in case of blank entries in the SIM Matrix
        end; %if PHRASE
end;

% convert T to column vector format
T=T';

% INITIALIZE NETWORK ARCHITECTURE
%================================
% Set input vector size R, layer sizes S1 & S2, batch size Q.
[R,Q] = size(P);
S1 = 50;
[S2,Q] = size(T);

% Initialize weights and biases.
%z = menu('Initialize Weights & Bias To', ...
%  'Stored Values in BPMAPPER.MAT', ...
```

```
%  'Random Nguyen-Widrow Values [Default]');
%disp('')
%if z == 1
        load('bpinit.mat');
%else
%       [W10,B10] = nwtan(S1,R);
%       W20 = rands(S2,S1)*0.5;
%       B20 = rands(S2,1)*0.5;
% save bpinit.mat W10 W20 B10 B20
%end


% TRAIN THE NETWORK
%==================

% TRAINING PARAMETERS

df = 10;   % Display Frequency
me = 900; % Maximum Epochs to calculate
eg = 0.1; % Error Goal
lr = 0.0001; % Learning Rate
im = 1.05; % learning rate IncreMent
dm = 0.7;  % learning rate DecreMent
mc = 0.95; % Momentum Constant
er = 1.04; % Error Ratio

% NETWORK PARAMETERS

load('bprtmapt.mat');
%W1 = W10;
%B1 = B10;
```

```
%W2 = W20;
%B2 = B20;

dW1 = W1*0;
dB1 = B1*0;
dW2 = W2*0;
dB2 = B2*0;
MC = 0;

% PRESENTATION PHASE

A1 = tansig(W1*P,B1);
A2 = purelin(W2*A1,B2);
E = T-A2;
SSE = sumsqr(E);

% TRAINING RECORD
TR = zeros(2,me);
SSE0 = SSE;
lr0 = lr;

% BACKPROPAGATION PHASE
D2 = deltalin(A2,E);
D1 = deltatan(A1,D2,W2);

figure(1)        % Create a new figure to plot the learning process (and print later)
clf reset
hold on

for epoch=1:me
```

```
% CHECK PHASE

if SSE < eg
        epoch=epoch-1;
        break;
end;

% LEARNING PHASE
[dW1,dB1] = learnbpm(P,D1,lr,MC,dW1,dB1);
[dW2,dB2] = learnbpm(A1,D2,lr,MC,dW2,dB2);
MC = mc;
TW1 = W1 + dW1; TB1 = B1 + dB1;
TW2 = W2 + dW2; TB2 = B2 + dB2;

% PRESENTATION PHASE
TA1 = tansig(TW1*P,TB1);
TA2 = purelin(TW2*TA1,TB2);
TE = T-TA2;
TSSE = sumsqr(TE);

% MOMENTUM & ADAPTIVE LEARNING RATE PHASE
if TSSE > SSE*er
  lr = lr * dm;
  MC = 0;
else
  if TSSE < SSE
    lr = lr * im;
  end

  W1=TW1; B1=TB1; W2=TW2; B2=TB2;
  A1=TA1; A2=TA2; E=TE; SSE=TSSE;
```

```
    % BACKPROPAGATION PHASE
    D2 = deltalin(A2,E);
    D1 = deltatan(A1,D2,W2);
  end

  % TRAINING RECORD
  TR(1,epoch) = SSE;
  TR(2,epoch) = lr;

  % DISPLAY RESULTS
  if rem(epoch,df) == 0
    plottr([[SSE0; lr0] TR(:,1:epoch)], ...
      '2-Layer Backpropagation with Adaptive LR & Momentum')
  end
  fprintf('\nEPOCH=%i SSE=%f LR=%f \n',epoch,SSE,lr);
end


% RETURN RESULTS
TR = [[SSE0; lr0] TR(:,1:epoch)];


% PLOT LEARNING RATES
%====================
figure(2)
clf reset
hold on

plotlr(TR(2,:));
pause(3)
```

```
% PLOT ERROR CURVE
%================
figure(3)
clf reset
hold on

ploterr(TR(1,:));
pause(3)

% PLOT ERRORS ASSOCIATED WITH EACH OUTPUT VECTOR
%===============================================
figure(4)
clf reset
hold on

barerr(A2,T);
pause(3)

% SUMMARIZE RESULTS
%==================
SSE = sumsqr(T-purelin(W2*tansig(W1*P,B1),B2));

diary trgdiary.txt
fprintf('Trained for %.0f epochs.\n',epoch)
fprintf('Training took %.0f flops.\n',flops);
fprintf('Sum squared error goal was %g.\n',eg);
fprintf('Final sum squared error is %g.\n',SSE);
diary off

fprintf('\nSAVING FINAL NETWORK VALUES:\n')
```

```
save tempfile W1 B1 W2 B2 FARTMAPFile
clear
load tempfile
save(['bp',FARTMAPFile(3:length(FARTMAPFile)-4),'.mat'])
```

Annex G

BPTESTER.M Program Code (*Matlab 4.0*)

```
function bptester(FormFile,WeightFile,PlotFlag)

% BPTester - Back Propogation Neural Network
%
% This program inputs a formant file created by FORMANT3.M and a weight file created by
% BPMapper.M to attempt to recognize the phrase.



StandardBFLength=500;
% The length the feature vector is time warped to fit for network input

%#########################################################################
% PERFORM FEATURE VECTOR REPRESENTATION PRIOR TO NETWORK INPUT
%#########################################################################

% Load formant vectors from FormFile

load(FormFile);



% Calculate the Tri Formant Ratio Vector (TFRV) representation

F3F2=(Formant(3,:)./Formant(2,:));
F2F1=(Formant(2,:)./Formant(1,:));
F3F1=(Formant(3,:)./Formant(1,:));
TFRV=[F3F2,F2F1,F3F1];
```

```
% Convert the TFRV to Normalized Polar Angle (NPA) form

NPA_TFRV=atan(TFRV)/(0.5*pi);


% Linear Time Warp (LTW) the Normalized TFRV to standard network input length

StandardLength=500;
LengthTFRV=length(TFRV);

for Index=1:StandardLength-1
   LTW_NPA_TFRV(Index)=NPA_TFRV(ceil(LengthTFRV/StandardLength*Index));
end;

LTW_NPA_TFRV(StandardLength)=NPA_TFRV(LengthTFRV);
% this is a work around a bug in the ceil function


if PlotFlag > 0
        % Plot the feature vector
        figure(1);
        subplot(3,1,1);
        plot(LTW_NPA_TFRV,'k');
        axis([0 StandardLength 0 1]); % a standard axis aids in visual comparison
        title(['Tri Formant Ratio Vector from ',WaveFile]);
        ylabel('NPA Ratio');
        grid on;
end;
```

```
% Assign feature vector to test vector P
P=LTW_NPA_TFRV;
P=P'; %convert to column vectors for compatibility with the BP algorithm below

% INITIALIZE NETWORK ARCHITECTURE
%================================
load(WeightFile);



% PRESENTATION PHASE
A1 = tansig(W1*P,B1);
A2 = purelin(W2*A1,B2);

% MAXNET - identify the node with maximum output
[MaxValue,MaxNode]=max(A2);

PHRASE=MaxNode+1; % an offset to make the node # coincide with the phrase number

if PHRASE==2 SIV='SHE HAD YOUR DARK SUIT IN GREASY WASH WATER ALL
YEAR';
elseif PHRASE==3 SIV='DON^T MAKE ME CARRY AN OILY RAG LIKE THAT';
elseif PHRASE==4 SIV='MAY WE ALL LEARN A YELLOW LION ROAR';
elseif PHRASE==5 SIV='WE WERE AWAY A YEAR AGO';
elseif PHRASE==6 SIV='I KNOW WHEN MY LAWYER IS DUE';
elseif PHRASE==7 SIV='EVERY SALT BREEZE COMES FROM THE SEA';
elseif PHRASE==8 SIV='OUR YACHT SLID AROUND THE POINT INTO THE BAY';
elseif PHRASE==9 SIV='I WAS STUNNED BY THE BEAUTY OF THE VIEW';
end;

if PHRASE==str2num(FormFile(8))
   fprintf('\n%s',SIV);
```

```
else
   fprintf('\n WRONG PHRASE = %s',SIV);
end;
fprintf('\n#########################################################################\n');
```